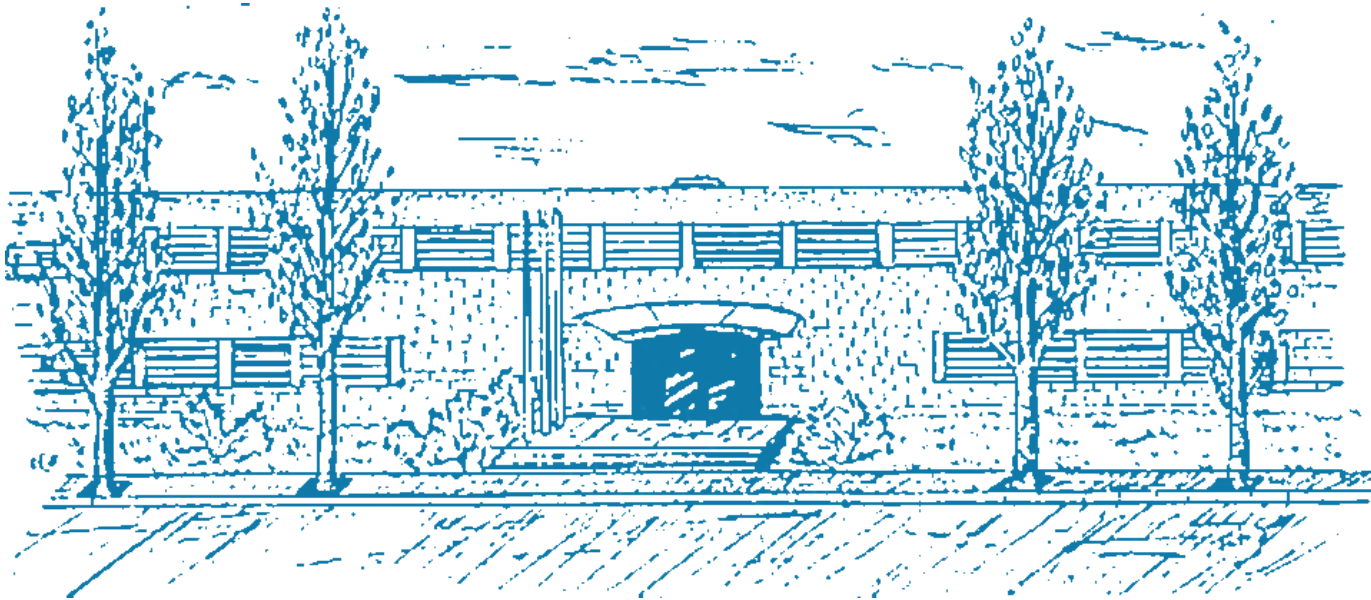


UNIVERSITAT POLITÈCNICA DE CATALUNYA - BarcelonaTech (UPC)
FME - FACULTAT DE MATEMÀTIQUES I ESTADÍSTICA

MASTER'S DEGREE IN STATISTICS AND OPERATIONS RESEARCH (MESIO UPC-UB)
MASTER'S THESIS

TRAFFIC SIMULATION WITH IRRATIONAL DRIVERS



Author: Miguel Ángel Donado Fernández

Director: Dr. Manuel Molano Mazón

Codirector: Dr. María Paz Linares Herreros

Departments: Department of Mathematics

Department of Statistics and Operations Research

Barcelona, Sep 20, 2026



UNIVERSITAT DE
BARCELONA



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH
Facultat de Matemàtiques i Estadística



Abstract

Most traffic simulations assume that drivers make perfectly rational decisions, always choosing the fastest route, despite evidence from research on human decision-making showing that people are often influenced by biases and simplified decision rules. Traffic simulators account for external factors such as weather and calendar effects, but individual route-choice decisions are frequently simplified and overlooked. This thesis bridges traffic simulation and driver behavior modeling by studying how more realistic, imperfect driver behavior affects traffic patterns.

The work begins with a baseline of rational drivers, trained using the Bush–Mosteller reinforcement learning algorithm within SUMO (Simulation of Urban MObility), an open-source traffic simulator. Drivers learn to choose between available routes based on their previous travel experiences. This baseline converges to Dynamic User Equilibrium (DUE), a benchmark state in which no driver can reduce their travel time by unilaterally switching routes.

The remainder of the thesis examines how deviations from rationality affect driver behavior. It first starts examining the two behavioral parameters already included by the original Bush–Mosteller algorithm: memory, how much drivers weigh recent versus older experience, and learning rate, how quickly they update their beliefs. These parameters are varied across a range of values. Since the model is not calibrated against real-world data, this analysis shows how traffic patterns change for different parameter values without identifying which values best represent real drivers. The original Bush–Mosteller model is then extended by introducing three behavioral mechanisms into the decision-making process: aversion to unreliable travel times (risk aversion), aversion to time spent stopped (waiting-time aversion), and a nonlinear response to perceived travel-time differences, in which drivers do not react to differences below a certain threshold but respond increasingly strongly to larger differences.

Overall, the results show that variations in memory level and learning rate do not prevent convergence to DUE under the tested scenario. However, in more complex network and traffic-demand configurations, the influence of memory may become strong enough to prevent convergence to DUE. The introduction of additional behavioral mechanisms, including waiting-time sensitivity, risk aversion, and nonlinear response, can prevent convergence towards DUE. These results show that incorporating more realistic behavioral mechanisms into drivers’ decision-making can result in final network states further from DUE, although the effect depends on the behavioral parameters and the network and traffic-demand configuration.

Resumen

La mayoría de las simulaciones de tráfico asumen que los conductores toman decisiones perfectamente racionales y que siempre eligen la ruta más rápida, a pesar de las evidencias procedentes de la ciencia cognitiva que muestran que las decisiones humanas suelen estar influidas por sesgos y heurísticas. Los simuladores de tráfico tienen en cuenta factores externos, como las condiciones meteorológicas y los efectos del calendario, pero las decisiones individuales sobre la elección de ruta suelen simplificarse y pasarse por alto. Esta tesis conecta la simulación del tráfico con el modelado del comportamiento mediante el estudio de cómo un comportamiento de los conductores más realista e imperfecto afecta a los patrones de tráfico.

El trabajo comienza con un modelo de referencia compuesto por conductores racionales, entrenados mediante el algoritmo de aprendizaje por refuerzo de Bush–Mosteller dentro de SUMO (Simulation of Urban MObility), un simulador de tráfico de código abierto. Este modelo de referencia converge hacia el Equilibrio Dinámico del Usuario (DUE, Dynamic User Equilibrium), un estado de referencia en la asignación del tráfico en el que ningún conductor puede reducir unilateralmente su tiempo de viaje cambiando de ruta.

El resto de la tesis examina cómo las desviaciones respecto a la racionalidad afectan al comportamiento de los conductores. En primer lugar, se estudian los dos parámetros de comportamiento que ya están incluidos en el algoritmo original de Bush–Mosteller: la memoria, es decir, cuánto peso dan los agentes a las experiencias recientes frente a las más antiguas, y la tasa de aprendizaje, que determina la rapidez con la que actualizan sus creencias. Estos parámetros se varían dentro de un rango de valores. Dado que el modelo no está calibrado con datos del mundo real, este análisis muestra cómo cambian los patrones de tráfico para diferentes valores de los parámetros, sin identificar qué valores representan mejor a los conductores reales.

Posteriormente, el modelo original de Bush–Mosteller se amplía mediante la introducción de tres mecanismos de comportamiento en el proceso de toma de decisiones: aversión a los tiempos de viaje poco fiables (aversión al riesgo), aversión al tiempo pasado detenido (aversión al tiempo de espera) y una respuesta no lineal a las diferencias percibidas en los tiempos de viaje.

En general, los resultados muestran que las variaciones en el nivel de memoria y en la tasa de aprendizaje no impiden la convergencia hacia el DUE en el escenario analizado. Sin embargo, en configuraciones más complejas de red y demanda, la influencia de la memoria puede llegar a ser suficientemente fuerte como para impedir la convergencia hacia el DUE. La introducción de mecanismos de comportamiento adicionales, como la sensibilidad al tiempo de espera, la aversión al riesgo y la respuesta no lineal, también puede impedir la convergencia hacia el DUE. Estos resultados muestran que incorporar mecanismos de comportamiento más realistas en la toma de decisiones de los agentes puede dar lugar a estados finales de la red

más alejados del DUE, aunque el efecto depende de los parámetros de comportamiento y de la configuración de la red y la demanda.

Resum

La majoria de les simulacions de trànsit assumeixen que els conductors prenen decisions perfectament racionals i que sempre escullen la ruta més ràpida, malgrat les evidències procedents de la ciència cognitiva que mostren que les decisions humanes sovint estan influïdes per biaixos i heurístiques. Els simuladors de trànsit tenen en compte factors externs, com ara el temps meteorològic i els efectes del calendari, però les decisions individuals sobre l'elecció de la ruta sovint es simplifiquen i es deixen de banda. Aquesta tesi connecta la simulació del trànsit amb el modelatge del comportament mitjançant l'estudi de com un comportament dels conductors més realista i imperfecte afecta els patrons de trànsit.

El treball comença amb un model de referència format per conductors racionals, entrenats mitjançant l'algoritme d'aprenentatge per reforç de Bush–Mosteller dins de SUMO (Simulation of Urban MObility), un simulador de trànsit de codi obert. Aquest model de referència convergeix cap a l'Equilibri Dinàmic de l'Usuari (DUE, Dynamic User Equilibrium), un estat de referència en l'assignació del trànsit en què cap conductor pot reduir unilateralment el seu temps de viatge canviant de ruta.

La resta de la tesi examina com les desviacions respecte de la racionalitat afecten el comportament dels conductors. En primer lloc, estudia els dos paràmetres de comportament que ja estan inclosos en l'algoritme original de Bush–Mosteller: la memòria, és a dir, fins a quin punt els agents donen més pes a les experiències recents respecte de les més antigues, i la taxa d'aprenentatge, que determina la rapidesa amb què actualitzen les seves creences. Aquests paràmetres es varien dins d'un rang de valors. Com que el model no està calibrat amb dades del món real, aquesta anàlisi mostra com canvien els patrons de trànsit per a diferents valors dels paràmetres, sense identificar quins valors representen millor els conductors reals.

Posteriorment, el model original de Bush–Mosteller s'amplia mitjançant la introducció de tres mecanismes de comportament en el procés de presa de decisions: aversió als temps de viatge poc fiables (aversió al risc), aversió al temps passat aturat (aversió al temps d'espera) i una resposta no lineal a les diferències percebudes en els temps de viatge.

En general, els resultats mostren que les variacions en el nivell de memòria i en la taxa d'aprenentatge no impedeixen la convergència cap al DUE en l'escenari analitzat. Tanmateix, en configuracions més complexes de xarxa i demanda, la influència de la memòria pot arribar a ser prou important com per impedir la convergència cap al DUE. La introducció de mecanismes de comportament addicionals, com ara la sensibilitat al temps d'espera, l'aversió al risc i la resposta no lineal, també pot impedir la convergència cap al DUE. Aquests resultats mostren que incorporar mecanismes de comportament més realistes en la presa de decisions dels agents pot donar lloc a estats finals de la xarxa més allunyats del DUE, tot i que l'efecte depèn dels paràmetres de comportament i de la configuració de la xarxa i la demanda.

To God.

To my parents and my sisters.

To Mari Paz and Manuel Molano.

To those who encouraged me to pursue this master's degree.

To David Goggins, Kobe Bryant, and other role models who helped me change my inner dialogue. As Goggins said, "Never pick the easy road."

Contents

1. Introduction	11
1.1. Background	11
1.2. Motivation and Thesis Objectives	16
1.3. Thesis Contributions	17
1.4. Thesis Outline	18
2. Literature Review	19
2.1. Conventional Route Choice Models	19
2.2. Reinforcement Learning Route Choice Models	21
2.3. Synthesis and Research Gap	25
3. Traffic Modeling and Simulation	29
3.1. Traffic Assignment	29
3.2. Network Representation	33
3.3. Traffic Simulation	33
3.4. SUMO	35
3.5. Bounded Rationality	37
4. Reinforcement Learning	40
4.1. Bush-Mosteller Reinforcement Learning	40
4.2. Bush-Mosteller Model for Day-to-Day Route Choice	42
4.3. Bounded-Rationality Extensions	44
5. Experimental Design and Evaluation	50
5.1. Implementation	50
5.2. Randomness and Reproducibility	56
5.3. Experimental Design	57
5.4. Experimental Evaluation	65
6. Conclusion and Future Work	99
7. Declaration of AI Use	102
References	103
Appendix A. Derivation of the corrected probability update of the non selected routes	107
Appendix B. Justification of the Risk-Averse and Waiting-Time-Averse Formulations	109
Appendix C. Software Implementation	111

Appendix D. SA and Non-SA Hyperparameters	117
1. SA hyperparameters	117
1.1. min_distance_factor	117
1.2. random_factor	119
1.3. n_routes_per_od	121
1.4. threshold_density	123
1.5. warm_up_time	123
1.6. max_od_space	124
1.7. congestion_regimes	126
1.8. fixed_time_min	127
2. Non-SA hyperparameters	130
Appendix E. Additional Analyses	133
0.1. Heterogeneous Memory Levels	133
0.2. Spatial Traffic Comparison	135
Appendix F. SA hyperparameters Experimental Configuration	139
Appendix G. Research Questions Experimental Configurations	141
1. Rational Agents	141
2. Spatial Traffic Comparison	142
3. Memory Sensitivity	144
4. Disruption Recovery	145
5. Heterogeneous Agents	147
6. Learning Rate Sensitivity	149
7. Risk aversion	150
8. Waiting-time aversion	152
9. Nonlinear Response	153

1. Introduction

This section begins with a brief introduction to the fields of traffic assignment and bounded rationality, followed by a brief outline of the thesis.

1.1. Background

As the number of megacities continues to grow and more people live in urban areas, numerous challenges arise, one of the most important being traffic congestion. Congestion results in longer travel times, increased fuel consumption, and higher emissions.

Figure 1 shows the increasing trend in the number of people living in urban areas relative to rural areas.

Figure 2 shows how the number of cities and the share of population living in them have also increased since 1975. In particular, the number of megacities with more than 10 million inhabitants has increased from 8 to 33. The TomTom Traffic Index illustrates the extent of congestion across cities, with many major cities experiencing substantial levels of congestion [46]. Congestion can have considerable consequences. For example, a study conducted in the United States estimated that, in 2014, traffic congestion resulted in \$160 billion in economic losses, corresponding to 6.9 billion hours of additional travel time and 3.1 billion gallons of wasted fuel [39].

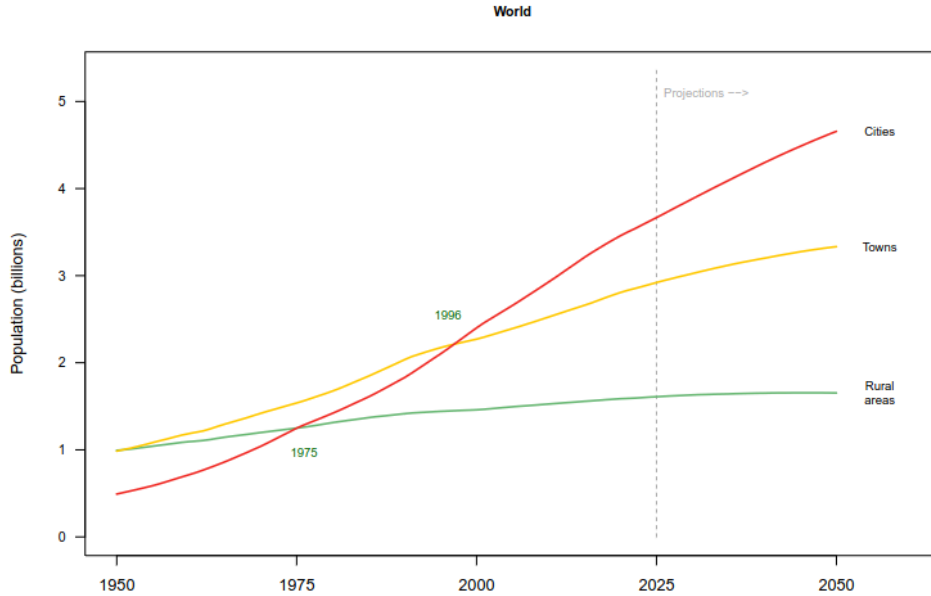


FIG. 1. World population living in cities, towns and rural areas, estimates, 1950–2025, and projections, 2025–2050. Reproduced from [16]

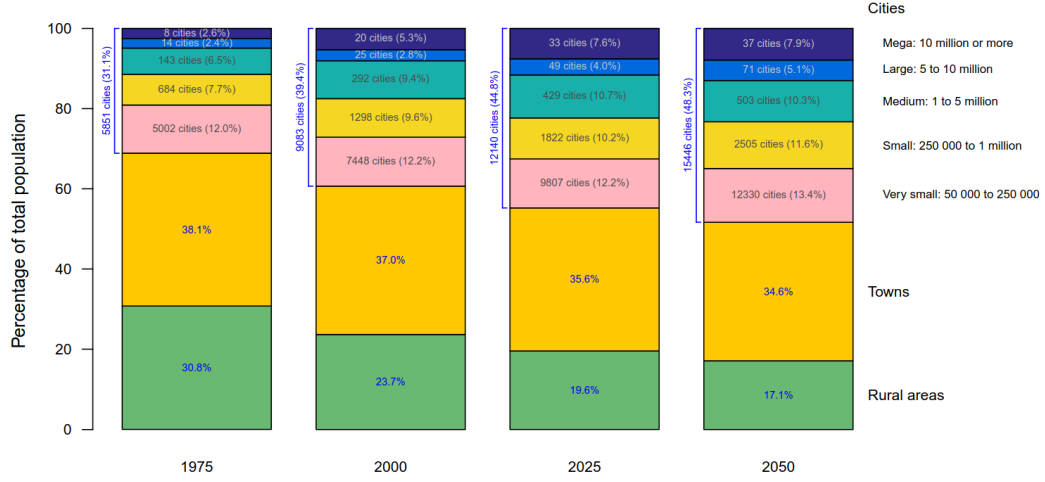


FIG. 2. World population distribution by City Size Class, estimates for 1975, 2000 and 2025 and projections to 2050. Reproduced from [16]

In this context, *transportation planning* becomes essential for addressing traffic congestion, as it helps ensure that transportation resources and policies are used efficiently and effectively. To support this process, models are used to predict the impact of potential changes in the transportation system, such as reducing the speed limit on a road or constructing a roundabout. The effects of these changes on congestion, emissions, and toll revenues are examples of factors that may be considered when evaluating these changes.

Transportation networks represent a complex system, where many factors play a role in determining the traffic dynamics of an area. Hence mathematical models are used to represent this complex system in a simplified way that can be unambiguously analyzed. As George Box famously stated: “All models are wrong, but some models are useful.”

A useful model is one which provides enough insight that good decisions can be made. To do this, a model must capture the most important characteristics of the underlying system; must not require more input data than what is available for calibration; and must not require more time and memory than what available hardware permits. [8]

To evaluate the impact of a change in the transportation system, it is typically necessary to estimate how many vehicles will use each road segment, which is called *link flow*, since most measures of interest, such as congestion, emissions, or toll revenue, can be derived from this quantity once it is known.

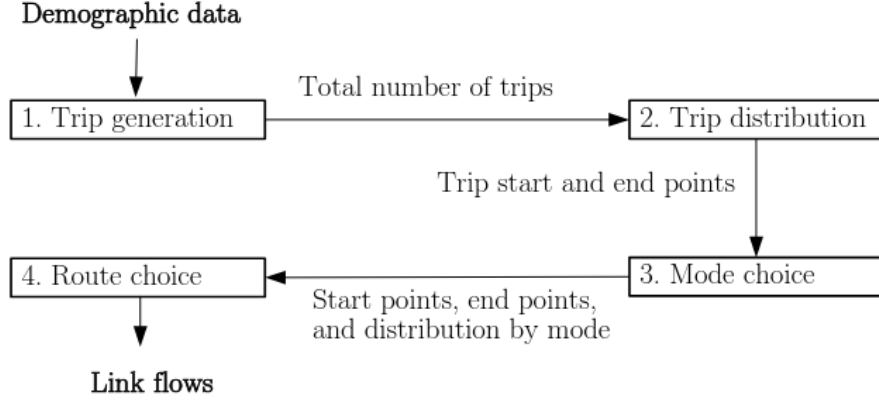


FIG. 3. Transportation planning four-step model. Reproduced from [8]

Link flows are traditionally obtained through what is known as the *four-step model*, shown in Figure 3. Starting from demographic data, the model first estimates how many trips will be generated (trip generation), then determines where those trips begin and end (trip distribution), and next assigns each trip to a mode of transport such as car, bus, or bicycle (mode choice). The final step, *route choice* — also referred to as *traffic assignment* — determines which specific paths travelers take through the network, and subsequently link flows are obtained. This thesis focuses exclusively on the traffic assignment of private cars.

Traffic assignment models represent the interaction between two main components: first, a *route choice model*, representing the *demand side*, which determines the routes chosen by travelers, and a *traffic-flow model*, representing the *supply side*, which determines the resulting congestion and travel times given the route choices and the characteristics of the transportation network. The resulting travel times then influence the route choices, creating a feedback loop between the two components. Figure 4 illustrates this interaction between the route-choice and traffic-flow models.

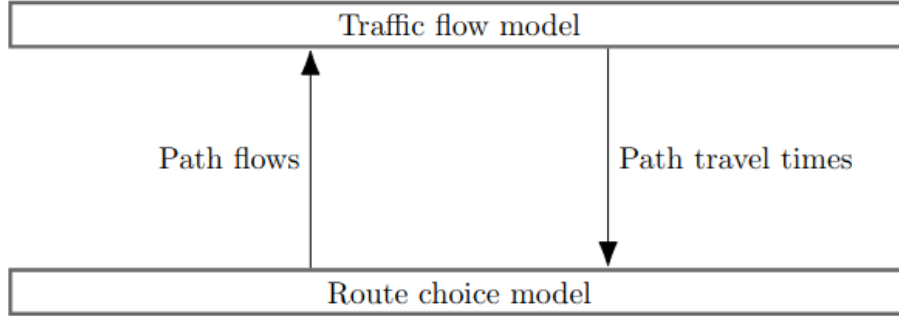


FIG. 4. Traffic assignment model as interacting systems. Reproduced from [8]

Figure 5 shows the traffic assignment model implemented in this thesis, which uses the Simulation of Urban MObility (SUMO) as its traffic flow model. Given the path flows, SUMO simulates vehicle interactions on the network and returns the resulting path travel times. The route-choice model, in turn, is implemented by the reinforcement learning agents, where each agent represents a driver and is responsible for selecting a route through the network. The agents use the travel times obtained from SUMO to update their route choices and generate the path flows for the next simulation. This process is repeated iteratively, forming the feedback loop between the two components.

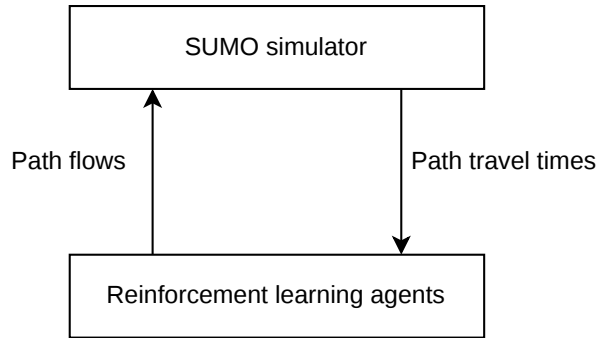


FIG. 5. Traffic assignment model used in the thesis

Traffic assignment models can be broadly classified as static or dynamic. The main distinction lies in the traffic flow model they use. *Static traffic assignment (STA)* represents the network in a steady state, in which every vehicle on a given link experiences the same travel time regardless of when it entered the link. *Dynamic traffic assignment (DTA)* models, by contrast, allow travel times on a link to vary over the simulated period, capturing the fact that congestion, and consequently

the travel times experienced by vehicles, change over time. This thesis implements a DTA model.

This distinction can be illustrated with a simple example. Consider a simulation of a single day in which vehicles travel from origin A to destination B via path 1. In an STA model, all vehicles using path 1 are assigned the same travel time, regardless of when they depart. In a DTA model, by contrast, the travel time of path 1 can vary throughout the day, so vehicles departing during the morning peak may experience a different travel time from those using the same path during off-peak periods. DTA therefore provides a more realistic representation of traffic conditions by accounting for their variation over time.

To make predictions, transportation planners seek a consistent solution known as an *equilibrium*. In general, an equilibrium is a state in which route choices and the resulting traffic conditions are mutually consistent. This means that the process shown in Figure 4 reaches a fixed point, at which further iterations produced the same state: the traffic conditions resulting from the chosen routes are the same conditions that determine those route choices. Different notions of equilibrium exist depending on how route choices are assumed to be made; the rule that governs these route choices is called the assignment rule and maps path travel times to route choices.

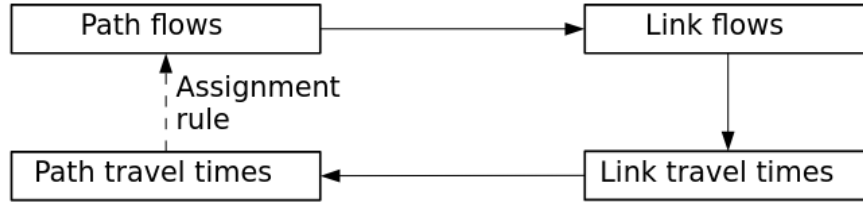


FIG. 6. Traffic assignment as an iterative process. Adapted from [8]

One of the most common assignment rules it is to assume that travelers independently seek to minimize their own travel time. In addition, travelers are assumed to have perfect knowledge of network conditions. Under this assignment rule and the assumption of perfect knowledge, the resulting equilibrium is known as *user equilibrium (UE)* in the static case. At UE, all paths used between the same origin and destination have equal and minimal travel times; otherwise, travelers would have an incentive to switch from slower to faster paths. In dynamic traffic assignment (DTA), congestion and travel times vary over time, meaning that the fastest path between an origin and destination may depend on the departure time. The corresponding equilibrium concept is dynamic user equilibrium (DUE), where travellers with the same origin and destination departing at the same time have no incentive to change routes because all used paths have the same, minimum travel time.

A fundamental problem underlying route choice is the *shortest-path problem*, which consists of finding the path between an origin and destination that minimises a given path cost, typically travel time. In the context of traffic assignment, this problem provides the basis for determining the path with the minimum travel time under the assumption of perfect knowledge and rational route choice.

In practice, however, travellers do not have perfect knowledge of network conditions and may not always choose the path with the minimum travel time, as human decision-making is often constrained by limitations in the information available, cognitive capacity, and the costs associated with evaluating alternatives [12]. Consequently, models based on perfect rationality may result in inaccurate estimates and predictions. *Bounded rationality* provides an alternative framework by explicitly accounting for limitations in human decision-making, including cognitive constraints, deliberation costs, habits, and myopia. To model both rational and boundedly rational drivers, reinforcement learning is particularly appropriate.

1.2. Motivation and Thesis Objectives

As explained in the previous section, traffic congestion has significant economic, environmental, and social consequences, and is expected to worsen in the coming years. To study its consequences and to evaluate interventions before committing to them in the real world, traffic simulations are widely used. For these simulations to be useful, the behavior they produce must resemble what would actually happen in reality; otherwise, conclusions drawn from the simulation cannot be reliably transferred to real-world traffic systems.

One characteristic that most traffic simulations leave out is that human decision-making is not fully rational — research in cognitive science has shown it is systematically biased and relies on heuristics.

Hence, with the aim of making simulations closer to reality, this thesis focuses on the decision-making process of drivers, also referred to as route choice. The objective is to model drivers whose route-choice decisions resemble those of real humans rather than idealised rational agents, and to examine how these decisions affect the traffic patterns produced by the simulation.

This thesis is guided by the following primary research questions:

- **(Rational Agent)** Does a multi-agent reinforcement learning approach, following the Bush-Mosteller learning rule, converge toward a dynamic user equilibrium (DUE) when applied within a microscopic traffic simulation environment?
- **(Memory Sensitivity)** Does reducing agents' memory level, how quickly older experiences lose weight relative to more recent ones when forming perceptions, prevent the model from converging toward a DUE state, and how does it affect route-choice behavior?

- **(Learning-Rate Sensitivity)** Does reducing agents' learning rate, how quickly they update their beliefs in response to new experience, prevent the model from converging toward a DUE state, and how does it affect route-choice behavior?
- **(Disruption Recovery)** If a link is temporarily disrupted and then restored, does the model recover the same route-choice equilibrium it had reached before the disruption?
- **(Risk Aversion)** How does incorporating travel-time variability into a route's perceived cost, making agents sensitive to risk, not just to mean travel time, affect route-choice behavior and convergence toward a DUE state?
- **(Waiting-Time Aversion)** How does incorporating time spent stopped into a route's perceived cost, making agents sensitive to waiting, not just to mean travel time, affect route-choice behavior and convergence toward a DUE state?
- **(Nonlinear Response)** How does introducing a nonlinear response to perceived travel-time differences, reflecting that people tend to ignore small differences, but once a difference becomes noticeable, react increasingly strongly, affect route-choice behavior and convergence toward a DUE state?

In addition to these primary questions, several other analyses were carried out even though their findings play a smaller role in the thesis's contributions: whether considering populations of drivers with different memory levels changes convergence behavior compared with a homogeneous population (Heterogeneous Memory); and a comparison between the implemented algorithm and the SUMO du-alterate benchmark algorithm (Spatial Traffic Comparison).

1.3. Thesis Contributions

The thesis makes the following contributions:

- It develops a software framework coupling a multi-agent reinforcement learning (MARL) route-choice model with a microscopic traffic simulator, designed so that new behavioral mechanisms can be integrated in the decision-making process of agents.
- Using this framework, it tests whether the MARL approach proposed by Wei et al.[49], previously validated only for STA, can also reach a DUE in a DTA setting, implemented here through a microscopic traffic simulator.
- It extends the model proposed by Wei et al. [49] to incorporate cognitively grounded deviations from full rationality, such as risk aversion, waiting-time aversion, and a nonlinear response to perceived travel-time differences. It also examines how each of these more realistic driver behaviours affects route-choice patterns and the network's convergence towards DUE.

Overall, this thesis bridges dynamic traffic assignment (DTA), multi-agent reinforcement learning (MARL), and bounded rationality by integrating them into a single framework for modelling driver route choice.

1.4. Thesis Outline

2. Literature Review

This section reviews the literature relevant to this thesis. First, the foundational concepts and classical approaches to traffic assignment are reviewed. The discussion then moves to bounded rationality and its application to traffic assignment, followed by an examination of reinforcement learning approaches to the problem. Finally, the research gaps are identified and the position of this thesis within the existing literature is discussed.

2.1. Conventional Route Choice Models

Initial traffic assignment models were developed under the assumption of perfect rationality. Over time, it became apparent that some phenomena observed in real-world traffic could not be adequately captured by these models, leading to the emergence of bounded-rationality models in traffic assignment.

Whether based on perfect or bounded rationality, route-choice models have generally followed an approach of adjusting aggregate route flows towards an equilibrium, with less emphasis on the individual learning process of each traveller. These models are referred to as conventional route-choice models in this thesis.

2.1.1. Perfect Rationality

The last step of the traditional four-step transportation planning model is route choice/traffic assignment, which determines how travel demand is distributed across the transportation network. Boyles et al. [8] provide a foundational book on traffic assignment, providing a broad overview of how route choices can be modelled. Generally, traffic assignment models represent traveller behaviour under the assumption of rationality, meaning that travellers act rationally and seek to minimise their travel time.

Following this assumption, the concept of User Equilibrium (UE) naturally arises in Static Traffic Assignment (STA) models. Under UE, for each origin–destination (OD) pair, all used routes have equal and minimal travel time, such that no traveler can reduce their travel time by unilaterally changing routes [48]. In Dynamic Traffic Assignment (DTA), Dynamic User Equilibrium (DUE) extends this equilibrium condition to time-dependent traffic conditions, requiring that, for each OD pair and departure-time interval, the travel time of used routes are equal and minimal [36].

A recent review by Manal Elimadi et al. [15] provides a broad overview of traffic assignment and its foundational concepts, organizing the literature around static and dynamic assignment approaches. The authors highlight how developments in intelligent transportation systems challenge classical traffic assignment approaches and motivate the development of more detailed and flexible models. In particular,

they identify agent-based approaches as a paradigm capable of representing individual users who can learn and adapt their behavior, providing a relevant direction for extending traditional assignment models.

However, the assumption of perfect rationality may be insufficient to capture the complexity of real-world driver behavior. Empirical evidence indicates that travelers often deviate systematically from the travel-time-minimizing behavior commonly assumed. This has motivated the development of behavioral approaches that seek to account for the cognitive and behavioral factors underlying route-choice decisions.

2.1.2. Bounded Rationality

The theory of bounded rationality accounts for limitations in human decision-making, which can help explain the deviations from perfect rationality observed in empirical studies [42, 1]. Simon’s work provides a foundational basis for the concept of bounded rationality, arguing that individuals may be unable to identify optimal solutions because of limited information, cognitive limitations or the complexity of the situation.

In transportation, bounded rationality has been incorporated into traffic assignment models to account for travelers who do not necessarily select the minimum travel time route. For example, Ke Han et al. [21] develop a boundedly rational dynamic user equilibrium (BR-DUE) model in which travelers do not always select the route and departure time with the minimum travel cost. Instead, an indifference band represents travelers’ tolerance for differences between their experienced travel costs and the minimum cost. The study provides a mathematical formulation of BR-DUE with user tolerances and investigates its existence and computation [21].

Xuan Di et al. [12] provide a comprehensive review of boundedly rational route-choice behavior, including empirical evidence showing that perfectly rational models cannot fully capture observed travel behavior. The authors identify cognitive limitations, heuristics and biases, deliberation costs, habits, and myopia as factors underlying deviations from perfect rationality. They organize the literature into *substantive* and *procedural* bounded rationality: the former focuses on the outcomes of boundedly rational decisions, whereas the latter focuses on the decision-making processes. Within both perspectives, bounded-rationality models have been developed for both static and dynamic settings. In the dynamic context, from the perspective of the decision time scale, models can be broadly distinguished into *day-to-day* and *within-day (en-route)* models. In day-to-day models, an agent makes one decision for each travel day, choosing which route to take, whereas in within-day models, an agent makes multiple decisions during a trip, typically deciding at each node which outgoing link to take next.

Finally, Li et al. [25] provide a useful connection between bounded rationality and equilibrium. They propose a day-to-day learning process in which travelers

make probabilistic route choices based on accumulated travel experiences. Their theoretical results show that, under certain conditions, this boundedly rational learning process can converge to User Equilibrium. This is relevant to the present study because it shows that probabilistic individual decisions do not necessarily prevent the aggregate traffic flow from reaching equilibrium.

Agent-based approaches provide a natural framework for modelling boundedly rational travellers and investigating how they learn and adapt their decisions. Reinforcement learning (RL) is particularly suitable for this purpose because it allows individual agents to learn from their travel experiences and adapt their route choices.

2.2. Reinforcement Learning Route Choice Models

In contrast to conventional traffic assignment approaches, where route flows are directly adjusted toward an equilibrium solution, RL-based models represent route choice at the individual-agent level, allowing travelers to learn and adapt their decisions based on past experiences. However, the information available to travelers may also include knowledge of the network and current traffic conditions, in addition to their own past experiences. The interaction of individual decisions gives rise to aggregate traffic patterns that may converge to an equilibrium state, without requiring equilibrium to be imposed directly as the solution of the model.

This has motivated increasing interest in *multi-agent reinforcement learning (MARL)* approaches for route choice, which provide a framework for explicitly modeling the learning and adaptation processes of individual travelers, which have received comparatively less attention in conventional traffic assignment approaches.

One of the early applications of RL to day-to-day route choice was presented by Wei et al. [49], who developed a MARL route choice model based on the Bush–Mosteller (BM) RL algorithm. Each driver is represented as an individual agent with a probability distribution over available routes, which is updated based on its own travel experience and used to select routes stochastically. The model explicitly incorporates behavioral factors such as memory, learning rate, and travel-time perceptions shaped by past experience. The authors show that these parameters can affect the evolution of network flows and the final network state. In particular when travelers have perfect memory, the resulting network flow converges to *User Equilibrium (UE)*. This is consistent with Li et al. [25], who show that boundedly rational models can also converge to UE under appropriate conditions. In contrast, limited memory can prevent such convergence. The model therefore provides an important example of how individual learning and stochastic route choice can generate aggregate traffic dynamics and, under appropriate conditions, lead to UE. The use of BM learning is particularly relevant to the present study because its simple and intuitive formulation provides a transparent representation of the agent’s learning process.

Bazzan and Grunitzki [6] extended the use of MARL to within-day, en-route choice. Their approach represents individual drivers as agents using Q-learning, allowing them to make sequential decisions during the trip. In particular, at each node, an agent decides which outgoing link to take. The results are comparable to those obtained with the Method of Successive Averages (MSA), which they use as a standard method for approximating User Equilibrium, while the proposed approach is nearly as efficient as conventional iterative assignment methods. This work demonstrates that MARL can be applied not only to day-to-day learning but also to within-day, en-route decision-making, and illustrates the use of more sophisticated RL approaches such as Q-learning in this setting.

Some studies consider other behavioral signals for learning. Ramos et al. [35] proposed a Q-learning approach that uses action regret rather than travel time alone as the learning signal. Agents estimate regret based on their own travel experiences. The learning process is shown to avoid high regret and converge toward a ϕ -approximate User Equilibrium, where ϕ is the allowed level of regret. This study is relevant because it shows how the learning signal can be designed to represent different aspects of decision-making.

However, the previous studies [49, 6, 35] rely on static traffic assignment (STA) and link performance functions, limiting their ability to capture how traffic conditions vary over time and to represent individual vehicles, as can be achieved through DTA and microscopic traffic simulation. In addition, these studies do not evaluate convergence to UE using the standard relative gap (R-gap) metric, which compares the travel costs experienced by travelers with the minimum travel costs they could have experienced. As the experienced travel costs approach the minimum possible travel costs, the R-gap approaches zero, indicating convergence towards UE in an STA setting or DUE in a DTA setting. Instead they rely on mean travel time and path flows, making it more difficult to directly assess the extent to which the resulting network state satisfies the UE conditions.

The development of MARL route-choice models has also moved from static to dynamic traffic environments. Shou et al. [40] proposed a within-day, en-route choice model. Their Mean-Field Multi-Agent Deep Q-Learning approach allows agents to make en-route decisions based on their current location and time, while also accounting for the influence of other travelers. In particular, each agent uses the traffic flow on the chosen link to capture the level of competition it faces on that link. Unlike earlier models based on static traffic assignment and link performance functions, their framework considers dynamic traffic assignment and can use different dynamic network loading models, including microscopic simulation. The results show that this approach can converge to the Dynamic User Equilibrium (DUE) under the assumptions considered. This study therefore represents an important step from static traffic assignment toward dynamic and more realistic representation of traffic. However, the model does not explicitly consider bounded rationality or cognitive limitations. In addition, the mean-field approach provides agents with external information about the traffic conditions created by

other travelers, which differs from the more limited information available to individual travelers in the present study. Sang Koh [38] also proposed a within-day, en-route choice model using a DTA setting and a microscopic simulator such as SUMO, combined with an improved Deep Q-Learning algorithm. In this framework, only one driver acts as an RL agent, while the remaining traffic is treated as background traffic. The RL agent has access to both its own experienced information and some global information about the traffic network, which it uses to decide at each node which outgoing link to take, with the objective of reducing travel time or emissions. The proposed approach outperformed SUMO's built-in navigation methods in both simplified and realistic networks, demonstrating the potential of deep RL for en-route optimization. This work is particularly relevant to the present study because it demonstrates the integration of RL-based route choice with microscopic traffic simulation. However, the model assumes access to global network information, whereas the present study considers more limited information available to individual travelers. Furthermore, it focuses on within-day, en-route decision-making, with travelers selecting the next link at each node. In contrast, the present study assumes that travelers select their complete route before starting their trip.

As MARL models are applied to increasingly large transportation networks, scalability becomes an important challenge. Modeling every traveler as an independent RL agent can be computationally heavy. A common approach to addressing this issue is to change the level at which agents are defined. Gu et al. [18] addressed this issue through the Two-Stage Ensemble Reinforcement Learning (TERL) framework. Rather than assigning a separate RL model to every vehicle, travelers are grouped into quasi-homogeneous clusters according to trip and socio-demographic characteristics, and a representative RL model is trained for each cluster. The experiences of vehicles within the same cluster are then jointly used to train the corresponding representative agent. The framework is implemented in a day-to-day DTA setting using the SUMO simulator. The results show that TERL reduces total travel time and the variation in experienced travel times compared with the shortest-path and DUE models. Increasing the number of clusters improves travel-time performance at the cost of higher computational requirements. Wang et al. [47] proposed to represent OD pairs rather than individual travelers as agents. Each OD-level agent determines how traffic demand is distributed across the available routes using an Independent Proximal Policy Optimization (IPPO) framework with a Dirichlet policy and action pruning. By defining agents at the OD level instead of the traveler level, the number of agents is greatly reduced. The model uses a R-gap-based reward to encourage convergence toward UE or system-optimal conditions. The main finding is that defining MARL agents at the OD-pair level can substantially improve the scalability of MARL approaches. However, the approach relies on a static traffic assignment (STA) model and therefore does not capture time-dependent traffic conditions.

However, this improvement in scalability achieved in [18, 47] comes at the cost of losing the ability to track how individual travelers learn and adapt their route choices, which is a key focus of the present study.

A related effort to improve the reproducibility of MARL route-choice research is RouteChoiceEnv [2]. The library provides a standard environment, benchmark networks. Individual drivers are represented as independent agents, with algorithms including regret-minimizing Q-learning and toll-based Q-learning. The framework is relevant for establishing a common benchmark for MARL route-choice research and improving reproducibility. However, its use of STA and link performance functions limits its representation of dynamic and individual traffic behavior.

More recent research has begun to explore richer representations of traveler decision-making. Sun and Li [45] proposed an LLM-guided reinforcement-learning framework. To improve scalability, it uses representative agents corresponding to homogeneous traveler groups. The LLM evaluates travel experiences and identifies routes that should receive positive reinforcement, after which an explicit probability-update mechanism modifies the route-choice probabilities. Two update rules are considered, one of which is inspired by the Bush-Mosteller RL algorithm. By using LLMs to interpret travel experiences in natural language (see Figure 7), the framework can account for a wider range of travel factors without requiring them to be explicitly converted into a single numerical reward, while also allowing different traveler profiles and preferences to be represented. The authors report that the resulting day-to-day dynamics converge rapidly toward UE across the tested networks, based on the R-gap. However, the framework relies on static traffic assignment (STA) and link performance functions, limiting its ability to capture time-dependent traffic conditions. Moreover, the use of representative agents limits the ability to capture and analyze the learning processes of individual travelers, which is a key focus of the present study.

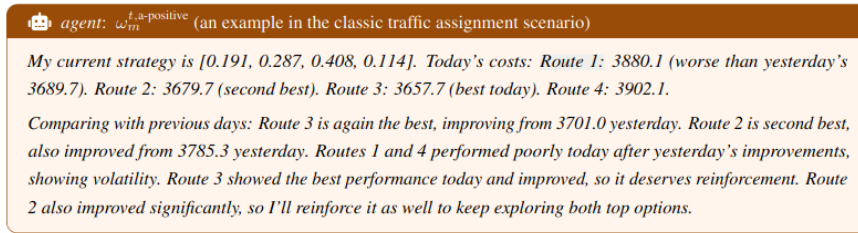


FIG. 7. Example of LLM interpretation of travel experiences in natural language from the model proposed by Sun and Li [45]. Reproduced from [45]

Jamal Amani et al. [34] uses Q-learning to investigate individual route-choice behavior and estimate psychologically interpretable learning parameters. They collect data by carrying out experiments in a simulator and stated preference surveys. In

particular, the results show that different travelers have different values of the psychologically interpretable parameters, indicating that they learn from experience and explore alternative routes in different ways. It also examines the effect of external information on the route choice process. An important finding is that although real-time information can influence the learning process, travelers appear to learn more strongly from their own travel experiences than from external information. In other words, travelers may underestimate information about a potentially poor choice until they experience its consequences themselves. For example, a navigation system may warn a traveler that the typically shortest route is congested or affected by road maintenance and recommend an alternative. However, the traveler may still choose that route because they expect the problem to be less severe than reported. After experiencing the delay, the traveler may substantially reduce the probability of choosing that route in the future. This study is relevant because it highlights the importance of modeling how individual travelers learn from experience and adapt their route choices over time. However, the model is not used to generate network-level traffic or simulate traffic dynamics.

2.3. Synthesis and Research Gap

The relationships between the main concepts discussed throughout this chapter are summarised in Figure 8. The figure provides an overview of the traffic assignment field, from its role within the four-step model to the distinction between static and dynamic traffic assignment. It then focuses on dynamic traffic assignment, distinguishing between conventional and MARL-based approaches, before further classifying MARL-based DTA models according to their decision-making horizon and the information available to agents. This overview allows the studies discussed in this chapter to be broadly positioned within the traffic assignment field.

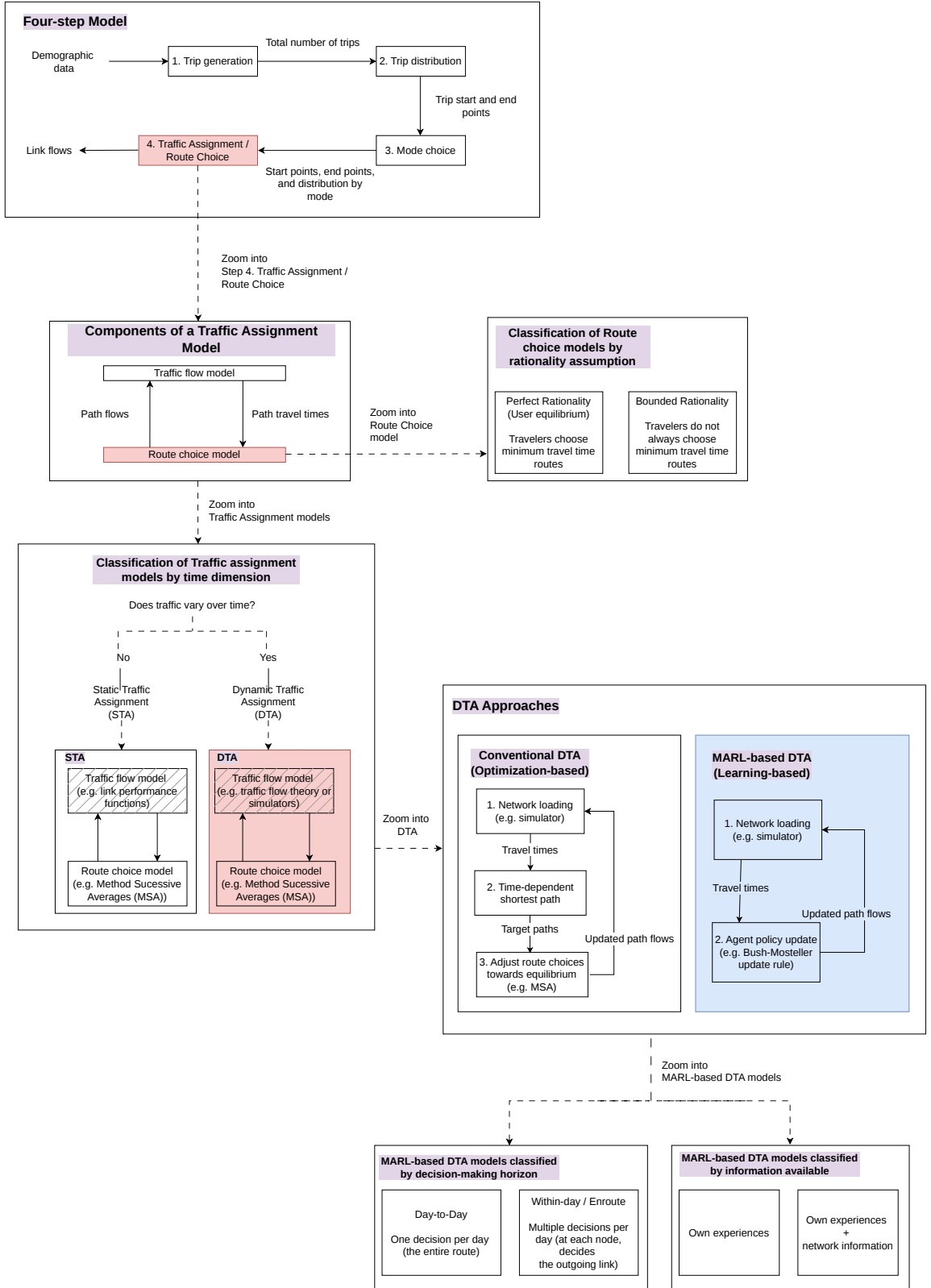


FIG. 8. Overview of the traffic assignment field and classification of MARL-based DTA models.

The literature reviewed in this chapter shows an evolution from relatively simple and behaviorally interpretable RL models, such as BM learning, toward more flexible but increasingly complex deep RL approaches. This has recently motivated research on improving scalability through approaches such as representative agents. However, several limitations remain.

First, most MARL route-choice models are developed using static traffic assignment (STA) and link performance functions, which provide a simplified representation of traffic. Relatively fewer studies combine MARL route choice with dynamic traffic assignment (DTA) and microscopic traffic simulation, which can represent time-dependent traffic conditions and interactions between individual vehicles more realistically. Moreover, the MARL-DTA studies reviewed in this thesis generally consider en-route route choice and provide agents with additional information about network conditions. In contrast, the present study considers day-to-day route choice in which agents learn solely from their own experiences.

Second, most MARL route choice studies focus primarily on checking whether the model converges to equilibrium, with less attention to the behavioral mechanisms underlying individual route choices. Although some studies consider stochastic decisions relatively few incorporate known behavioral characteristics of boundedly rational decision-making, such as cognitive limits, limited information, memory, inertia, and preferences.

Third, relatively few MARL route-choice studies use the relative gap (R-gap) to evaluate convergence to User Equilibrium. Many instead assess equilibrium through travel times, route flows, or other network-level measures. The R-gap provides a standard measure of the distance between the current traffic assignment and User Equilibrium, making it more appropriate for directly evaluating equilibrium convergence.

Finally, many recent studies employ increasingly complex RL algorithms, such as *Deep Reinforcement Learning (DRL)*. While these approaches can handle richer state spaces and sequential decisions, their learning processes are generally less transparent and more computationally demanding. This has motivated scalability strategies such as representative agents. However, reducing the number of agents or aggregating travelers can also limit the ability to represent and analyze how individual travelers learn and adapt their route choices.

These limitations reveal a gap that this study addresses by combining multi-agent reinforcement learning with interpretable boundedly rational behavioural mechanisms in a DTA setting using a microscopic traffic simulation environment, where each driver is an agent that learns solely from their own experiences. The aim is to model how individual travelers learn and adapt their route choices using limited information and other behavioral factors, while evaluating the resulting traffic patterns and their convergence toward DUE using the R-gap. By combining individual-agent reinforcement learning with DTA and microscopic traffic simulation, the model allows individual decisions and interactions to be represented

explicitly, so that aggregate traffic patterns emerge from individual learning and behavioral adaptation rather than being imposed directly through an aggregate traffic-assignment procedure.

3. Traffic Modeling and Simulation

This section first provides further details on the traffic assignment field and briefly explains how traffic networks are represented. It then introduces traffic simulation and describes in detail the main characteristics of the traffic simulator used in this thesis. Finally, bounded rationality is discussed in greater depth.

3.1. Traffic Assignment

Figure 4 provides a general representation of traffic assignment models, including both static and dynamic approaches. As mentioned previously, the main difference between the two is the traffic flow model they use.

In *static traffic assignment (STA)*, the traffic-flow model is typically represented by link performance functions, which map the flow on a link to its travel time. For example, $t_a(x_a)$ denotes the travel time on link a as a function of the flow x_a on that link. These functions are generally nondecreasing. One of the most common is the Bureau of Public Roads (BPR) function. STA models are relatively simple to formulate and solve, but their assumption that traffic conditions are constant over time—the travel time of a path is considered the same whether a traveller departs at 8:00, 8:30, or 10:00—limits their ability to capture time-varying congestion and phenomena such as queue formation, where vehicles accumulate when demand exceeds the capacity of a link.

Dynamic traffic assignment (DTA), in contrast, uses a dynamic traffic-flow model, which may be based on traffic-flow theory or a traffic simulator. This allows it to represent time-dependent traffic conditions, in which congestion and travel times vary over time—the travel time of a path depends on the departure time of the traveller. DTA can consider both route choice and departure-time choice, although this thesis focuses on route choice.

Figure 9 illustrates an example to further develop the intuition behind the distinction between STA and DTA. Under STA, because the network is assumed to be in a steady state, the travel time of Path 1 is the same regardless of the departure time. In contrast, under DTA, time-dependent traffic conditions are represented, so the travel time of Path 1 depends on the departure time of the vehicle.

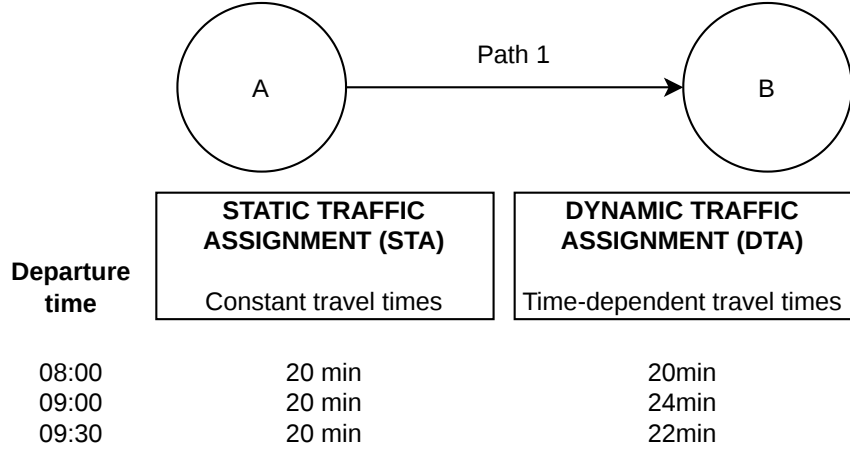


FIG. 9. Comparison STA vs DTA.

As introduced in the background, DUE extends the UE concept to time-dependent traffic conditions. In DUE, the relevant travel time is the *experienced travel time*, which reflects the traffic conditions encountered throughout the entire trip, rather than the *instantaneous travel time*, which reflects the conditions at the time of departure. Figure 10 illustrates how the use of instantaneous travel times can lead to inappropriate route choices in a dynamic setting. The figure focuses on the decision-making process of the vehicle indicated by the red arrow, which will be referred to as the agent to distinguish it from the other vehicles in the network. The agent has perfect knowledge of the current network state. The upper and lower diagrams represent the network at time steps t_0 and t_1 , respectively. At t_0 , the agent observes the current network conditions and decides that the best route is through the lower link because it has fewer vehicles. However, by t_1 , another vehicle is about to enter the lower link, while the agent is still on its way to it. Since both the upper and lower links now have two vehicles, the upper link would have been the better choice. This shows that instantaneous travel times do not account for future network conditions, motivating the use of experienced travel times instead.

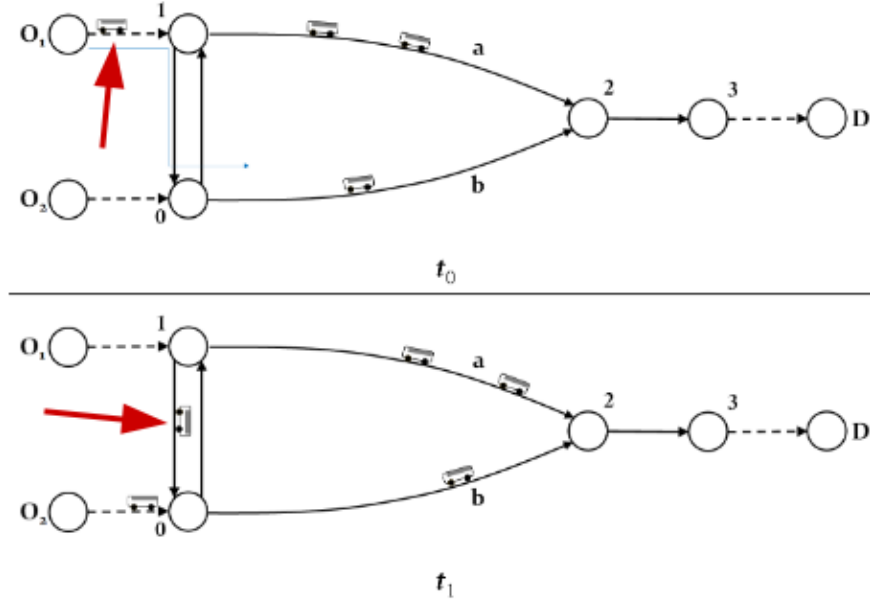


FIG. 10. Example of a wrong choice when considering instantaneous travel time in a dynamic setting. Reproduced from [15].

As travel conditions vary over time the shortest path for a traveler may also depend on the departure time. The time-dependent shortest-path (TDSP) problem determines the least-cost path for a given departure time. Unlike a conventional shortest-path problem, the cost of each link depends on the time at which it is entered, which in turn depends on the travel times of the preceding links. For a more detailed description of the TDSP problem, see [26, Section 4.3.5].

In the conventional DTA framework, the interaction between route choices, traffic conditions, and time-dependent shortest paths can be represented as an iterative process consisting of three main steps. Figure 11 illustrates this process.

First, given a set of route choices and departure times, the traffic-flow model performs *dynamic network loading* and determines the resulting time-dependent travel times on each link of the network. Second, these travel times are used to find the time-dependent shortest paths. Third, route flows are updated based on the calculated shortest paths. A common algorithm used to update the flows is the *Method of Successive Averages (MSA)*. These three steps are repeated until convergence is reached, if possible.

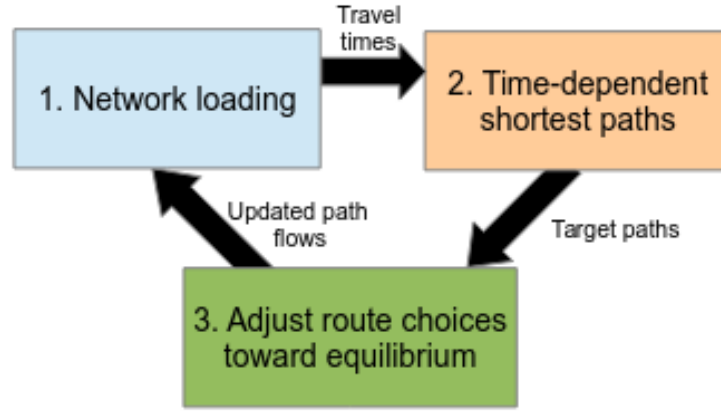


FIG. 11. Classical DTA iterative process. Reproduced from [8]

In contrast to conventional DTA models, reinforcement learning-based DTA models represent route choice at the individual level, where individual agents select routes according to their policies. Here, a policy refers to the probability distribution over the available routes for an agent. Once the agents have selected their routes, these choices are provided to the traffic environment, which simulates the resulting traffic conditions and returns the corresponding travel times. These travel times are then used by the agents to update their policies. Figure 12 illustrates this process.

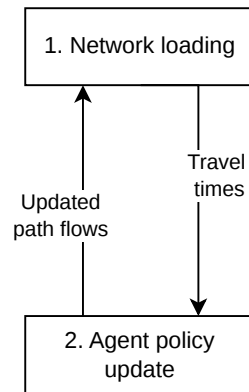


FIG. 12. Reinforcement Learning DTA iterative process.

Rather than explicitly determining shortest paths and adjusting aggregate route flows towards equilibrium, as in conventional models (Figure 11), the final state of the network in the RL approach is a consequence of the individual decisions made by the agents.

Compared with STA, DTA requires greater computational resources and more detailed input data (time-dependent OD matrices), but allows the temporal evolution of congestion to be represented.

Other assignment rules exist besides user equilibrium, such as *system optimum (SO)*. Under UE, travelers independently minimize their travel times, whereas under SO, a central coordinator minimizes total system travel time.

3.2. Network Representation

A transportation network can be represented as a directed graph consisting of *nodes* and *links*. Nodes represent intersections, while directed links represent road segments connecting two nodes. Here, links are used instead of edges because they have a direction. An *origin* is the location where a vehicle starts its journey, and a *destination* is where it finishes. In this thesis, origins and destinations are represented by nodes. A *path* or *route* is a sequence of connected links between an origin and a destination. The combination of an origin and destination is referred to as an *OD pair*.

Travel demand is commonly represented using an *OD matrix*, where rows represent origins, columns represent destinations, and each cell contains the demand between the corresponding OD pair.

3.3. Traffic Simulation

In many cases, conducting experiments in real-world environments is impractical or resource-intensive. Simulation provides an alternative by enabling the study of systems within controlled artificial environments, allowing their behavior to be evaluated before real-world deployment and at a relatively low cost and time. This is particularly useful for traffic systems, where the traffic-flow model can be either an *analytical traffic-flow model*, or a *traffic simulation model*. Analytical models are generally more suitable for simpler cases and provide aggregate information, such as the average speed on a link at a given time. Traffic simulation models can represent more complex traffic conditions and provide information at both the aggregate and individual levels, such as the speed and position of individual vehicles. Because this thesis focuses on the decision-making process of individual drivers, a microscopic traffic simulator is used (see Figure 5).

Traffic simulation allows hypothetical scenarios to be evaluated under controlled conditions and at relatively low cost, while enabling experiments to be run faster or slower than in reality, for example by compressing hours of traffic into minutes

or slowing down events for detailed analysis. These capabilities make simulation particularly useful for studying complex traffic systems and interactions that are difficult to represent analytically or investigate in the real world.

Traffic simulation models are commonly classified according to their level of granularity, ranging from low to high fidelity as follows:

- Macroscopic simulation models
- Mesoscopic simulation models
- Microscopic simulation models

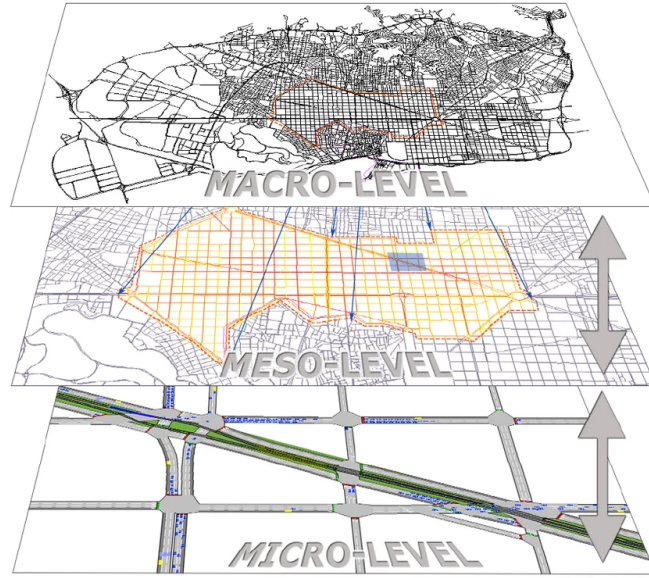


FIG. 13. Overview of traffic simulation models. Reproduced from [5]

Macroscopic models represent traffic as aggregate flow without explicitly modeling individual vehicles. Routing decisions are therefore also represented in aggregate terms, such as the proportion of flow assigned to each route. Microscopic models, in contrast, represent individual vehicles and their interactions using behavioral models such as car-following and lane-changing. Route choice can also be modeled at the individual level. Mesoscopic models lie between these approaches, representing individual vehicles with simplified interaction dynamics.

Microscopic models can be further classified by their spatial and temporal representation as either discrete or continuous. A model is *discrete in time* when its state is updated at fixed time steps.

The thesis uses SUMO (Simulation of Urban MObility), a discrete-time microscopic traffic simulator. The main reasons for this choice are:

- **Open-source**
- **Performance and stability**
- **Output capabilities:** SUMO provides detailed simulation outputs that can be configured according to the needs of the study.

3.4. SUMO

The development of SUMO (Simulation of Urban MObility) was initiated in 2001 by the German Aerospace Center (DLR). It is a comprehensive suite of traffic simulation tools that covers a wide range of functionalities. SUMO relies extensively on XML-based file formats to define networks, demand, and configuration settings. It also generates a variety of XML outputs that provide detailed information about the simulation which can be used for post-simulation analysis.

An overview of the main characteristics of SUMO is presented below:

- **Network building:** SUMO provides multiple approaches for network creation. It includes tools for *automatically* generating synthetic networks, as well as importing real-world networks from external sources such as OpenStreetMap or other traffic simulators (e.g., VISUM) using the `netconvert` tool. Additionally, the `netedit` tool enables *manual* creation and modification of networks. In this thesis, the network was created and edited using `netedit`.
- **Demand modeling:** Demand in traffic simulation describes the vehicles that enter the network, including their origins, destinations, departure times, and, where relevant, their routes.
 - (1) **Demand representation:** SUMO supports different demand definitions. A route is an ordered sequence of links representing the path followed by a vehicle from its origin to its destination.
 - **Trip:** Defines a vehicle by specifying its origin link, destination link, and departure time. The route can subsequently be computed automatically using `duarouter`.
 - **Flow:** Defines a stream of vehicles over a specified time interval, specifying their origin, destination, and either a total number of vehicles or a flow rate. Routes can be computed automatically using `duarouter`.
 - **Vehicle with predefined route:** Defines a vehicle together with an explicitly specified route.

Origins and destinations are typically nodes, but SUMO specifies them using links instead.
 - (2) **Demand generation:** Demand can be generated from different data sources:
 - **OD matrices:** The `od2trips` tool converts OD matrices into trips.
 - **Traffic counts:** Tools such as `routeSampler` can generate demand from turn counts or link counts.

- **Synthetic demand:** The `randomTrips` tool generates synthetic trips when observed demand data are unavailable.
- **Manual specification:**

In this thesis, `randomTrips.py` is used to generate the synthetic demand used in the experiments.

- (3) **Routing:** When demand is specified as trips or flows, routes must be computed. SUMO provides several routing approaches:
- **Shortest-path routing:** `duarouter` provides shortest-path routing using specified link travel-time costs.
 - **Dynamic User Equilibrium (DUE):** The `duaIterate` tool iteratively performs traffic assignment and simulation to approximate a dynamic user equilibrium.

In this thesis, `duaIterate.py` is used to obtain a DUE benchmark, while `duarouter` is used to compute shortest-path computations in several parts of the implementation.

- **API (TraCI):** SUMO provides the TraCI (Traffic Control Interface), which enables interaction with a running simulation. It allows simulation data to be retrieved (e.g., vehicle speeds) and the simulation state to be modified (e.g., vehicle behavior) during execution.

In the day-to-day route-choice setting, TraCI is not required, as the feedback signal received by the RL agents is obtained from output files generated after the simulation has finished. Therefore, interaction with the simulation during execution is not necessary.

- **Vehicle and driver behavior:** SUMO provides a high degree of flexibility in modeling individual vehicles and their behavior. Traffic dynamics are governed by car-following and lane-changing models, which determine how vehicles accelerate, decelerate, and interact with their surroundings. These models take as input both *vehicle-specific characteristics*, such as vehicle length, maximum speed, and acceleration capabilities, and *driver behavior parameters*, including impatience and cooperation. In this work, the default parameter settings provided by SUMO are used to focus the analysis on route-choice behavior.

FIG. 14. Vehicle type configuration in SUMO, showing parameters related to car-following, lane-changing, and vehicle characteristics

3.5. Bounded Rationality

Building on the concept introduced in the Background, the following briefly reviews the empirical evidence that motivates the use of the *bounded rationality* (BR) framework and the main approaches to modeling it.

Previous research has shown that human decision-making does not always conform to the assumptions of perfect rationality. For example, people may use heuristic decision rules to reduce the cognitive effort required to evaluate alternatives, which can lead to systematic biases [9]. In route choice, several studies on commuting (work-related trips) show that travelers do not always select the shortest path. GPS-based studies found that approximately 60% of commuter trips in the Twin Cities, Minnesota, did not follow the shortest path [52], while a high proportion of commuting routes in Nagoya, Japan, also differed from the shortest paths [31].

Habit and inertia provide another example of boundedly rational behavior. Individuals may favor alternatives associated with the status quo [37], and habits can hinder adaptation when conditions change and better alternatives become available [23]. In the context of route choice, [27] examined the effects of information on familiar and unfamiliar drivers using traffic simulators. The results indicated that familiar drivers tended to consider only prominent information and often remained on their usual routes, without necessarily minimizing travel time.

Cognitive limitations can also affect route choice; for example, limited memory may cause travelers to place greater weight on recent travel experiences [7], while *myopia* refers to a greater emphasis on short-term consequences [9]. Together, these findings show that route choices are influenced not only by travel costs but also by behavioral preferences, cognitive limitations, and deliberation costs.

BR models can be broadly classified into *substantive* and *procedural* approaches [43]. Substantive approaches assume a boundedly rational behavioral rule and

its parameter values, and examine their implications for the system. Procedural approaches, in contrast, focus on the decision-making process itself and try to infer both the behavioral rule and its parameter values from people’s behavior. The proposed model is closer to a substantive approach.

The distinction between perfect and bounded rationality can be further described in terms of the assumptions made about the decision-making process. Under perfect rationality, the route-choice model assumes that [48]:

- (1) drivers have perfect information about the costs of all paths;
- (2) drivers know all feasible paths connecting their origin-destination pair;
- (3) drivers choose the optimal path; and
- (4) drivers always switch to the current optimal path.

The bounded rationality framework relaxes these assumptions and can be summarized by four corresponding aspects [44]:

- (1) travelers collect only some of the available information about possible paths when forming path-cost estimates;
- (2) travelers consider only a subset of the feasible paths;
- (3) travelers do not always choose the optimal path, either because they use a non-optimal route-choice mechanism or because they use an optimal mechanism with perception errors; and
- (4) during repeated route-choice learning, travellers may update only the information that stands out to them and may fail to switch to shorter or otherwise better routes that are not sufficiently noticeable.

The route-choice model implemented in this thesis uses reinforcement learning agents, as illustrated in Figure 5. The proposed RL framework can represent both rational and irrational drivers, depending on how the agents’ decision-making is modelled. For the purposes of this thesis, it is important to clearly define what is meant by the two simplified terms *rational driver* and *irrational driver*. The term *rational driver* refers specifically to a Bush–Mosteller agent with perfect memory, meaning that past travel experiences are not discounted when forming route perceptions. However, this term should not be interpreted as implying strict perfect rationality. A perfectly rational driver would always select the route with the lowest perceived travel cost. In the proposed Bush–Mosteller model, drivers instead choose a route stochastically according to their probability distribution over the available routes. Consequently, although drivers with perfect memory are referred to as *rational drivers*, their route-choice behavior still incorporates a small degree of bounded rationality. The term *irrational driver*, as used in the title of this thesis, refers to drivers who do not have perfect memory and whose decision-making may incorporate additional behavioral mechanisms, such as risk sensitivity, waiting-time sensitivity, and a nonlinear response. Hence, in this thesis, the difference between *rational drivers* and *irrational drivers* is understood in terms of their degree of

bounded rationality, with rational drivers exhibiting a relatively small degree and irrational drivers exhibiting a higher degree.

It is therefore relevant to understand how the proposed model, which can represent drivers with different degrees of bounded rationality, captures the assumptions of the bounded-rationality framework listed in 3.5. The first assumption is common to all drivers, independently of their degree of bounded rationality, as path-cost estimates are based on agents' past travel experiences rather than complete information. The second assumption is also common to all drivers, independently of their degree of bounded rationality, since agents consider only a subset of feasible routes. The third assumption is captured by the *irrational drivers* through the risk-averse and waiting-time-averse mechanisms, which modify route-choice behaviour such that agents do not necessarily select the route with the shortest travel time. The fourth assumption is captured by the *irrational drivers* through the nonlinear response mechanism, which makes agents less responsive to small differences in route costs.

4. Reinforcement Learning

Reinforcement learning (RL) is a framework in which an agent learns how to make decisions through feedback from the environment. An agent selects an action and receives a reward or penalty, which is used to improve its future decisions.

In this thesis, the classical Bush–Mosteller (BM) reinforcement learning model is implemented as the route choice model, with each RL agent representing an individual driver (see Figure 5).

4.1. Bush–Mosteller Reinforcement Learning

The Bush–Mosteller (BM) model is a classical reinforcement learning model in which agents maintain and update a probability distribution over actions, from which decisions are drawn [28]. The intuition behind the algorithm is that actions that perform well become more likely to be selected, while actions that perform poorly become less likely to be selected.

After each decision, the agent receives a *stimulus* that reflects how satisfactory the action was. This stimulus is typically determined by comparing the *payoff of the decision* with an *aspiration level* (a reference value comparable to a baseline or expected payoff). A positive stimulus occurs when the payoff of the selected action exceeds the aspiration level and increases the probability of selecting that action in the future, while a negative stimulus occurs when the payoff falls below it and decreases that probability. Each agent learns independently from its own past experiences.

Let $u_i^t(c)$ denote the payoff obtained by agent i when selecting action c at time t , and let A_i^t be the corresponding aspiration level. The corresponding stimulus is defined as

$$(1) \quad S_i^t(c) = \frac{u_i^t(c) - A_i^t}{\max_k |u_i^t(k) - A_i^t|}$$

which represents a normalized measure of how the obtained payoff compares to the agent’s aspiration level (baseline). The denominator normalizes the stimulus to the interval $[-1, 1]$.

After computing the stimulus, the agent updates the probability of the selected action c according to

$$(2) \quad p_i^{t+1}(c) = \begin{cases} p_i^t(c) + \lambda (1 - p_i^t(c)) S_i^t(c), & \text{if } S_i^t(c) \geq 0, \\ p_i^t(c) + \lambda p_i^t(c) S_i^t(c), & \text{if } S_i^t(c) < 0, \end{cases}$$

where $p_i^t(c)$ denotes the probability that agent i selects action c at time t , and $\lambda \in (0, 1)$ is the *learning rate*. When the stimulus is positive, the probability of the selected action increases proportionally to both the stimulus and the remaining probability mass $(1 - p_i^t(c))$. Thus, an action with a high probability can only increase by a small amount, whereas an action with a low probability can experience a larger increase when it receives a strong positive stimulus. This can be interpreted behaviorally: a positive stimulus produces little change when an action was already highly preferred, but can substantially change the agent's perception of an action when it was previously unlikely to be selected. Conversely, when the stimulus is negative, the probability decreases proportionally to its current value, so highly probable actions are more strongly affected by a negative stimulus than actions that were already unlikely.

The probabilities of the remaining actions are adjusted accordingly to ensure that they sum to one.

The overall Bush-Mosteller learning procedure followed by each agent is summarized in Algorithm 1

Input: Set of actions D

Output: Updated probabilities p_i^t

Initialize $t \leftarrow 1$;

Initialize probabilities p_i^t over $k \in D$ such that $\sum p_i^t(c) = 1$;

Initialize aspiration level A_i^t ;

while *stopping criterion is not met* **do**

 Select action $c \in D$ according to p_i^t ;

 Observe payoff $u_i^t(c)$;

 Compute stimulus $S_i^t(c) \propto u_i^t(c) - A_i^t$;

Update probability of chosen action c

if $S_i^t(c) \geq 0$ **then**

 Increase probability $p_i^t(c) \leftarrow p_i^t(c) + \lambda(1 - p_i^t(c))S_i^t(c)$;

end

else

 Decrease probability $p_i^t(c) \leftarrow p_i^t(c) + \lambda(p_i^t(c)S_i^t(c))$

end

 Update aspiration level (model-dependent);

 Update probabilities of the other actions, so that $\sum p_i^t(k) = 1$

$t \leftarrow t + 1$;

end

Algorithm 1: Bush-Mosteller Learning Process for Agent i

The learning process can be stopped after a fixed number of iterations or when it reaches a quasi-stationary state, characterized by sufficiently small changes in action probabilities across iterations.

4.2. Bush-Mosteller Model for Day-to-Day Route Choice

The formulation presented in this section is based on the day-to-day route choice model proposed by Wei et al. [49], which adapts the Bush-Mosteller learning framework introduced in the previous section to traffic assignment.

Let R_W denote the set of available routes for OD pair W . At each day t , traveler i selects a route $c \in R_W$ according to the probability distribution over the available routes.

The model distinguishes between two travel-time measures. The *expected travel time (ET)*, denoted by A_i^t , represents the traveler's baseline and is computed from past travel times across all routes. The *perceived travel time (PT)*, denoted by $\bar{T}_i^t(k)$, is route-specific and is computed from the traveler's past experiences on route k . Both measures use a memory-decay mechanism that gives greater weight to more recent experiences, reflecting the cognitive limitations discussed in Section 3.5.

Since travelers only observe the travel time of the selected route, perceived travel times are used to estimate route performance. Although the actual travel time of the selected route is observed, it can vary substantially from day to day. The perceived travel time, based on past experiences, therefore provides a more stable estimate of route performance.

The stimulus is calculated by comparing the perceived travel time of the selected route $\bar{T}_i^t(k)$ with the expected travel time A_i^t :

$$(3) \quad S_i^t(c) = \begin{cases} \frac{A_i^t - \bar{T}_i^t(c)}{\max_k |A_i^t - \bar{T}_i^t(k)|}, & \text{if } A_i^t - \bar{T}_i^t(c) \geq 0, \\ \frac{A_i^t - \bar{T}_i^t(c)}{\min_k |A_i^t - \bar{T}_i^t(k)|}, & \text{if } A_i^t - \bar{T}_i^t(c) < 0. \end{cases}$$

Here, positive values indicate that the perceived travel time of the selected route is lower than the traveler's baseline, while negative values indicate a higher perceived travel time. The denominator normalizes the positive and negative stimulus separately.

The expected travel time is defined as

$$(4) \quad A_i^t = \frac{\sum_{j=1}^{t-1} \gamma^{t-1-j} T_i^j}{\sum_{j=1}^{t-1} \gamma^{t-1-j}},$$

where T_i^j denotes the travel time experienced by traveler i on day j and $\gamma \in (0, 1)$ is the memory parameter.

Similarly, the perceived travel time of route k is given by

$$(5) \quad \bar{T}_i^t(k) = \frac{\sum_{j=1}^t \gamma^{t-j} T_i^j \xi_{ik}^j}{\sum_{j=1}^t \gamma^{t-j} \xi_{ik}^j},$$

where ξ_{ik}^j is an indicator variable equal to 1 if traveler i selected route k on day j , and 0 otherwise.

It is important to note that the ET is computed using past travel times up to time $t - 1$, and therefore reflects experience prior to the current decision, whereas the PT includes travel times up to time t , incorporating the most recent observation.

Once the stimulus $S_i^t(c)$ has been computed, the probability of selecting route c is updated according to the BM learning rule in Eq. (2). The corresponding update for the non-selected routes in Wei et al. [49] is missing a term. The corrected formulation used in this thesis is given in Eq. (6) with its derivation provided in Appendix A.

$$(6) \quad p_k^{t+1} = \begin{cases} p_k^t - l p_k^t S_i^t(c), & \text{if } S_i^t(c) \geq 0, \\ \frac{p_k^t - p_k^t p_c^t - p_k^t p_c^t l S_i^t(c)}{1 - p_c^t}, & \text{if } S_i^t(c) < 0. \end{cases}$$

The overall Adapted Bush-Mosteller learning procedure followed by each agent is summarized in Algorithm 2

Input: OD pair W , set of routes R_W , initial baseline A_1

Output: Updated route probabilities p_i^t

Initialize $t \leftarrow 1$;

Initialize $p_i^1(k) = 1/|R_W|$ for all $k \in R_W$;

Initialize baseline $A_i^1 \leftarrow A_1$;

while *stopping criterion is not met* **do**

Choose route;

 Select route $c \in R_W$ according to p_i^t ;

Experience travel time;

 Observe travel time T_i^t ;

Update travel-time measures;

 Update baseline A_i^t using past travel times;

 Update perceived travel times $\bar{T}_i^t(k)$ for all $k \in R_W$;

Compute stimulus;

 Compute $S_i^t(c)$ according to Eq. (3);

Update probability of selected route;

 Update $p_i^t(c)$ according to Eq. (2);

Update probabilities of remaining routes;

 Update $p_i^t(k)$ for all $k \neq c$ according to Eq.(6);

$t \leftarrow t + 1$;

end

Algorithm 2: Bush–Mosteller Model for Day-to-Day Route Choice of Traveler i

4.3. Bounded-Rationality Extensions

The Bush–Mosteller model for day-to-day route choice described in the previous section 4.2 already incorporates some aspects of bounded rationality through its stochastic route choice mechanism. Travelers select routes according to a probability distribution rather than always choosing the route with the lowest perceived travel time. However, the model forms perceptions solely from travel time and assumes a linear response to deviations from expectations.

To incorporate additional behavioral realism, two extensions are considered. First, risk-averse and waiting-time-averse travelers consider travel-time variability and waiting time when forming their perceptions. Second, a nonlinear mechanism is introduced to represent how travelers respond to deviations from their expectations. These extensions are related to the bounded rationality framework discussed in Section 3.5.

As mentioned in Section 3.5, the risk-averse and waiting-time-averse travellers capture the intuition of the third assumption of the bounded-rationality framework described in that section, according to which travellers do not always choose the optimal path. Here, optimality is understood as choosing the path with the shortest travel time. The nonlinear response mechanism captures the intuition of the fourth assumption, according to which travellers may not update their beliefs or switch to the shortest route when the difference in travel times is small.

4.3.1. Risk-averse and Waiting-time averse drivers

When choosing among the available routes, travelers may consider several characteristics, including travel time, travel-time variability, distance, monetary cost, waiting time, and other route-specific attributes. The relative importance of these characteristics depends on the context of the trip. For example, a driver travelling through the Pyrenees on a summer weekend may place considerable importance on the scenery of a route, even if this comes at the expense of a longer travel time. In contrast, the present thesis focuses on urban traffic during the morning and evening peak periods, which are predominantly associated with work-related trips (commuting). In this context, travel time, waiting time, distance and monetary cost are expected to be among the most relevant route characteristics and are often correlated. The extended model considers travel-time variability and waiting time as additional factors influencing drivers' route perceptions.

Travel-time variability is particularly relevant when considering commuting trips. [22] conducted a stated-preference survey of US commuters and found that travel-time variability has an important effect on route-to-work choice, with substantial differences across individuals. Similarly, [11] collected survey data in Paris and found differences in risk aversion across traveler groups, with greater risk aversion among transit users, blue-collar workers, and travelers making business-related trips. These findings support incorporating *travel-time variability* as an additional component of perceived route cost.

Humans are generally assumed to be averse to waiting. From an economic perspective, time spent waiting represents an opportunity cost, while from a psychological perspective, waiting may also generate additional costs, such as stress or anxiety. [32] developed a theoretical model showing that waiting time entails a psychological cost. [51] explored the perception of waiting time at signalized intersections using an online stated-preference survey, with results suggesting a quadratic relationship between perceived and actual waiting time. Similarly, [33] conducted an extensive user-perception questionnaire and videographic survey at 15 signalized intersections and found that perceived waiting time was higher than actual waiting time. These findings support incorporating *waiting time* as an additional component of perceived route cost.

To account for travel-time variability and waiting time, the baseline and route-specific perceived travel times are extended to form an *aspiration cost* and a *perceived cost*, respectively. The aspiration cost is defined as

$$(7) \quad AC_i^t = A_i^t + \theta_i \sigma_i^{t,\text{base}} + \phi_i WT_i^{t,\text{base}}$$

where A_i^t is the expected travel time defined in Eq. (4), $\sigma_i^{t,\text{base}}$ is the baseline standard deviation, and $WT_i^{t,\text{base}}$ is the baseline waiting time. The parameters $\theta_i \geq 0$ and $\phi_i \geq 0$ represent the risk sensitivity and waiting-time sensitivity, respectively. When $\theta_i = 0$ and $\phi_i = 0$, the original model (see Section 4.2) is recovered.

Similarly, the perceived cost of route k is defined as

$$(8) \quad PC_i^t(k) = \bar{T}_i^t(k) + \theta_i \sigma_i^t(k) + \phi_i WT_i^t(k),$$

where $\bar{T}_i^t(k)$ is the perceived travel time of route k , $\sigma_i^t(k)$ is the perceived standard deviation, and $WT_i^t(k)$ is the perceived waiting time.

For the baseline standard deviation (across all routes),

$$(9) \quad \sigma_i^{t,\text{base}} = \sqrt{\frac{\sum_{j=1}^{t-1} \gamma^{t-1-j} (T_i^j - A_i^t)^2}{\sum_{j=1}^{t-1} \gamma^{t-1-j}}},$$

The perceived standard deviation (route-specific),

$$(10) \quad \sigma_i^t(k) = \sqrt{\frac{\sum_{j=1}^t \gamma^{t-j} (T_i^j - \bar{T}_i^t(k))^2 \xi_{ik}^j}{\sum_{j=1}^t \gamma^{t-j} \xi_{ik}^j}},$$

The baseline waiting time (across all routes),

$$(11) \quad WT_i^{t,\text{base}} = \frac{\sum_{j=1}^{t-1} \gamma^{t-1-j} WT_i^j}{\sum_{j=1}^{t-1} \gamma^{t-1-j}},$$

The perceived waiting time (route-specific),

$$(12) \quad WT_i^t(k) = \frac{\sum_{j=1}^t \gamma^{t-j} WT_i^j \xi_{ik}^j}{\sum_{j=1}^t \gamma^{t-j} \xi_{ik}^j}.$$

These extensions affect the perceived costs used to evaluate routes, while leaving the Bush–Mosteller learning rule unchanged. Further justification for the formulation of the risk-averse and waiting-time-averse extensions is provided in Appendix B.

4.3.2. Nonlinear Stimulus

The literature provides evidence that travelers do not necessarily respond to small differences in route costs. [13] shows that small differences in utility may not affect travelers’ route choices and introduce an indifference band representing a threshold below which route switching does not occur.

Similarly, [17] analyze the stated route-choice preferences of 496 participants and find that travelers evaluate travel-time differences in relative rather than absolute terms. This finding is consistent with reference-dependent theory, in which outcomes are assessed relative to a reference point, and with the Weber–Fechner law, which suggests that perception depends more on relative than absolute changes [20]. The study also finds heterogeneous indifference bands across travelers. For the average participant, relative travel-time differences below approximately 31% fall within the estimated indifference band, indicating that differences of this magnitude do not necessarily affect route choice.

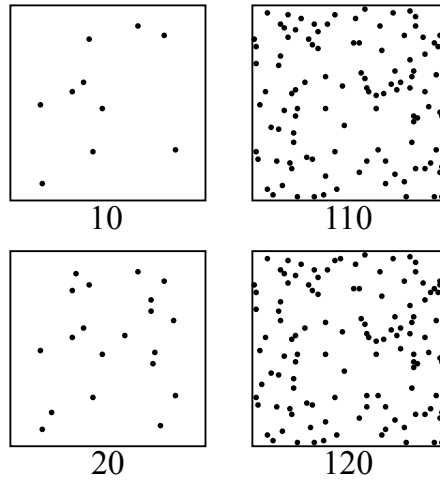


FIG. 15. Illustration of the Weber–Fechner law. Although the lower square contains 10 more dots than the upper square in both cases, the perceived difference is much more noticeable on the left than on the right. Reproduced from [50].

These findings motivate a nonlinear response in which small deviations between the perceived cost of a route $PC_i^t(k)$ and the baseline AC_i^t do not necessarily lead to a probability update, while larger deviations generate progressively stronger responses. Throughout this section, the term *perceived deviation* refers to the

difference between the aspiration cost and the perceived route cost, $AC_i^t - PC_i^t(k)$, which corresponds to the numerator in Eq. (3). The nonlinear transformation is applied to the *perceived deviation* before the stimulus is computed, rather than being applied to the stimulus within the learning update rule (2).

In particular, the stimulus in (3) does not directly represent how large the *perceived deviation* is. Due to the normalization, the stimulus instead indicates how the chosen route performs relative to the best or worst alternatives. Consequently, if the chosen route is the best alternative, it yields a stimulus of 1 regardless of whether the *perceived deviation* $AC_i^t - PC_i^t(c)$ is large or small. To prevent small *perceived deviations* from leading to a response, the nonlinear transformation introduces a dead zone.

It is important to understand that if the chosen route c is the best available route and its *perceived deviation* $AC_i^t - PC_i^t(c)$ exceeds the threshold, its stimulus will remain equal to 1, regardless of the magnitude of the *perceived deviation*. This behavior is not necessarily problematic, since the stimulus reflects how well the chosen route performs relative to the alternatives, rather than the magnitude of its *perceived deviation*. Under this interpretation, a route that is the best available alternative receives full positive reinforcement whether it is much better or only slightly better than the baseline.

Following the evidence that travelers evaluate travel-time differences in relative terms, the perceived deviation $AC_i^t - PC_i^t(k)$ is first expressed as a *relative margin*:

$$(13) \quad x_i^t(k) = \frac{AC_i^t - PC_i^t(k)}{AC_i^t}.$$

A nonlinear transformation is then applied to the relative margin:

$$(14) \quad g_i^t(k) = \text{sign}(x_i^t(k)) \left(\frac{\max(0, |x_i^t(k)| - \tau)}{1 - \tau + \epsilon} \right)^2,$$

where $g_i^t(k)$ is the transformed margin, $\tau \in [0, 1)$ is the threshold that defines the minimum *relative margin* required to trigger a response, and ϵ is a small numerical constant. Relative margins below this threshold are ignored, while larger ones produce increasingly strong responses through the quadratic transformation.

The stimulus is obtained by applying the same normalization as in the original formulation, Eq. (3), to the transformed margins in Eq. (14):

$$(15) \quad S_i^t(c) = \begin{cases} \frac{g_i^t(c)}{\max_k |g_i^t(k)|}, & g_i^t(c) \geq 0, \\ \frac{g_i^t(c)}{\min_k |g_i^t(k)|}, & g_i^t(c) < 0. \end{cases}$$

To illustrate the effect of the dead-zone, consider an aspiration cost of 100, with the chosen route having a perceived cost of 99 (1% lower than the aspiration cost), while another available route has a perceived cost of 105 (5% higher than the aspiration cost). With $\tau = 0.2 = 20\%$, both deviations fall within the threshold:

$$|x_i^t(c)| = 0.01 < 0.2, \quad |x_i^t(k)| = 0.05 < 0.2.$$

Consequently, both transformed margins are zero,

$$g_i^t(c) = g_i^t(k) = 0,$$

and the resulting stimulus is zero.

To illustrate the quadratic effect, consider an aspiration cost of 100 and a threshold of $\tau = 0.2$. Suppose the chosen route has a perceived cost of 75 (25% lower than the aspiration cost), while another available route has a perceived cost of 50 (50% lower than the aspiration cost). Under the original linear formulation, the stimulus associated with the chosen route would be

$$S = \frac{25}{50} = 0.5.$$

Under the nonlinear transformation, the corresponding transformed margins are

$$g(0.25) = \left(\frac{0.25 - 0.2}{0.8} \right)^2 = 0.0039$$

and

$$g(0.50) = \left(\frac{0.50 - 0.2}{0.8} \right)^2 = 0.1406.$$

Consequently, the stimulus becomes

$$S = \frac{0.0039}{0.1406} \approx 0.028.$$

Thus, the route that is only slightly above the threshold has a much smaller transformed margin and, consequently, a much smaller stimulus when chosen.

5. Experimental Design and Evaluation

This section first describes the implementation of the proposed MARL route-choice model, including its software architecture and the main design choices. It then presents the experimental design, covering the network used, the main characteristics of the simulation environment, the simulation output data considered for analysis, and the hyperparameters and their selected values. The evaluation metric used to assess whether the resulting network state corresponds to DUE is also introduced, together with a detailed explanation of its implementation. Finally, the experimental evaluation is presented, covering the research questions defined in the thesis objectives.

5.1. Implementation

This section describes the implementation of the MARL route choice model based on the Bush–Mosteller algorithm, applied to individual drivers within a Dynamic Traffic Assignment (DTA) setting, using the microscopic traffic simulator SUMO. All simulations were conducted using SUMO version 1.26.0.

First, a brief overview of the program logic is provided through a flowchart, followed by a discussion of the key design decisions made during the development of the program. The source code developed for this thesis is publicly available in the GitHub repository.

5.1.1. Software Architecture

The implementation is organised around a clear separation of concerns. *Demand generation* determines the origin, destination, and departure time of each driver. *Scenario* handles the static data used throughout the simulations, including the SUMO configuration file and the available routes for each OD pair, which remain unchanged across episodes. *Environment* runs each episode in the traffic simulator, while *agent* determines the route choice of each driver. *Orchestration* coordinates these components to run the complete training process. A detailed description of the functions associated with each component is provided in Appendix C.

Figure 16 illustrates the overall logic of the program. The execution begins by loading the configuration file containing the hyperparameter values. The program then creates the directories needed to store the files generated during the simulations and sets up MLflow, an open-source platform for experiment tracking and management [30].

Next, the demand is generated by defining the identifier, origin, destination, and departure time of each driver. Once the agents are defined, the set of available routes for each OD pair is determined, from which the agents select a route during each episode. The SUMO configuration file used for the simulations is also defined.

Subsequently, the `Environment` object is created, and each driver is initialized as a Bush–Mosteller agent. At each iteration (day), every agent samples one of its available routes according to its current probability distribution. A route file is then generated using the departure times and selected routes, and SUMO is executed using the SUMO configuration file and the generated route file. This approach avoids the overhead associated with TraCI, SUMO’s interface for interacting with a running simulation, and significantly improves computational efficiency.

Once the episode is completed, the resulting travel times and rewards are obtained. The rewards are used by the agents to update their policies. In addition, simulation outputs are stored. After each episode, the convergence of the MARL algorithm is checked. If the convergence criteria are met, training stops. Otherwise, another episode is run using the updated agent policies.

Once the training phase is complete, the collected data are exported to Parquet files, an open-source data file format designed for efficient data storage and retrieval and widely used in Big Data applications [3]. The final network state is then evaluated to assess DUE convergence. The corresponding results are compared with those obtained using the *dualIterate* algorithm. Finally, the results are logged to MLflow, and the program terminates.

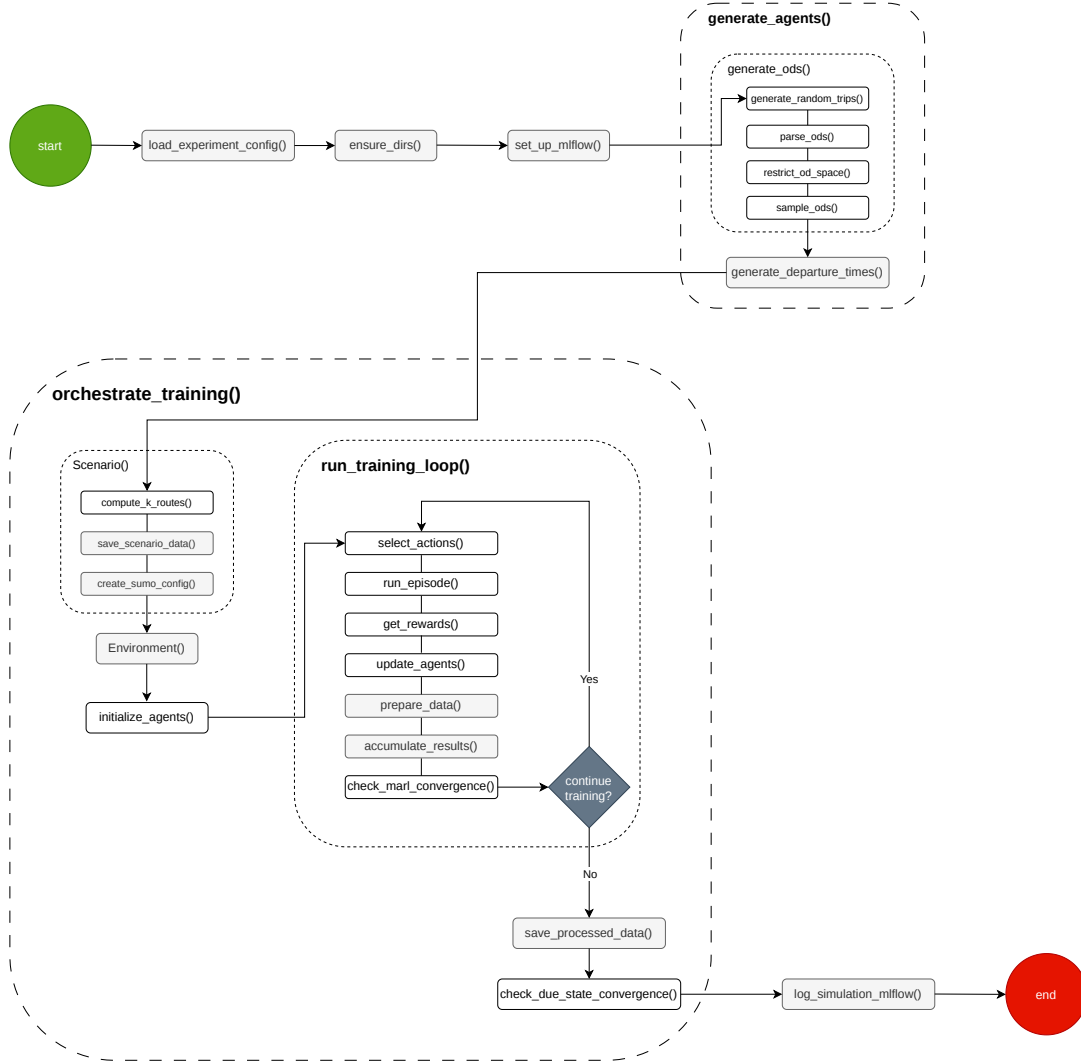


FIG. 16. Overall program logic of the MARL route choice implementation.

SUMO provides two main programs for running simulations. The first, **sumo**, is a command-line application designed for efficient batch execution of simulations. The second, **sumo-gui**, provides a graphical user interface (GUI) that allows the simulation to be visualized, including the network and vehicle movements.

For computational efficiency, most of the training is performed using **sumo**. However, selected episodes can be visualized using **sumo-gui**. In particular, visualizing the initial and final episodes provides a qualitative comparison of traffic conditions before and after training.

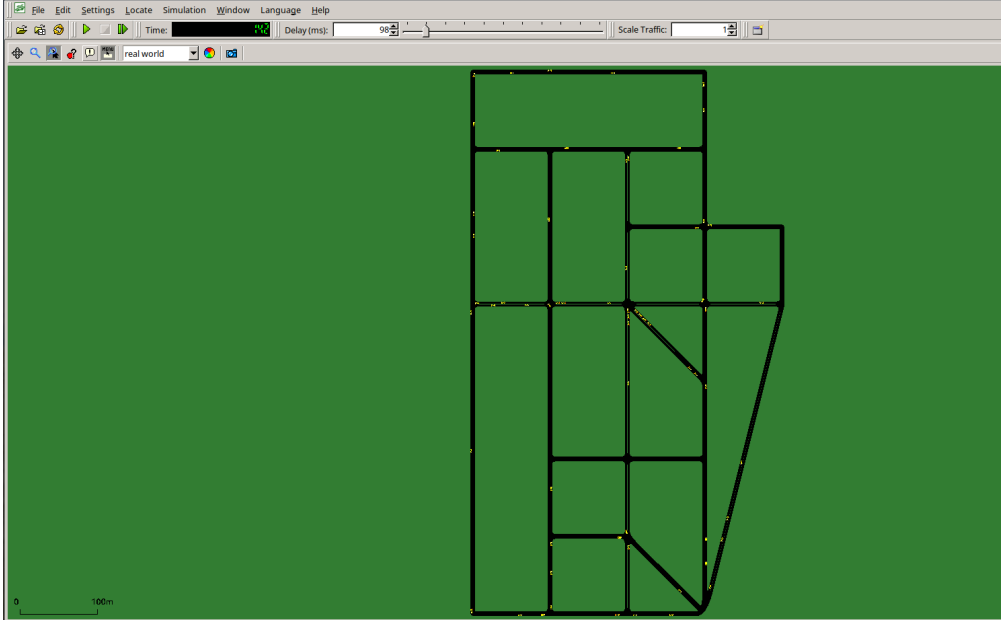


FIG. 17. Visualization of the SUMO simulation using `sumo-gui`.

5.1.2. Design Choices

Several design choices were made in the implementation of the program.

Convergence Criteria

Two distinct notions of convergence are considered in the proposed framework. First we consider the convergence of the resulting traffic assignment towards a Dynamic User Equilibrium (DUE). Convergence towards DUE is evaluated using the R-gap, as described in Section 5.3.4. Second, we have the convergence of the MARL algorithm. These two forms of convergence are distinct and do not necessarily occur simultaneously.

This section focuses on the convergence of the MARL algorithm. In RL, the policy of an agent defines the probability distribution over its available actions. In this case, the actions are the available routes. In this thesis, MARL algorithm is considered to have converged when *policy stability* is achieved. The training process stops when MARL convergence is achieved or when the maximum number of episodes is reached.

Policy stability can be assessed in different ways. In this thesis, it is evaluated by first measuring the change in each agent's policy between consecutive episodes. Since the policy of agent i is represented by a probability distribution over its available routes, the policy change between episodes $t - 1$ and t is quantified using the L_1 -norm:

$$\Delta p_i^{(t)} = \left\| p_i^{(t)} - p_i^{(t-1)} \right\|_1,$$

where $p_i^{(t)}$ denotes the route choice probability distribution of agent i at episode t . The L_1 -norm measures the total probability mass that is redistributed across the available routes. An agent is considered stable when its policy change falls below a predefined threshold.

Since multiple agents are involved, the mean policy change across all agents is used as the global convergence metric.

$$\Delta p_{\text{mean}}^{(t)} = \frac{1}{N} \sum_{i=1}^N \left\| p_i^{(t)} - p_i^{(t-1)} \right\|_1,$$

where N is the total number of agents.

The MARL algorithm is considered converged when the mean policy change remains below the predefined threshold for a fixed number of consecutive episodes. This prevents temporary small policy changes caused by the stochastic nature of the environment and learning process from being interpreted as convergence.

Quantifying the change in policies using the L_1 -norm, aggregating the changes across agents using the mean, defining the convergence threshold, and requiring stability over a fixed number of consecutive episodes are all design decisions in the proposed implementation. Other approaches could be considered. For example, the maximum policy change across agents could be used instead of the mean. However, this criterion was too restrictive in the proposed MARL scenario and prevented convergence.

Simulation Time

The simulation horizon is set to one hour to represent a peak-hour period of commuting travel, preceded by a 5-minute warm-up period used to remove the transient phase associated with starting the simulation from an empty network. Only agents departing during the post-warm-up period are considered in the analysis; agents departing during the warm-up period are excluded. The simulation time and warm-up period are defined as hyperparameters. The analysis used to determine the warm-up period is provided in Appendix 1.5.

A *time interval* is also used to discretize the simulation horizon into consecutive periods. In this study, a time interval of 30 seconds is used. This interval is mainly used for the computation of the R-gap and the time-dependent shortest paths (TDSP), as described in Section 5.3.4. The analysis used to determine this hyperparameter is provided in Appendix 1.8.

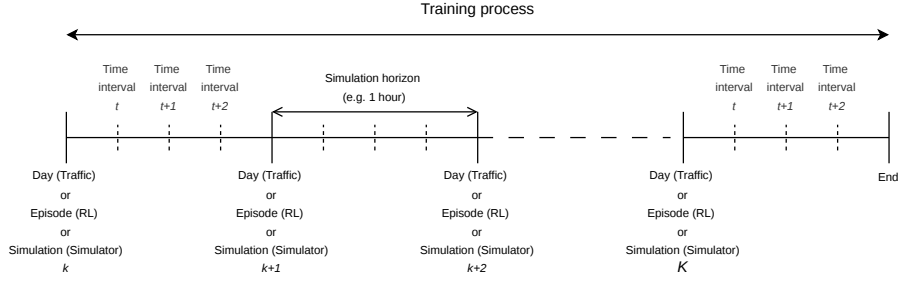


FIG. 18. Relationship between traffic days, reinforcement-learning episodes, simulations, simulation horizons, time intervals and training process

For clarity, Figure 18 shows the relationship between the different terms used throughout the thesis to describe the simulation and training process. In particular, *day*, *episode* and *simulation* refer to the same concept, with *day* used in the traffic context, *episode* in the reinforcement-learning context, and *simulation* from the simulator perspective. These terms refer to one simulation run. The simulation horizon represents the duration simulated within each run, which in this thesis consists of one hour plus a five-minute warm-up period. The simulation horizon is divided into smaller time intervals. The complete sequence of simulation runs is referred to as the *training process*.

Demand Modeling

Traffic assignment commonly follows a zone-based approach, in which a city is divided into multiple Traffic Analysis Zones (TAZ) to represent differences in land use and population characteristics. Trips are then defined using an origin zone and a destination zone. Each zone is typically represented by a centroid, with trips assumed to begin and end at these centroids.

In contrast, in this thesis, zones are not used and each trip is specified by an origin link and a destination link, consistent with the way trips are defined in SUMO.

Demand Generation

Demand is generated using the `randomTrips.py` tool, which produces the origin-destination (OD) pairs used in the simulations. These OD pairs are then reduced to a smaller set, from which agents sample. Departure times are randomly assigned within the simulation horizon. A detailed description of the demand generation procedure is provided in Appendix C.

Route Set Construction

The proposed RL framework uses a fixed set of routes generated before training. For each OD pair, a set of alternative routes is generated using `duarouter`. First, `duarouter` determines the shortest path based on the link costs. To obtain alternative paths, the link costs are randomly perturbed and the shortest-path computation is repeated. Different perturbations can therefore produce different paths, which are collected to form the route set. The magnitude of these perturbations is controlled by the `random_factor` hyperparameter. A detailed description of this process is provided in Appendix 1.2.

Bush–Mosteller Agents

Bush–Mosteller agents are initialized with a uniform probability distribution over their available routes.

A warm-up phase is also applied to the RL agents. During this phase, agents select routes at random and do not update their policies. The warm-up phase ends once each agent has visited all available routes at least once and a minimum number of episodes has elapsed.

This approach provides initial exploration and offers a simple way to ensure that agents have the required route perceptions before calculating (3).

Data Storage

Simulation outputs were stored using a flat tabular structure to facilitate analysis in R. This structure simplifies data manipulation within the `tidyverse` framework. The output datasets were stored in Apache Parquet format, an open-source data file format designed for efficient data storage and retrieval [3].

5.2. Randomness and Reproducibility

Several sources of randomness are present in the proposed framework. These include agents sampling the route to take at each episode from their policy; the generation of OD pairs, the sampling of agents from the restricted OD space, and the selection of departure times, as described in Section 5.1.2; the random perturbation of edge costs by `duarouter` when generating alternative paths, as described in Section 5.1.2; the internal simulation seed used by SUMO, which controls stochasticity during the simulation, including driver behavior and vehicle interactions; and the sampling of memory levels in the heterogeneous memory experiment, as described in Appendix 0.1.

To ensure reproducibility and assess the robustness of the results, multiple random seeds are used throughout the experiments.

5.3. Experimental Design

This section describes the main aspects of the experimental setup, including the network and characteristics of the simulation environment, the simulation outputs considered for analysis, the baseline hyperparameter configuration used in the experiments, and the evaluation metric used throughout the analysis. The implementation details of the latter are also described.

5.3.1. Network and Simulation Environment

The experimental network is based on the Sioux Falls network and was constructed using the SUMO network editor (NetEdit). Its moderate size and widespread use in traffic assignment research make it well suited for evaluating the proposed framework. The network consists of 24 nodes and 76 directed links. Each link consists of a single lane, resulting in 76 lanes in total, and the network contains 178 permitted turning movements. The network has a total length of 9.19 kilometres. Link lengths range from 81.07 m to 389.60 m, with a mean length of 120.89 m.

All vehicles in the simulation use the default SUMO vehicle class (Figure 14). Using a single vehicle class avoids introducing heterogeneity in vehicle characteristics, such as vehicle type and acceleration behavior, so that differences between travelers can be attributed primarily to their route-choice learning process.

To further isolate the learning process, the network and traffic environment were simplified. Accordingly, no roundabouts or traffic lights are included, each link consists of a single lane, and a uniform speed limit of 50km/h is imposed throughout the network. Furthermore, all network nodes are eligible to serve as origins and destinations.

Figure 19 illustrates the Sioux Falls network used in the experiments.

- **edgeData:** Provides aggregated information for individual links over predefined time intervals.

5.3.3. Hyperparameter Configuration

Three categories of hyperparameters are distinguished in this thesis:

- **RQ hyperparameters:** Hyperparameters whose effects are explicitly investigated in the research questions.
- **SA hyperparameters:** Hyperparameters that are not investigated in the research questions, but for which a sensitivity analysis was required to determine their value.
- **Non-SA hyperparameters:** Hyperparameters that are not investigated in the research questions and for which a sensitivity analysis was not necessary to determine their values.

Classical hyperparameter tuning techniques commonly used in deep learning, such as grid search or random search, were not applied in this thesis. In supervised learning, these methods typically evaluate different hyperparameter configurations and select the configuration that optimizes a metric, such as validation accuracy or loss. In the present work, however, the objective was not to optimize a single metric, but to obtain learning dynamics exhibiting several desirable properties simultaneously.

More specifically, those properties are relatively high R-gaps during the early learning episodes, predominantly positive R-gap values throughout the learning process and low R-gaps on last episodes. These requirements represent multiple behavioral criteria rather than a simple optimization objective.

Consequently, applying a classical hyperparameter tuning procedure would have required defining a scalar objective function that appropriately combines these criteria. In the absence of such an objective function, a grid or random search would generate multiple candidate configurations, but the final selection would still have to be made manually by assessing which configuration best satisfies the desired behavioral characteristics. Therefore an alternative approach based on exploratory experimentation was used to determine the hyperparameter values.

Table 1 presents the final values for the SA hyperparameters. The detailed procedure used to determine those values together with a discussion of Non-SA hyperparameters is provided in the Appendix D.

Hyperparameter	Final value
min_distance_factor	0.8
random_factor	50
n_routes_per_od	8
threshold_density	40 vehicles/km/lane
warm_up_time	300 seconds (5 min)
max_od_space	10
fixed_time_min	30 seconds (0.5 min)
congestion_regimes	Near free-flow : 1300 vehicles Moderate congestion: 1550 vehicles High congestion: 2000 vehicles

TABLE 1. Final values selected for the SA hyperparameters

Three representative congestion regimes are considered in the experiments: near-free-flow, moderate congestion, and high congestion. These regimes correspond to different demand levels and are used consistently throughout the subsequent analyses to evaluate the behavior of the proposed model under different traffic conditions.

5.3.4. Evaluation Metric: Relative Gap (R-gap)

In traffic assignment problems, the relative gap (R-gap) is commonly used to measure the convergence of the network state towards a Dynamic User Equilibrium (DUE). In this thesis, the system is considered to reach DUE when the R-gap remains below 5%. Intuitively, the R-gap measures the additional travel cost experienced by travelers relative to the cost that would be incurred if they used their corresponding time-dependent shortest paths.

The notation used in this section follows the convention adopted by [26].

Three formulations of the R-gap are considered, which differ in their level of aggregation. The first formulation (the most important), introduced by [24], corresponds to the global R-gap and aggregates the measure across all OD pairs and departure time intervals:

$$RGap^k = \frac{\sum_{o,d,t} \sum_{p \in P_{odt}^{k-1}} f_{odtp}^{k-1} (c_{odtp}^k - c_{ody_{odt}t}^k)}{\sum_{o,d,t} q_{odt} c_{ody_{odt}t}^k}$$

The second formulation, based on [29], is referred to as the *refined R-gap* and computes the R-gap for a specific departure time interval:

$$RGap_t^k = \frac{\sum_{o,d} \sum_{p \in P_{odt}^{k-1}} f_{odtp}^{k-1} (c_{odtp}^k - c_{ody_{odt}t}^k)}{\sum_{o,d} q_{odt} c_{ody_{odt}t}^k}$$

The third formulation computes the R-gap for a specific OD pair and is referred to as the *OD-specific R-gap* in this thesis.:

$$RGap_{od}^k = \frac{\sum_t \sum_{p \in P_{odt}^{k-1}} f_{odtp}^{k-1} (c_{odtp}^k - c_{ody_{odt}t}^k)}{\sum_t q_{odt} c_{ody_{odt}t}^k}$$

where:

- f_{odtp}^{k-1} denotes the flow assigned to path p for OD pair (o, d) during time interval t . The superscript $k - 1$ follows the notation commonly used in the DUE literature and represents the path flows used as input for episode k .
- P_{odt}^{k-1} denotes the set of paths used for OD pair (o, d) during time interval t .
- c_{odtp}^k denotes the travel cost experienced on path p during the simulation of episode k .
- $c_{ody_{odt}t}^k$ denotes the travel cost of the corresponding time-dependent shortest path for OD pair (o, d) during time interval t at episode k .
- q_{odt} is the travel demand associated with OD pair (o, d) during time interval t .
- y_{odt} denotes the time-dependent shortest path for OD pair (o, d) at time interval t .

Intuitively, the numerator represents the total additional travel cost incurred by travelers relative to the cost that would have been incurred if they used their corresponding time-dependent shortest paths. The denominator represents the total travel cost that would be incurred if all travelers used their corresponding time-dependent shortest paths.

The time-dependent shortest-path y_{odt} is determined from the traffic conditions generated during simulation k . Conceptually, it can be understood by considering the following question: given the traffic conditions observed during the episode k , if a traveler were to depart from o to d during time interval t , which route would provide the lowest travel cost? The resulting route is y_{odt} , and its travel cost is $c_{ody_{odt}t}^k$.

Implementation of the R-gap Computation

Although the mathematical formulation of the R-gap is relatively simple, its implementation required several steps.

Temporal Discretization

A table defining the temporal discretization of the simulation was first constructed. This table associates each time interval with its corresponding start and end times.

OD Demand by Time Interval

A second table containing the travel demand associated with each OD pair and time interval was constructed. This table stores the values q_{odt} introduced previously.

Path Flows

A table storing the number of travelers selecting each path for each OD pair and departure time interval was constructed for every simulation episode. This table provides the path flows f_{odtp}^{k-1} required for the R-gap computation.

Path Travel Times

A separate table was constructed to store the average travel time experienced on each path for every OD pair, departure time interval, and simulation episode. These travel times are calculated from the travelers who selected the corresponding path and whose departure times belong to the associated interval. The resulting values correspond to c_{odtp}^k .

Time-Dependent Link Costs

One of the most important structures required for the R-gap computation is the table containing the time-dependent travel costs of the links. This table stores, for every link, time interval, and episode, the average travel time required to traverse the link. These values are required as input for the time-dependent shortest-path (TDSP) computation.

The link travel times are calculated using the entry and exit times of vehicles traversing each link. For a given time interval, only vehicles entering the link during that interval are considered when calculating the corresponding average travel time. The `vehroutes` output provides the required link exit-time information.

For the subsequent TDSP computation, the link-cost table must contain the complete grid of combinations of links, time intervals, and simulation episodes. However, some combinations may have missing values because, in a given episode, no vehicle entered the corresponding link during a particular time interval. The strategy adopted to impute these missing values is described next.

Missing Value Imputation

The presence of missing values in the time-dependent link-cost table represents one of the main challenges in the R-gap computation. These missing values arise if no vehicle enters a particular link during the corresponding time interval.

Two main situations may explain the absence of observations for a given link and interval. First, the link may remain highly congested from previous intervals,

preventing new vehicles from entering during the current interval. Second, the link may simply be unused because no vehicles selected routes containing that link.

To distinguish between these two situations, the density information provided by the SUMO `edgeData` output was used. More specifically, for every link and interval with a missing travel-time value, the link density (measured in vehicles per kilometer) was examined.

If the density exceeds a predefined threshold, the missing value is interpreted as a consequence of congestion. In this case, a forward-fill strategy is applied, propagating the last observed average travel-time value for the corresponding link and episode.

Otherwise, if the density remains below the threshold, the link is assumed to be uncongested and unused. The missing value is therefore replaced by the free-flow travel time of the link, computed as the ratio between the link length and its maximum allowed speed.

After imputation, one XML file is generated per episode to provide the time-dependent link costs for TDSP computation with `duarouter`.

TDSP Computation with `duarouter`

The SUMO `duarouter` tool was used to solve the time-dependent shortest-path (TDSP) problem and obtain the quantities y_{odt} and $c_{ody_{odt}t}^k$ introduced previously.

For each episode, `duarouter` requires a trips file containing the relevant combinations of origin, destination, and departure time, together with a time-dependent link-costs file containing the travel costs associated with each link for every time interval.

A limitation that should be acknowledged is that the time-dependent link-cost file may violate the FIFO assumption. The FIFO (First-In-First-Out) property states that if two vehicles enter a link, the first vehicle should leave the link earlier than the second vehicle. However, FIFO violations may occur because of the temporal discretisation. An example may help illustrate this property: If the travel time of an link is 10s at time t and 2s at time $t + 1$, a vehicle entering at the end of time t would spend 10s traversing the link, whereas a vehicle entering the link at the beginning of time $t + 1$ would spend only 2s. Consequently, the vehicle entering later would exit the link first, violating the FIFO property. Most time-dependent shortest-path algorithms assume the FIFO property. In this case, particularly under high congestion, the R-gap showed the expected behavior, indicating that the impact of FIFO violations is limited.

Time-Dependent Shortest-Path Costs

Once the TDSP costs c_{odt}^k are obtained, they are combined with the previously described path-flow, path-travel-time, and OD-demand data to compute the R-gap.

Negative R-gap Values

Although the R-gap is theoretically non-negative, negative values can occur in the present implementation because c_{odt}^k are estimated from a computed time-dependent link-cost table rather than from the true time-dependent network conditions.

The table is constructed using after doing temporal discretization and imputing values where observations are missing. The obtained time-dependent shortest-path cost are an approximation of the true values. Because of temporal discretization all vehicles departing within the same interval are assumed to have the same time-dependent shortest path. However, this assumption may not hold because traffic conditions can change within the interval. For example, a vehicle leaving at 8:02 may have a different optimal route than one leaving at 8:09. Consequently, the estimated time-dependent shortest-path cost can be higher than the travel time experienced by some vehicles, resulting in a negative contribution to the R-gap.

To illustrate it with an example, consider two vehicles departing within the same time interval. Suppose their true time-dependent shortest-path costs are 20 and 60 minutes, respectively. Due to temporal aggregation, the estimated shortest-path cost for the interval may be 45 minutes. The two vehicles then contribute

$$(20 - 45) + (60 - 45) = -10$$

to the numerator of the R-gap, despite both having travelled on their respective true shortest paths.

Additional Implementation Details

Vehicles departing close to the end of the simulation horizon may continue traveling beyond the last defined time interval. In such cases, when solving the TDSP problem, `duarouter` uses free-flow travel times for future intervals.

The R-gap was computed for both the proposed Bush–Mosteller model and the `duaIterate.py` benchmark. Since the benchmark is not guaranteed to converge to DUE for every network and demand configuration, its convergence was first verified before evaluating the proposed model.

5.4. Experimental Evaluation

This section addresses the research questions defined in the thesis objectives. The evaluation begins by analysing the rational-agent case and comparing the proposed approach with the `duaIterate` benchmark. It then examines the sensitivity of the model to behavioural parameters already incorporated in the original formulation, namely memory and learning rate, as well as its ability to recover from a temporal link disruption. Finally, the evaluation considers the impact of the bounded-rationality extensions introduced in the proposed model, including risk aversion, waiting-time aversion, and nonlinear response.

5.4.1. Rational Agents

Research question: Can a MARL approach based on the Bush–Mosteller algorithm, implemented within a DTA setting, converge towards a DUE state under the assumption of rational drivers?

In this thesis, the term *rational driver* refers specifically to a Bush–Mosteller driver with perfect memory, meaning that past travel experiences are not discounted when forming route perceptions (see Section 3.5).

As a secondary analysis, the R-gap across departure intervals and OD pairs is also examined.

Experimental setup: The experiment uses the baseline hyperparameter configuration described in Section D, with the memory level set to 1 and the learning rate set to 0.3. The complete configuration file used for the experiment is provided in Appendix 1.

Results and Interpretation:

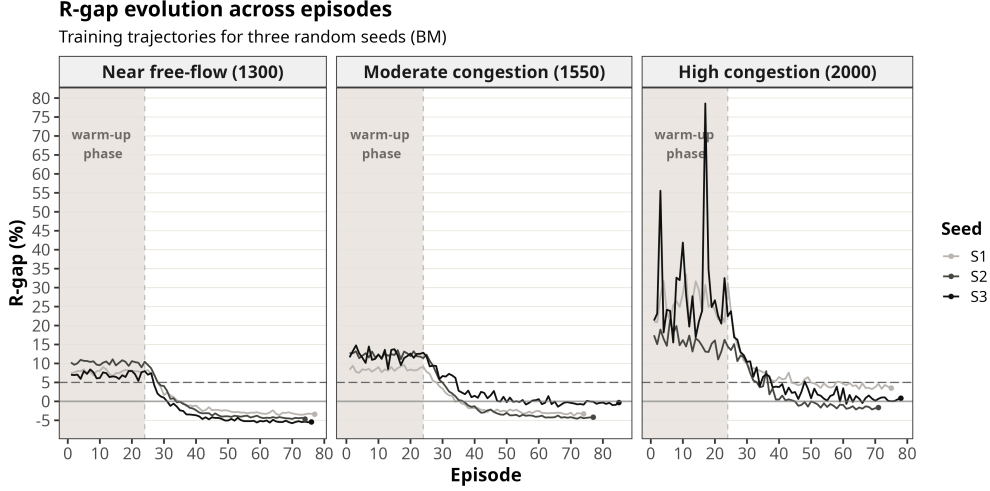


FIG. 20. R-gap evolution across training episodes for three representative random seeds under near free-flow, moderate congestion, and high congestion. The shaded region denotes the warm-up phase preceding learning.

Figure 20 shows the R-gap evolution across episodes for the three congestion levels and three random seeds. The shaded area represents the 24-episode warm-up phase, after which learning begins. The dashed line indicates the convergence threshold for DUE, which is set at 5%, a commonly used threshold in the traffic assignment literature.

After learning begins the R-gap decreases across all three congestion levels, indicating convergence towards DUE.

Negative R-gap values can be observed during the learning phase, particularly under the near-free flow and moderate-congestion scenarios. As discussed in Section 5.3.4, these values can result from the approximation of the time-dependent shortest-path costs c_{odt}^k and do not necessarily indicate a deviation from DUE. Interestingly, negative R-gap values are less frequent under high congestion. This may be related to the higher number of vehicles, which can result in more accurate estimates of the time-dependent link costs and reduce the need for missing-value imputation.

Under near free-flow conditions (1300), all three seeds converge rapidly. Convergence is slightly slower under moderate congestion (1550), while the high-congestion scenario (2000) requires more episodes to converge. This scenario also exhibits larger fluctuations during the learning phase.

The trajectories also show that once the R-gap reaches the DUE region, it remains relatively stable for the remainder of the simulation. The largest differences between seeds occur under high congestion. Overall, the results indicate that increasing congestion makes the route-choice problem more difficult, as reflected by

higher initial R-gaps, slower convergence, and greater variability across random seeds.

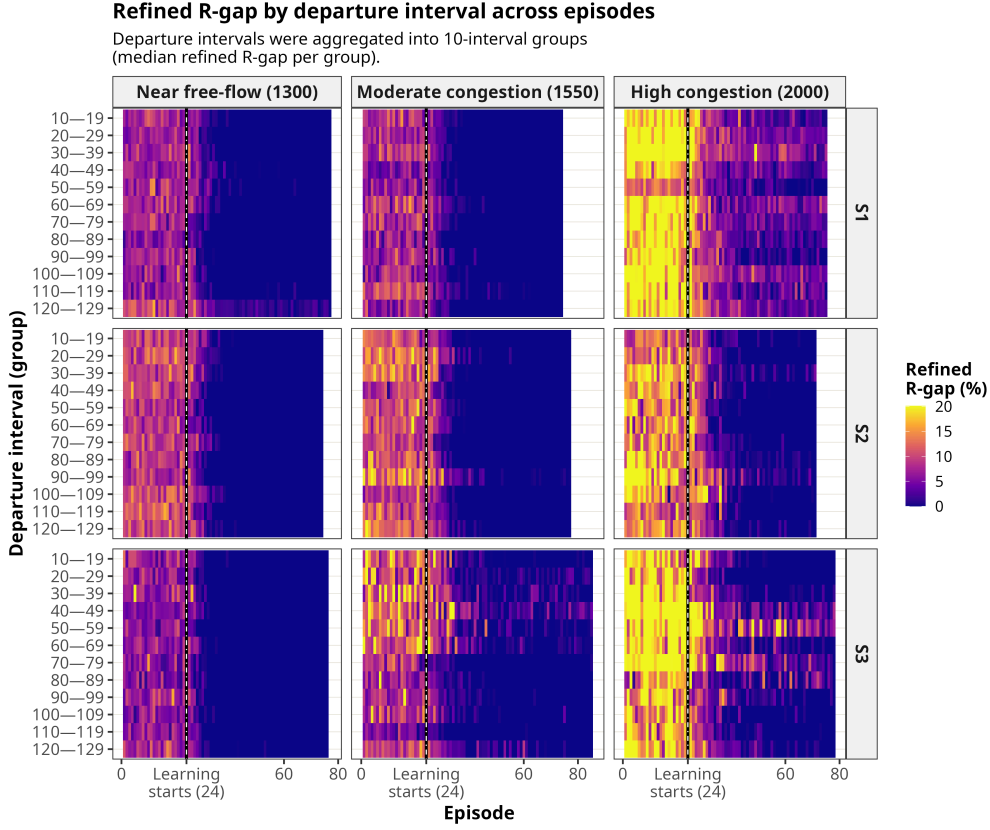


FIG. 21. Refined R-gap across departure intervals and training episodes for three representative random seeds under different congestion regimes. Departure intervals are aggregated into 10-interval groups; the vertical marker indicates the end of the warm-up phase.

Figure 21 shows the evolution of the refined R-gap across departure intervals under three congestion regimes. The refined R-gap is the R-gap calculated for a specific departure time interval, as defined in Section 5.3.4. For visualization, the departure time intervals are aggregated into groups of 10. The first 24 episodes constitute a warm-up phase during which the agents explore without learning, with learning beginning at episode 25. The refined R-gap for each group decreases after the start of learning in all three congestion regimes.

In conventional traffic assignment approaches, later departure intervals typically take longer to converge, which can result in a diagonal pattern in the heatmaps. This pattern is not observed in the proposed agent-based approach. Instead, most departure intervals improve at roughly the same time, suggesting that convergence does not systematically take longer for later intervals.

The heatmaps do not show a strong diagonal pattern across departure intervals. Such a pattern would indicate that later intervals take longer to converge than earlier ones. Instead, most intervals improve at roughly the same time.

It can be easily visualized that higher congestion results in higher refined R-gap values, visible as brighter bands in the heatmaps. The high-congestion scenario also shows more variability across intervals during the learning phase.

Some intervals become unstable after initially converging. This is observed in the moderate-congestion scenario and occurs more frequently under high congestion, particularly for seeds 2 and 3.

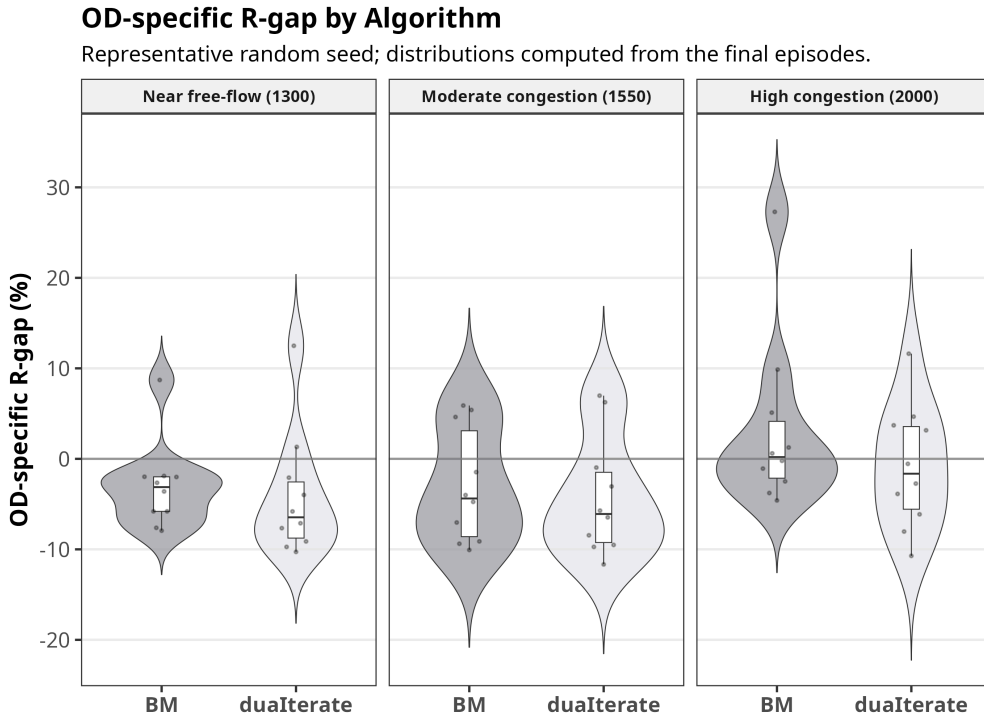


FIG. 22. Distribution of final episode OD-specific R-gap by algorithm and congestion regime. Distributions are computed from the final episode for one random seed, with each observation corresponding to the R-gap of an origin–destination pair.

Figure 22 makes it possible to assess whether a small overall R-gap is accompanied by consistently good equilibrium quality across all OD pairs. The figure combines violin plots with the underlying observations, where each point represents the OD-specific R-gap computed for an OD pair (see Section 5.3.4), allowing the distribution and individual values to be examined simultaneously. The figure shows that both algorithms generally achieve good equilibrium quality across the 10 OD pairs considered in the OD matrix for this experiment.

As mentioned before, negative values can be observed, particularly under the near-free-flow and moderate-congestion scenarios. Under the high-congestion scenario, the OD-specific R-gap values are generally closer to zero, although negative values are still observed.

Overall, the results show that the proposed MARL approach based on the Bush–Mosteller algorithm can converge towards a DUE state within a DTA setting under the assumption of rational drivers with perfect memory. Another implication is that MARL can provide an alternative approach for solving for DUE, where equilibrium emerges through the adaptation of individual drivers rather than through a conventional iterative assignment procedure.

Previous work by Wei et al. [49] showed that a MARL approach based on the Bush–Mosteller algorithm can converge towards a UE when drivers have perfect memory. Their analysis was conducted in a STA setting using link performance functions to determine travel times. The present study extends this investigation to a DTA setting using a microscopic traffic simulator. Furthermore, this study introduces the use of the Relative Gap (R-gap) metric to evaluate convergence towards equilibrium.

5.4.2. Spatial Traffic Comparison

Research question: How does the spatial traffic distribution produced by the BM model compare with that of `duaIterate`?

Similar aggregate network performance does not necessarily imply that traffic is distributed across the network in the same way. Comparing the two approaches at the link and path levels provides insight into whether they produce similar spatial traffic patterns and how traffic is distributed across alternative routes.

The comparison is primarily qualitative, using the final episode of the BM model and the final iteration of `duaIterate`.

Experimental setup The experiment uses the baseline hyperparameter configuration described in Section D, with the memory level set to 1 and the learning rate set to 0.3. The complete configuration file used for the experiment is provided in Appendix 2.

Results and Interpretation:

The qualitative comparison of link flows shows that both algorithms produce similar spatial traffic distributions. The detailed link-level analysis is reported in Appendix 0.2.

Path-level route composition for a representative OD

Top 3 highest-flow paths per algorithm
 OD pair: E1_2_WB → E20_22_SEB | Demand: 225 veh

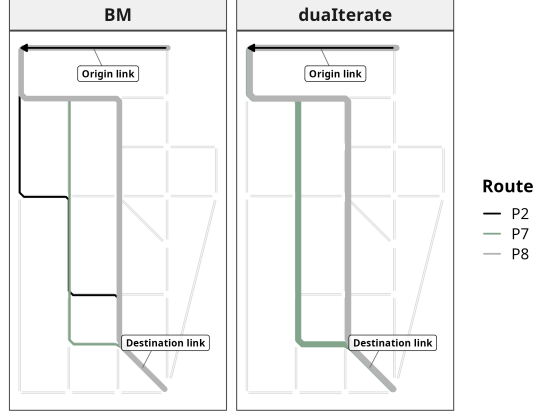


FIG. 23. Three highest-flow routes for a representative origin–destination pair under the proposed Bush–Mosteller MARL algorithm and the `duaIterate` tool, for one random seed.

TABLE 2. Percentage of traffic assigned to each route under the proposed Bush–Mosteller MARL algorithm and the `duaIterate` tool for a representative OD pair shown in Figure 23.

Path	BM share	duaIterate share
P8	44.9%	50.7%
P7	14.7%	49.3%
P2	10.7%	0.0%
P1	3.6%	0.0%
P4	10.2%	0.0%
P3	9.8%	0.0%
P5	4.4%	0.0%
P6	1.8%	0.0%

The path-level comparison, however, reveals a difference in route-flow concentration: BM tends to distribute traffic across a larger number of routes, whereas `duaIterate` concentrates traffic on fewer routes. This difference is illustrated in Figure 23 and Table 2. The same pattern is also observed for the other OD pairs analysed, which are reported in Appendix 0.2. The more dispersed route-flow distribution observed for BM is consistent with bounded rationality framework (see Section 3.5), under which drivers may not always select the shortest path. Under this assumption, traffic can be expected to be distributed across multiple alternative routes rather than concentrated exclusively on the shortest ones.

These results show that similar link flows can result from different path-flow distributions.

5.4.3. Memory Sensitivity

Research question: Does reducing the memory level of agents prevent the BM model from converging towards a DUE state, and how does it affect route-choice behavior?

The memory parameter determines how strongly past travel experiences are taken into account when forming perceptions about routes. With perfect memory, all observations have the same weight. As memory decreases, the agent relies more on recent observations and gradually forgets older experiences.

Investigating memory is important because perfect memory is an unrealistic assumption (see Section 3.5). The question is therefore whether DUE can still emerge under more realistic drivers with limited memory.

Experimental setup: The experiment uses the baseline hyperparameter configuration described in Section D, with the learning rate set to 0.3, and performs a sensitivity analysis over the memory level. The complete configuration file used for the experiment is provided in Appendix 3.

Results and Interpretation: Figure 24 shows the evolution of the R-gap for different memory levels. No clear increase in oscillatory behavior is observed as memory decreases. Across all three congestion regimes, the different memory levels eventually reach a DUE state. The results are also broadly consistent across random seeds.

However, the memory level has a clearer effect on the convergence of the MARL algorithm, that is measured by the mean policy change of the agents (see Section 5.1.2). Higher memory levels lead to earlier stabilization of the agents' policies, as indicated by the trajectories ending before episode 150. In contrast, lower memory levels prevent convergence of the MARL algorithm, resulting in continued adaptation, with the trajectories continuing until the maximum number of episodes, 150.

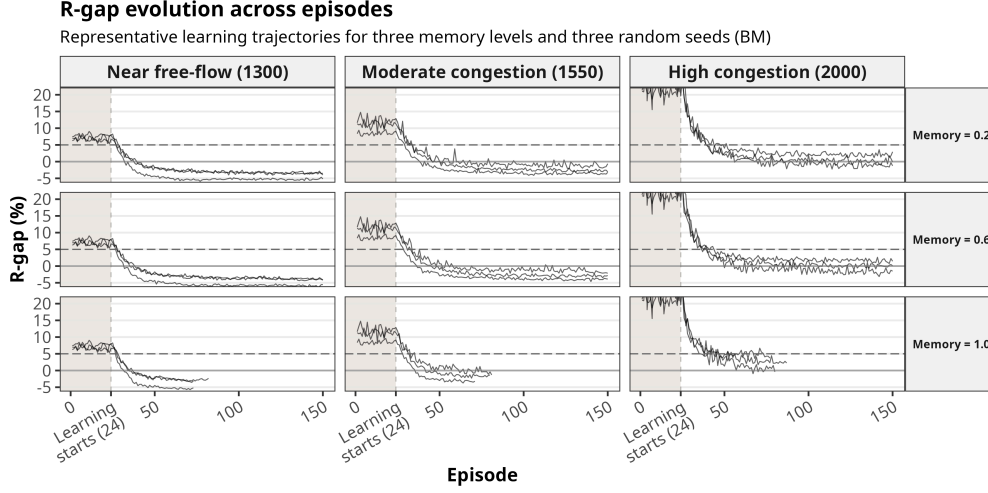


FIG. 24. R-gap trajectories for three memory levels under near free-flow, moderate, and high demand congestion regimes. Each panel shows trajectories from three random seeds; the shaded region indicates the warm-up period before learning starts (episode 24), and the dashed horizontal line marks the 5% R-gap level.

Figure 25 and Figure 26 illustrate the impact of memory on the learning process of a representative agent under high congestion, using memory levels 1 and 0.1, respectively. To understand these figures, a basic understanding of the Bush–Mosteller algorithm and its main quantities and equations is required (see Section 2).

The annotation box in each facet shows the selected route and its experienced travel time, the expected travel time (ET), stimulus, and route-choice probabilities at different episodes. For example, $P1 .33 \rightarrow .23 \quad 66s$ indicates that the probability of choosing P1 (path 1) decreases from 0.33 to 0.23 during the agent’s policy update, while 66 s indicates that the perceived travel time for P1 after that episode was 66 s.

Evolution of Route Choice During Learning

Agent 1009 | Departure time: 2011s | Memory level = 1

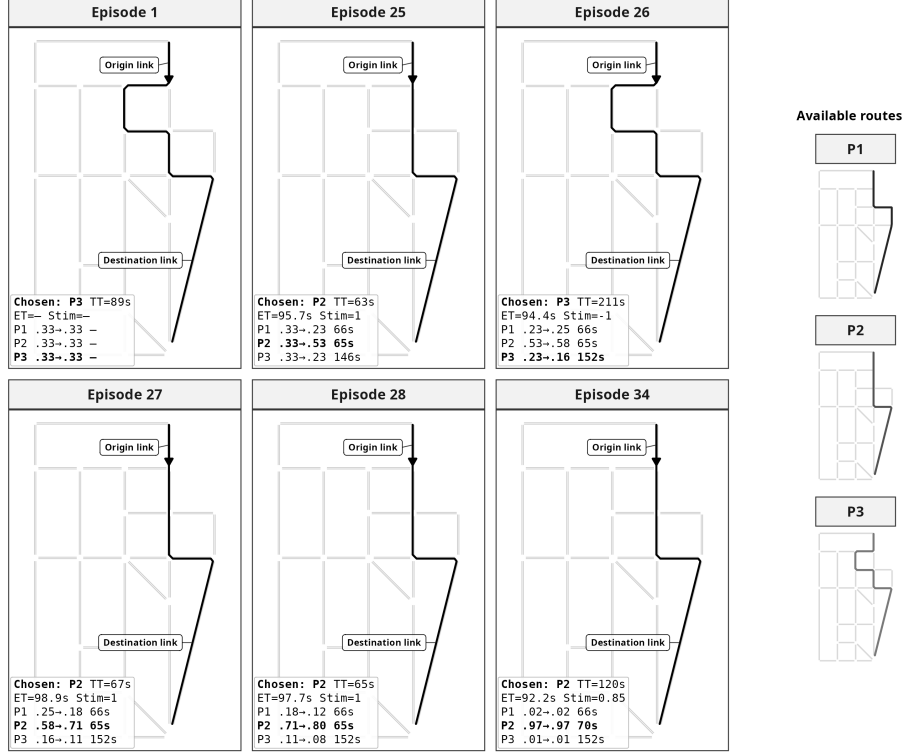


FIG. 25. Evolution of route choice for a representative agent with memory level 1 under a high congestion scenario. The panels show the selected route and an annotation box containing the ID of the chosen route and the corresponding experienced travel time, expected travel time (ET), stimulus, route-choice probabilities, and perceived travel times (PT) of each route for that episode. The right sidebar shows the routes available to the representative agent.

Figure 25 provides an example of the learning process for an agent with memory level 1. Learning starts at episode 25, after the 24-episode warm-up phase. In episode 25, the agent samples P2 and experiences a travel time of 63 s. This observation is used to update the perceived travel time of P2, resulting in a value of 65 s. The expected travel time is 95 s, which remains relatively high because the agent explored all three routes during the warm-up phase, including P3, which produced substantially higher travel times. With memory level 1, these past P3 experiences are retained with equal weight, inflating the expected travel time.

Since the PT of P2 is substantially lower than the ET, the resulting stimulus is positive. After normalization, the stimulus reaches 1 because P2 is also the optimal route. The agent therefore strongly reinforces P2, increasing its probability of being selected in the following episode while reducing the probabilities of the other routes.

In episode 26, the agent instead samples P3 and experiences a travel time of 211 s. Its PT increases to 152 s, which is substantially higher than the ET of 94.4 s, leading to a negative stimulus. After normalization, the stimulus reaches -1 because P3 is the worst route, and the agent therefore strongly penalizes P3. In episode 27, P2 is sampled again and produces a travel time of 67 s. Its perceived travel time remains low relative to the expected travel time, resulting in another stimulus of 1 and further reinforcement of P2. A similar pattern occurs in episode 28, after which the probability of selecting P2 becomes dominant.

Episode 34 illustrates an important consequence of the agent’s memory. Although the agent experiences a relatively high travel time of 120 s on P2, its perceived travel time only increases to 70 s because the perception incorporates the previous experiences of P2. Since this value remains below the ET of 92.2 s, the stimulus is still positive (0.85), and P2 is not penalized. This illustrates the role of high memory in stabilizing route-choice behavior. Because past experiences continue to influence the perceived travel time, a single poor experience on P2 does not substantially change its perception or lead to a strong reduction in its selection probability.

At this point, the PT of P1 (66 s) is actually lower than that of P2 (70 s), yet the probability of selecting P2 remains very high at 0.97. Moreover, stochasticity also plays a role, since the agent does not simply choose the route with the lowest perceived travel time, but instead selects a route by sampling from its probability distribution. In this case, the repeated reinforcement of P2 eventually causes its selection probability to reach 1, even though P1 has a lower perceived travel time.

Agent 1009 | Departure time: 2011s | Memory level = 0.1

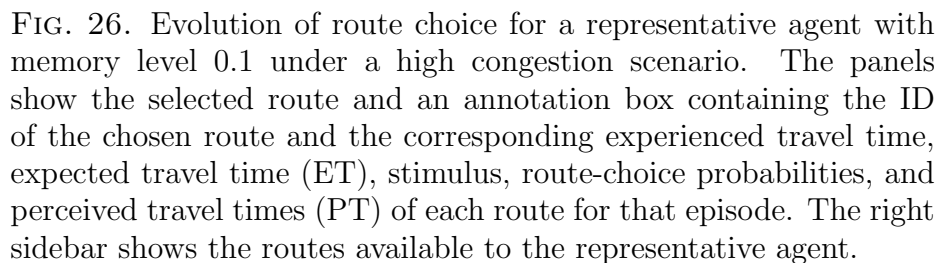


FIG. 26. Evolution of route choice for a representative agent with memory level 0.1 under a high congestion scenario. The panels show the selected route and an annotation box containing the ID of the chosen route and the corresponding experienced travel time, expected travel time (ET), stimulus, route-choice probabilities, and perceived travel times (PT) of each route for that episode. The right sidebar shows the routes available to the representative agent.

Figure 26 provides an example of the learning process for an agent with memory level 0.1. With this memory level, the most recent experience receives a weight of 1, the previous experience a weight of 0.1, with the weights decreasing exponentially for older experiences. Consequently, the perceived travel times are mainly determined by the most recent experience.

Episode 1 is omitted here because it provides limited insight and was already included for illustrative purposes in Figure 25. Learning starts in Episode 25, where the agent selects P2 and experiences a travel time of 63 s. This observation is used to update the PT of P2, resulting in a value lower than the ET of 72.5 s. Consequently, a positive stimulus is generated. Since P2 also has the lowest PT among the available routes, after normalization the stimulus reaches 1. P2 is therefore strongly reinforced.

In Episode 27, P2 is selected again and a travel time of 63 s is experienced. The PT of P2 is updated using this observation, resulting in a value lower than the ET and, consequently, a positive stimulus. Since P2 remains the optimal route, the normalized stimulus reaches 1, and P2 is strongly reinforced.

Episode 46 illustrates the effect of the low memory level. At the beginning of the episode, almost all probability mass is concentrated on P2. The agent selects P2 but experiences a considerably higher travel time of 82 s. Due to the low memory level, the most recent observation has a dominant influence on its PT. Consequently, the PT of P2 increases to approximately 80 s. In contrast, the ET is 64.5 s, reflecting primarily the travel time observed in the previous episode. Since the PT of P2 is now higher than the ET, a negative stimulus is generated. As P2 is not the worst route, the stimulus after normalization is -0.31, resulting in a penalization of P2.

This behaviour contrasts with that observed for memory level 1. When past experiences are equally weighted, an individual poor observation has a limited effect because each new experience contributes only $1/N$ to the average, where N is the number of accumulated experiences. Thus, as the number of accumulated experiences increases, the effect of a new observation on the PT becomes increasingly smoothed by the previous experiences.

Consequently, for memory level 1 the influence of a relatively poor travel time on the PT of the currently most likely route is limited, as illustrated in Episode 34 of Figure 25, where a poor travel time observation for P2 did not result in its penalization. With a memory level of 0.1, however, a poor observation on the currently most likely route can substantially increase its perceived travel time, leading to its penalization.

In Episode 59, P1 is selected despite P2 having almost all the probability mass, as routes are chosen stochastically by sampling from the choice probabilities. The experienced travel time on P1 is 66 s, which is used to update its PT, resulting in a value lower than the ET and therefore producing a positive stimulus. P1 also has the lowest PT; hence, after normalization, the stimulus is equal to 1, resulting in strong reinforcement of P1.

In Episode 61, P1 is sampled again and produces a travel time of 64 s. Its perceived travel time remains low relative to the expected travel time, resulting in another stimulus of 1 and further reinforcement of P1. The probability of selecting P1 consequently increases from 0.32 to 0.52. In subsequent episodes, the probability of selecting P1 continues to increase, eventually becoming dominant.

Episode 135 provides a similar example to Episode 46, in which a poor observation on the currently most likely route substantially increases its perceived travel time, resulting in its penalization. This further illustrates the greater responsiveness of low-memory agents to recent travel experiences.

Overall, Figures 25 and 26 show that, for this network and demand configuration, the agent can distinguish between attractive and clearly unattractive routes regardless of the memory level. In both cases, P3 is identified as unattractive, and the agent ultimately concentrates its probability on an attractive route. Which attractive route is ultimately selected is influenced by the stochasticity of the learning process. Consequently, all agents ultimately concentrate their choice probabilities on near-optimal routes, allowing the network to reach a DUE state, as shown in Figure 24. The following paragraphs discuss the specific characteristics of the network and demand configuration that contribute to this behavior and explain why the influence of memory may be more pronounced under different conditions.

The main network and demand characteristics affecting the learning algorithm and the impact of memory are the presence of near-optimal routes and the few occurrences of cost-rank flipping between episodes. When one route is consistently significantly worse than the alternatives, the algorithm can identify and avoid that route regardless of the memory level. Likewise, when one route is consistently superior to the alternatives and its cost ranking does not change over episodes, the algorithm can identify it and concentrate most of its probability mass on it. Therefore, for this network and demand configuration, the absence of cost-rank flipping allows the algorithm to identify the optimal or near-optimal routes regardless of the memory level, resulting in a DUE state for all memory levels considered.

The more relevant case for traffic assignment arises when cost-rank flipping occurs between episodes. For example, a shorter route may be more attractive under low congestion, but as more vehicles choose it, its travel time can increase and make a longer route more attractive. Under these conditions, memory may play a more important role in the convergence of the network to DUE, and lower memory levels may prevent the network from reaching DUE.

The simplified scenario considered in this analysis—characterized by the absence of traffic lights, a single lane, grid network, homogeneous vehicle-specific characteristics such as vehicle length, maximum speed, and homogeneous driver-behaviour parameters such as impatience and cooperation, and an OD matrix with a limited number of OD pairs—makes it more difficult to create a sufficiently rich environment in which route-cost rankings flip between episodes.

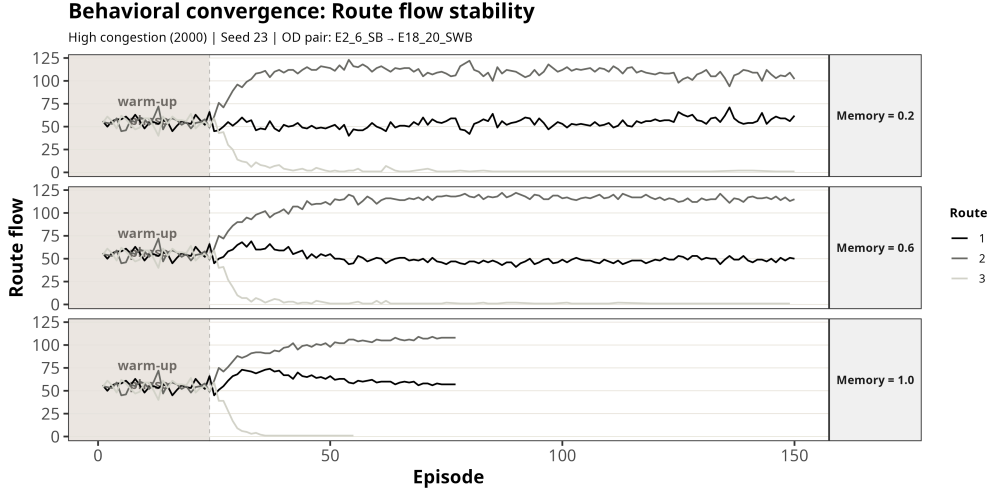


FIG. 27. Route flow evolution across episodes for different memory levels under a high congestion regime. The plot shows the flow of the available routes for a representative origin–destination pair and random seed. The shaded area indicates the 24-episode warm-up phase.

Figure 27 shows the route-flow stability for the same OD pair and scenario used in Figures 25 and 26. As observed in the learning process, lower memory levels make the agent more responsive to new observations. For example, a poor travel-time observation on the most likely route can substantially reduce its choice probability and shift probability towards other routes. The greater responsiveness to recent observations at lower memory levels therefore results in more frequent changes in route choice and greater variability in route flows. In contrast, with higher memory levels, agents tend to stabilise their route choices earlier.

The qualitative comparison is considered sufficient to assess the differences in route-flow variability, and therefore no separate quantitative metric is considered. If required, route-flow variability could be quantified using a metric such as the flow standard deviation (FSD), as used by Wei et al. [49].

Overall, reducing the memory level increases route-flow variability and makes agents more responsive to recent travel-time observations. However, in the synthetic network and demand configuration considered, all memory levels tested still lead to convergence towards DUE. A stronger influence of memory on DUE convergence may arise in settings with more frequent cost-rank flipping between routes.

Previous work by Wei et al. [49] showed that reducing the memory level of Bush–Mosteller agents in a MARL approach prevented convergence towards UE. Their study used an STA setting in which cost-rank flipping occurred between episodes. They also found that lower memory levels resulted in greater route-flow variability.

This thesis also performed an additional analysis in which heterogeneous memory levels across agents was considered. For the tested scenario, no meaningful differences were observed between homogeneous and heterogeneous populations with the same mean memory level. This suggests that, under these conditions, the population mean memory level has a greater influence than how memory is distributed across agents. The analysis is reported in Appendix 0.1.

5.4.4. Disruption Recovery

Research question: How does the implemented MARL Bush–Mosteller route-choice model adapt to a network disruption, and can it recover the initial equilibrium traffic state once the network is restored?

The main objective is to assess the adaptability and recovery of the proposed route-choice model under temporary network disruptions. Real transportation networks are subject to temporary disruptions such as road works or reductions in infrastructure capacity. A realistic model should therefore be able to adapt when network infrastructure conditions change and recover when the disruption disappears.

Experimental setup: The experiment uses most of the baseline hyperparameter configuration described in Section D, with the learning rate set to 0.3 and several memory levels tested. Table 3 summarizes the hyperparameter modifications introduced for this research question.

TABLE 3. Specific experimental settings for the disruption recovery analysis.

Hyperparameter	Value
max_episodes	450
k_no_change	5000
degradation_start_episode	151
degradation_end_episode	301
network_normal	Normal_Sioux_Falls.net.xml
network_degraded	10km.h.Degraded_10-11_10-15_Sioux_Falls.net.xml

A total of 450 episodes are simulated. The first 150 episodes are conducted under normal network conditions, followed by 150 episodes in which the network is affected by the disruption. In the final 150 episodes, the network is restored to its normal conditions. The stopping criterion of the MARL algorithm is disabled to ensure that all 450 episodes are simulated.

The complete configuration file used for the experiment is provided in Appendix 4.

Figure 28 shows the links affected by the disruption in the network. In particular, the disruption is simulated by reducing their maximum speed to 10 km/h to represent road maintenance, compared with the normal speed limit of 50 km/h on the remaining links.

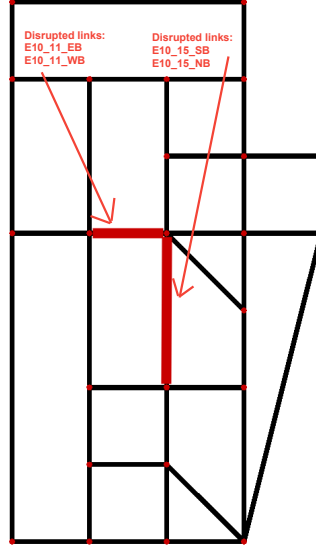


FIG. 28. Network links affected by the temporary speed reduction to 10 km/h to simulate road maintenance.

Results and Interpretation:

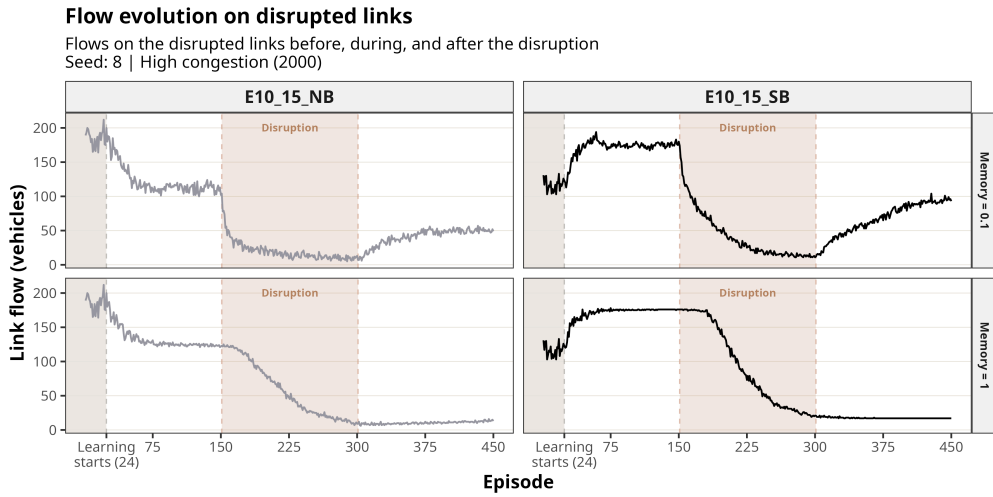


FIG. 29. Flow evolution on two disrupted links before, during, and after the temporary network disruption for different memory levels, using a representative seed.

Figure 29 shows the evolution of flow on the disrupted links for different memory levels. During the disruption, the flow on the affected links decreases for both memory levels, indicating that drivers adapt their route choices in response to the disruption. However, the speed of adaptation differs substantially with the memory level. With a value of 0.1, the flow decreases relatively quickly after the disruption begins, whereas adaptation is considerably slower in the case of perfect memory (when memory = 1).

This behavior is consistent with the effect of memory discussed in Section 5.4.3. With a lower memory level, recent travel experiences have a stronger influence on route perceptions. Thus, unfavorable experiences on the disrupted links affect their perceived costs more rapidly, leading drivers to shift towards alternative routes. With perfect memory, each new observation has a limited effect because it contributes only $1/N$ to the average, where N is the number of accumulated experiences, resulting in slower adaptation. Hence, a larger number of unfavorable experiences is required before the disrupted links become considerably less attractive.

The effect of memory on the speed of adaptation is also evident after the network is restored. With a value of 0.1, traffic gradually returns to the previously disrupted links, although their flows do not recover to their pre-disruption levels within the simulated period.

In contrast, in the case of perfect memory, the flows on the disrupted links do not recover at all after the network is restored. Drivers do not forget the unfavorable travel-time experiences accumulated during the disruption. Hence, these negative experiences continue to strongly influence their route perceptions even after the network has returned to its original conditions. Consequently, the disrupted links remain unattractive and their traffic flow does not recover.

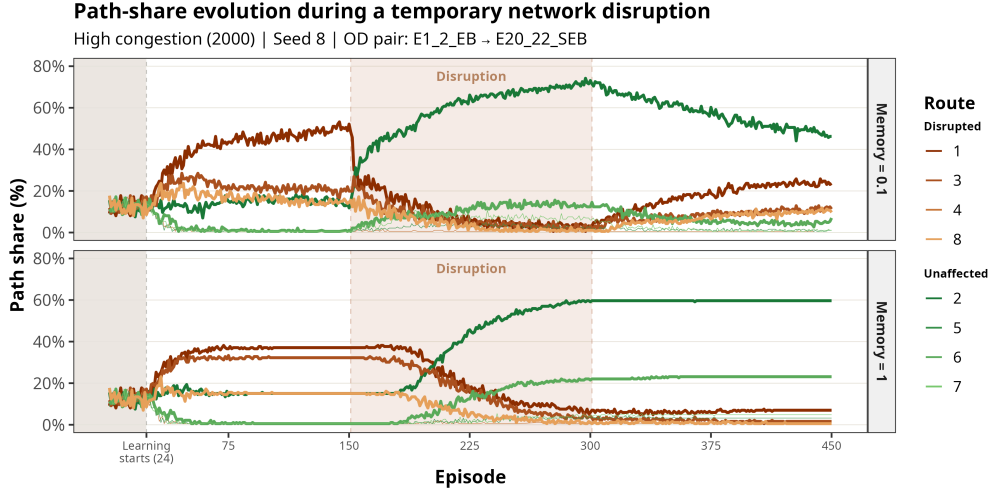


FIG. 30. Evolution of route shares for a representative OD pair before, during, and after the temporary network disruption for different memory levels, using a representative seed

A similar pattern can be observed at the path level in Figure 30. The figure considers a representative OD pair and shows the evolution of route shares before, during, and after the disruption. Before the disruption, the routes that were later affected carried a substantial share of the traffic. In particular, three disrupted routes, shown using the red tones, carry most of the route share, while only one of the unaffected routes, shown in green tones, carried a notable share of the traffic.

Once the disruption begins, the route shares of the affected paths decrease for both memory levels. With memory equal to 0.1, the bad experiences on the disrupted routes strongly influence route perceptions, causing drivers to shift away from the disrupted paths. At the same time, the unaffected paths absorb the displaced traffic, with one unaffected route becoming particularly dominant. With memory equal to 1, the redistribution occurs more gradually because the new unfavorable experiences have a smaller effect on perceptions.

After the network is restored, with memory equal to 0.1, traffic gradually returns to the previously disrupted paths, although their flows do not recover to their pre-disruption levels within the simulated period. In contrast, when memory is equal to 1, the flows on the disrupted paths do not recover at all after the network is restored. These results are consistent with the link-level results.

Previous work by Wei et al. [49] investigated disruption recovery using the same Bush–Mosteller-based MARL approach, with a memory level of 0.8, but in a STA setting. Their results generally showed complete recovery of the initial traffic state, whereas the recovery observed in this study is only partial.

Several factors may explain the difference in recovery. First, the simulation period after the restoration may not be sufficiently long. The upward trend in the

disrupted paths after restoration, together with the corresponding decrease in the unaffected paths, suggests that recovery is still taking place at the end of the simulation period. Second, the characteristics of the synthetic network and demand discussed in Section 5.4.3 may also contribute.

In particular, when an agent concentrates most of its probability on a near-optimal route, substantially worse alternatives can make it difficult to move away from that route.

The algorithm samples a route according to the agent’s probability distribution, so escaping the preferred route requires either receiving a poor travel time experience that reduces its probability or sampling an alternative route with low probability. However, this reduction may be limited when the alternative routes are substantially worse, as the penalization depends on how bad the experienced route is compared to the worst alternative (3). This may be particularly relevant in the present case, where some routes carry almost no traffic throughout the 450 episodes. Consequently, during the disruption, agents concentrate their probability on another near-optimal route, from which may be difficult to escape during the recovery period, if it remains near-optimal and there exists substantially worse alternatives.

In a real transportation network, the restoration of infrastructure does not imply that all drivers immediately return to their original routes. Previous studies have shown that recovery after a disruption can be partial rather than complete, as drivers may develop habits and inertia towards alternative routes during the disruption and retain them after the original route has been restored. For example, [19] develops a model to represent irreversible network changes, where the initial equilibrium is not fully restored after the network is reopened. The extent of recovery may also depend on whether the disruption was expected or unexpected. [10] found lower recovery levels for unexpected disruptions, whereas expected disruptions resulted in greater recovery.

Finally, the results highlight a limitation of the model when dealing with temporary disruptions. In this scenario, having perfect memory is detrimental because the model lacks an explicit mechanism to inform drivers that the disruption has ended and relies only on experienced travel times to infer changes in network conditions. A workaround could therefore be to introduce a mechanism through which drivers receive explicit information when a temporary disruption has ended. This information could be used to discount the unfavorable experiences accumulated during the disruption and promote exploration of those routes.

Overall, the results indicate that lower memory enables faster adaptation to the temporary disruption and facilitates partial recovery once the network is restored, while perfect memory leads to slower adaptation and persistent avoidance of the previously disrupted routes.

5.4.5. Learning Rate Sensitivity

Research question: The learning rate determines how strongly drivers update their route-choice probabilities in response to the stimulus. This research question investigates the impact of the learning rate on the learning process.

Experimental setup: The experiment uses the baseline hyperparameter configuration described in Section D, with the memory level set to 1, and performs a sensitivity analysis over the learning rate. The complete configuration file used for the experiment is provided in Appendix 6.

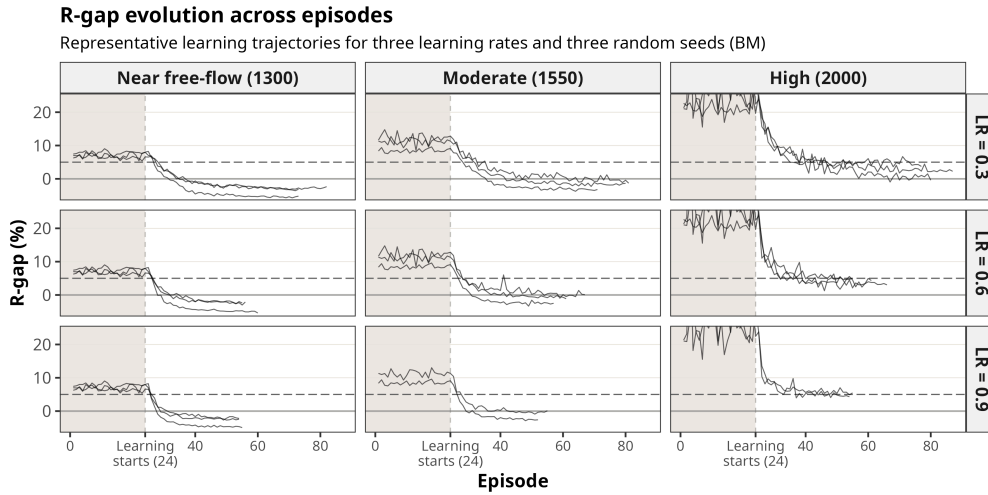


FIG. 31. R-gap trajectories for three learning rates under near free-flow, moderate, and high demand congestion regimes. Each panel shows trajectories from three random seeds; the shaded region indicates the warm-up period before learning starts (episode 24), and the dashed horizontal line marks the 5% R-gap level.

Figure 31 shows the evolution of the R-gap for different learning rates. No clear increase in oscillatory behavior is observed as learning rate increases. Under high congestion, all three learning rates converge to a DUE state, as indicated by the 5% R-gap threshold. However, lower learning rates are associated with slightly lower values, whereas higher learning rates result in values closer to the 5% threshold. The learning rate also affects the convergence speed of the MARL algorithm, defined as the point at which agents cease updating their policies. Higher learning rates lead to earlier convergence of the MARL algorithm, as reflected by the shorter trajectories.

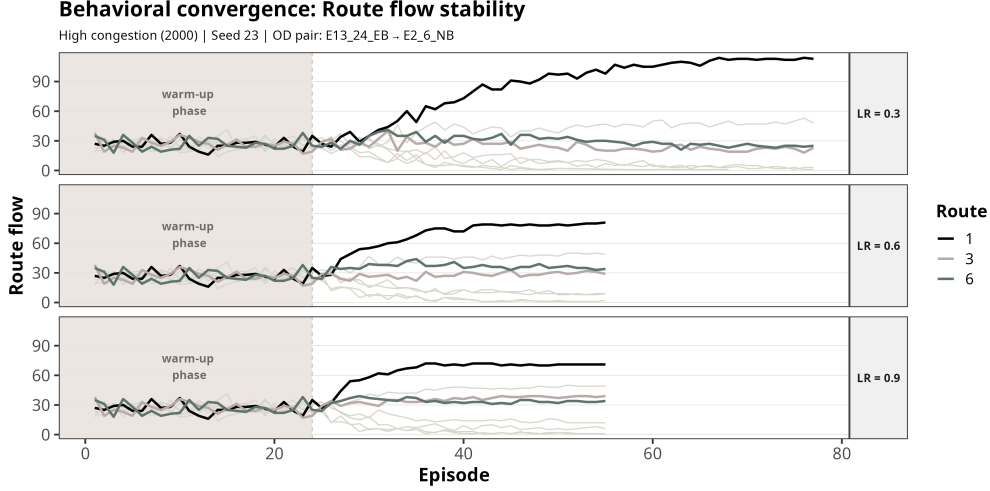


FIG. 32. Route flow evolution across episodes for different learning rates under a high congestion regime. The plot shows the flow of the available routes for a representative origin–destination pair and random seed. The shaded area indicates the 24-episode warm-up phase. Only the relevant routes are highlighted.

TABLE 4. Route characteristics of the paths belonging to the representative OD pair shown in Figure 32 and their route shares for different learning rates.

Path	Median PT_r (s)	Route share $l = 0.3$	Route share $l = 0.6$	Route share $l = 0.9$
1	83.3	53%	38%	33%
3	100.0	11%	14%	18%
6	97.3	12%	16%	16%
2	109.6	0%	4%	3%
4	110.2	1%	4%	6%
5	140.0	0%	0%	0%
7	94.7	23%	23%	23%
8	220.0	0%	1%	0%

Figure 32 illustrates the evolution of route flows for the three learning rates under high congestion. It can be seen that the learning rate affects how flow is distributed across routes. For $LR = 0.3$, flow becomes more concentrated on a single route, whereas higher learning rates result in a more distributed flow across several routes.

The patterns observed in the R-gap trajectories and route-flow evolution can be better understood by considering the characteristics of the synthetic network and demand, as discussed in Section 5.4.3. In particular, the presence of near-optimal and clearly unattractive routes, together with the absence of cost-rank flipping, plays an important role. Since the relative cost ranking of the routes remains

stable, higher learning rates do not induce the oscillatory behavior that could otherwise occur when a large number of drivers shift to a route and subsequently change its cost ranking. Therefore, in this network configuration, higher learning rates allow faster adaptation without causing instability. Moreover, because the algorithm can distinguish between attractive and unattractive routes and the cost ranking remains stable, a higher learning rate allows the probability mass to be concentrated on attractive routes more quickly, resulting in faster convergence of the MARL algorithm, while also allowing the DUE state to be reached slightly earlier.

The remaining effects, specifically, the slightly higher final R-gap values observed at higher learning rates and the more distributed route flows, can be explained by the magnitude of the probability updates. With a higher learning rate, a favorable route experience leads to a larger increase in the probability of selecting that route. If the experienced route is near-optimal rather than optimal, this can cause the agent to quickly concentrate its probability mass on a near-optimal route before having the opportunity to sample the optimal one. In contrast, lower learning rates produce smaller updates, allowing more opportunities to sample the optimal route and potentially concentrate the probability mass on it. Consequently, higher learning rates result in a more distributed allocation among near-optimal routes, whereas lower learning rates lead to greater concentration on the optimal route.

This effect is reflected in Figure 32 and Table 4. In particular, the flow share of the optimal Route 1 decreases as the learning rate increases, from 53% at $l = 0.3$ to 38% at $l = 0.6$ and 33% at $l = 0.9$, with part of this flow being redistributed among the near-optimal Routes 3 and 6. The median values reported in Table 4 are calculated from the perceived travel times of all agents in the final episode of the scenario with $l = 0.3$.

These results differ from those reported by Wei et al. [49], where higher learning rates led to stronger flow fluctuations. This difference may be explained by the different network settings: the STA with link performance functions considered in that study allows more frequent cost-rank flipping, whereas such flipping is not very frequent in the present DTA synthetic network and scenario.

Overall, for the considered synthetic network and demand configuration, all tested learning rates reach a DUE state, although higher values lead to slightly higher final R-gap. At the same time, increasing the learning rate accelerates the convergence of the MARL algorithm.

5.4.6. Risk Aversion

Research question: In the original Bush–Mosteller formulation, agents evaluate routes based on their average past travel times. This research question incorporates travel-time variability into the perceived route cost (see Section 4.3.1) to account for risk-sensitive drivers who may prefer longer but more stable routes. The main

objective is to assess whether the proposed model converges to a DUE state when agents incorporate this aspect of human route-choice behavior.

Experimental setup: The experiment uses the baseline hyperparameter configuration described in Section D, with the memory level set to 1 and the learning rate set to 0.3. It also incorporates the risk-sensitivity hyperparameter, which is tested with several values. Intuitively, the risk-sensitivity parameter θ determines how strongly travel-time variability is penalized in the perceived route cost (see (7) and (8)). The waiting-time sensitivity is set to zero throughout the experiment, so that only the effect of risk sensitivity is examined. The complete configuration file used for the experiment is provided in Appendix 7.

For Figures 33 and 35 and Table 5, the reported perceived route costs and route shares correspond to the final episode of the learning process in a scenario without risk-sensitive drivers.

Risk aversion can have a meaningful effect on route choice when the shortest route is less stable than alternative routes. In this case, risk-averse drivers may prefer a more stable route even if it has a higher mean travel time. The network-demand configuration used in this experiment has this characteristic for some OD pairs, making it suitable for studying the effect of risk sensitivity on route choice.

Results and Interpretation:

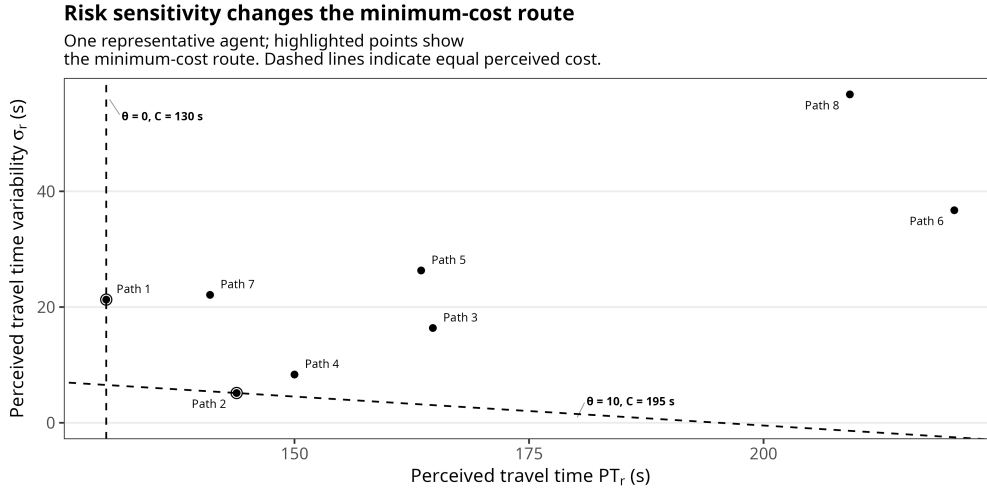


FIG. 33. Illustration of the effect of risk sensitivity on perceived route costs for a representative agent and random seed. Highlighted points indicate the minimum-cost route for $\theta = 0$ and $\theta = 10$, while dashed lines indicate equal perceived cost, where perceived route cost is calculated as $PT_r + \theta \cdot \sigma_r$, with $\varphi = 0$

Figure 33 illustrates how risk sensitivity changes the route perceived as most attractive from the perspective of a representative agent. As defined in Section 4.3.1,

identifying the minimum-cost route requires considering both perceived travel time and perceived travel-time variability. The dashed lines represent contours of equal perceived route cost. When $\theta = 0$, the preferred route is Path 1, as it has the lowest perceived travel time. However, when the agent is risk-sensitive ($\theta = 10$), Path 2 becomes the minimum-cost route. Despite having a longer perceived travel time, Path 2 has a lower perceived travel-time variability, which leads to a lower overall perceived cost. This illustrates how risk-sensitive drivers may prefer a more stable route when travel-time variability is taken into account.

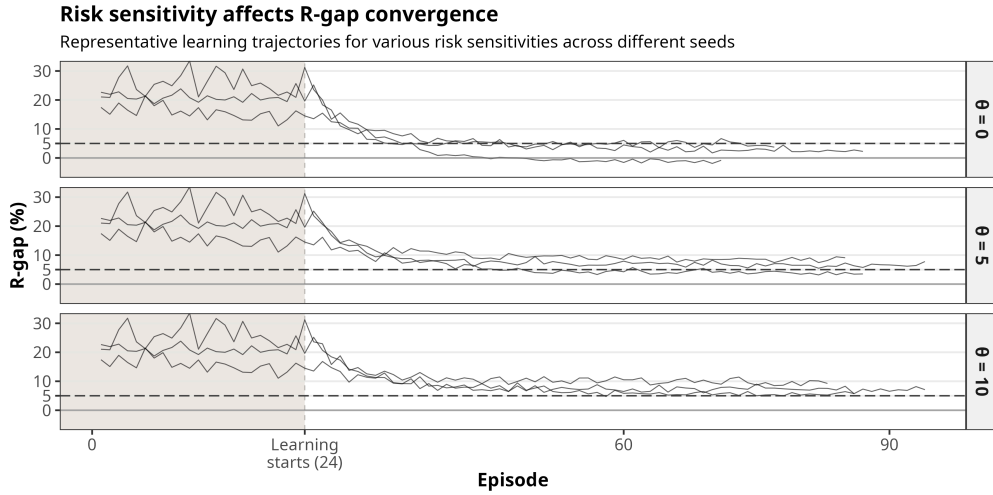


FIG. 34. Effect of different levels of risk sensitivity on R-gap convergence across three representative random seeds.

Figure 34 shows the R-gap trajectories for a homogeneous population of drivers across different risk-sensitivity levels. Unlike the risk-insensitive case ($\theta = 0$), the tested risk-sensitive populations generally do not converge towards the DUE state. Moreover, increasing the risk-sensitivity parameter leads to larger deviations from DUE.

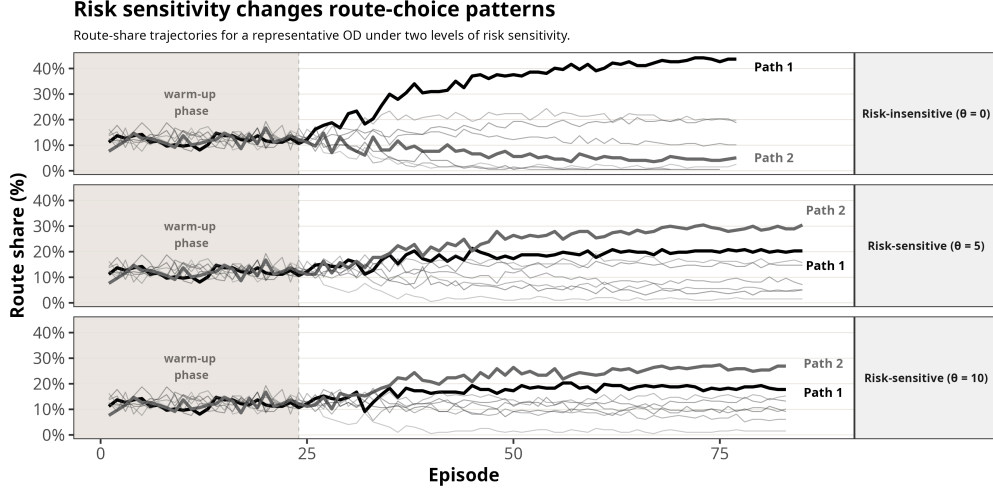


FIG. 35. Effect of risk sensitivity on route shares for a representative OD pair and random seed.

TABLE 5. Route characteristics of the paths belonging to the representative OD pair shown in Figure 35 and their route shares for different levels of risk sensitivity

Path	Median PT_r (s)	Median σ_{T_r} (s)	Route share $\theta = 0$	Route share $\theta = 5$	Route share $\theta = 10$
1	128	12.8	44%	20%	18%
2	146	4.4	5%	30%	27%
3	163	13.0	0%	5%	9%
4	135	13.3	19%	15%	13%
5	146	16.4	10%	7%	10%
6	173	14.6	0%	5%	6%
7	137	14.5	20%	16%	15%
8	229	54.7	3%	2%	2%

Figure 35 shows how route shares change for a representative OD pair as different levels of risk sensitivity are introduced. The clearest change can be observed in Paths 1 and 2. With no risk sensitivity ($\theta = 0$), Path 1 carries the largest route share at 44%, while Path 2 carries only 5%. Path 1 has the lowest median perceived travel time (128 s), making it an attractive route when only travel time is considered. In contrast, Path 2 has a higher median perceived travel time (146 s) than several other routes, which helps explain its lower route share of 5% under $\theta = 0$.

Under risk-sensitive populations ($\theta = 5$ and $\theta = 10$), Path 2 becomes the route with largest share: In particular Path 2 reaches 30% and 27%, respectively, while the share of Path 1 decreases to 20% and 18%. This shift can be explained by the

substantially lower perceived travel-time variability of Path 2, while its perceived travel time remains relatively close to that of Path 1.

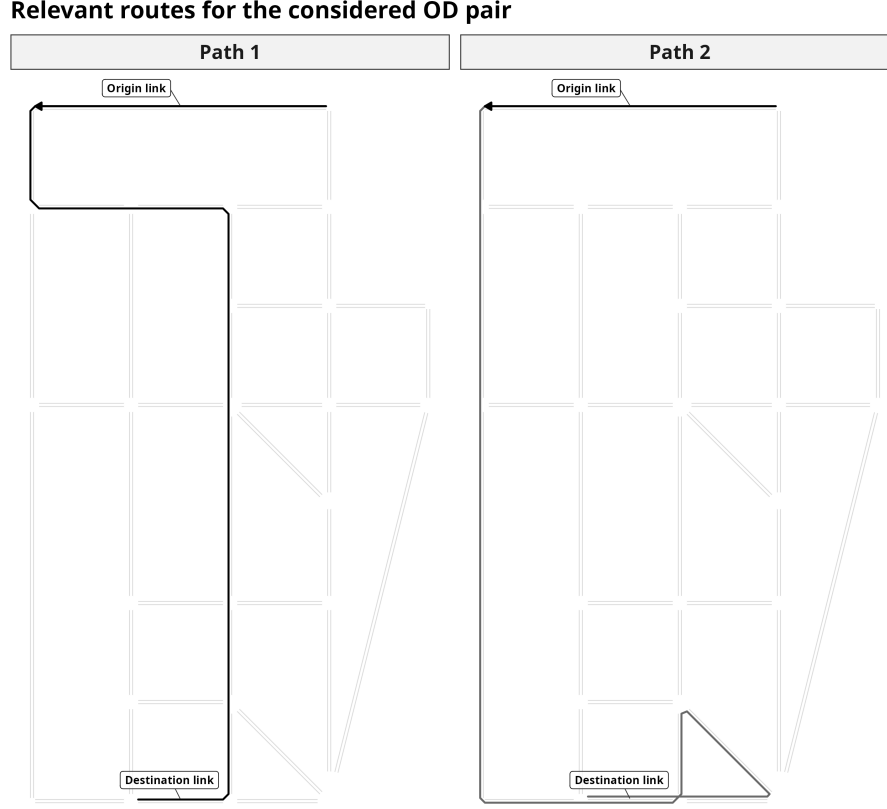


FIG. 36. Route alternatives relevant to the risk-aversion and waiting-time analysis.

As shown in Figure 36, Path 1 passes through the central part of the network, whereas Path 2 follows a more peripheral route, which may contribute to the higher travel-time variability observed for Path 1. This makes Path 2 more attractive to risk-sensitive drivers.

Interestingly, increasing risk sensitivity from $\theta = 5$ to $\theta = 10$ does not further increase the route share of Path 2. One possible explanation is that the increased demand on Path 2 changes its travel-time characteristics, reducing the advantage that initially made it attractive to risk-sensitive drivers.

Overall, the results indicate that the tested homogeneous populations of risk-sensitive drivers generally fail to converge towards the DUE state.

5.4.7. Waiting-Time Aversion

Research question: In the original Bush–Mosteller formulation, agents evaluate routes based on their average past travel times. This research question incorporates waiting time into the perceived route cost (see Section 4.3.1) to account for waiting-time averse drivers who may prefer longer routes if they involve less time spent fully stopped. The main objective is to assess whether the proposed model converges to a DUE state when agents incorporate this aspect of human route-choice behavior.

Experimental setup: The experiment uses the baseline hyperparameter configuration described in Section D, with the memory level set to 1 and the learning rate set to 0.3. It also incorporates the waiting-time sensitivity hyperparameter, which is tested with several values. Intuitively, the waiting-time sensitivity parameter φ determines how strongly waiting time is penalized in the perceived route cost (see (7) and (8)). The risk sensitivity parameter is set to zero throughout the experiment, so that only the effect of waiting-time sensitivity is examined. The complete configuration file used for the experiment is provided in Appendix 8.

For Figures 37 and 39 and Table 6, the reported perceived route costs and route shares correspond to the final episode of the learning process in a scenario without waiting-time-sensitive drivers.

Waiting-time aversion can have a meaningful effect on route choice when the shortest route involves more time spent waiting than alternative routes. In this case, waiting-time-averse drivers may prefer a route with higher mean travel time if it involves less time spent stopped. The network-demand configuration used in this experiment has this characteristic for some OD pairs, making it suitable for studying the effect of waiting-time sensitivity on route choice.

Results and Interpretation:

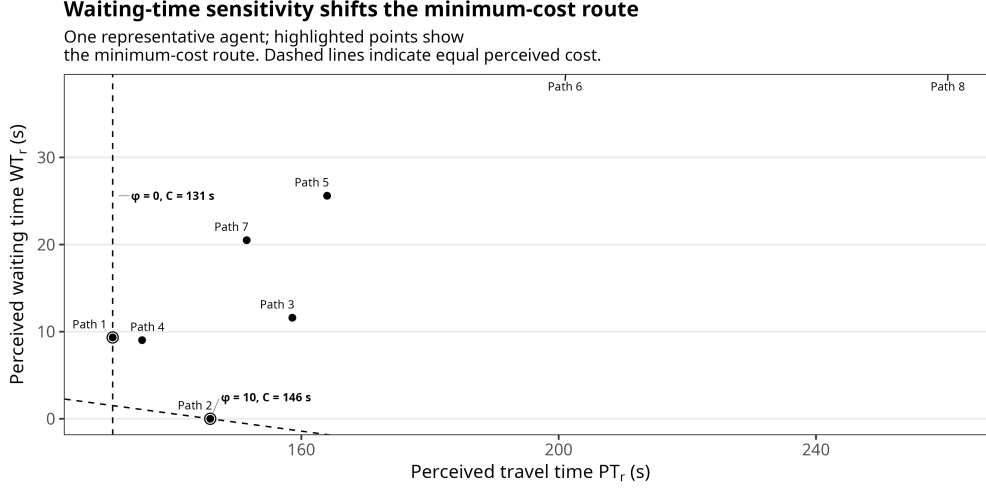


FIG. 37. Illustration of the effect of waiting-time sensitivity on perceived route costs for a representative agent. Highlighted points indicate the minimum-cost route for $\varphi = 0$ and $\varphi = 10$, while dashed lines indicate equal perceived cost, where perceived route cost is calculated as $PT_r + \varphi \cdot WT_r$, with $\theta = 0$

Figure 37 illustrates how waiting-time sensitivity changes the route perceived as most attractive from the perspective of a representative agent. As defined in Section 4.3.1, identifying the minimum-cost route requires considering both perceived travel time and perceived waiting-time. The dashed lines represent contours of equal perceived route cost. When $\varphi = 0$, the preferred route is Path 1, as it has the lowest perceived travel time. However, when the agent is waiting-time sensitive ($\varphi = 10$), Path 2 becomes the minimum-cost route. Despite having a longer perceived travel time, Path 2 has a lower perceived waiting-time, which leads to a lower overall perceived cost. This illustrates how waiting-time-sensitive drivers may prefer a longer route if it involves less waiting time.

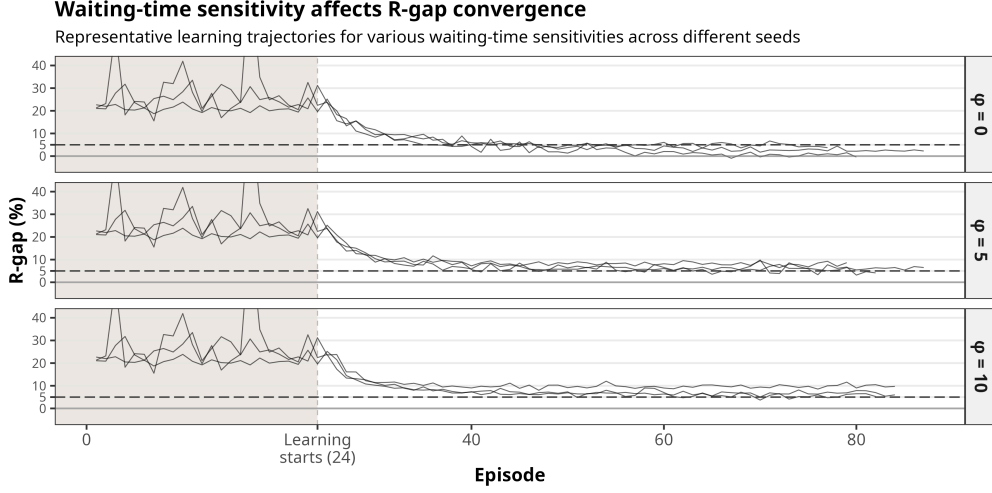


FIG. 38. Effect of different levels of waiting-time sensitivity on R-gap convergence across three representative random seeds.

Figure 38 shows the R-gap trajectories for a homogeneous population of drivers across different waiting-time-sensitivity levels. Unlike the waiting-time-insensitive case ($\varphi = 0$), the tested waiting-time-sensitive populations generally do not converge towards the DUE state. Moreover, increasing the waiting-time-sensitivity parameter leads to larger deviations from DUE.

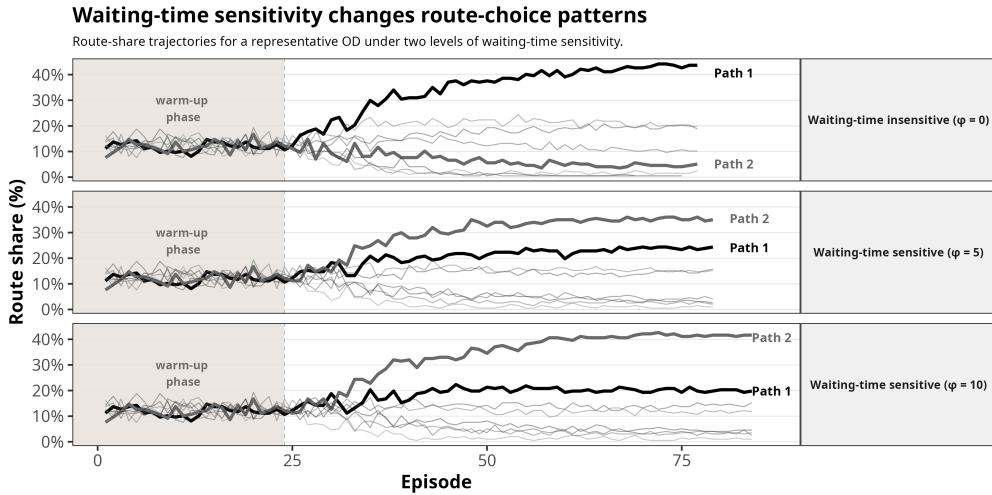


FIG. 39. Effect of waiting-time sensitivity on route shares for a representative OD pair and random seed.

Figure 39 shows the changes in route shares for the representative OD pair used in the previous analysis as different levels of waiting-time sensitivity are introduced. The clearest change can be observed in Paths 1 and 2. With no waiting-time

TABLE 6. Route characteristics of the paths belonging to the representative OD pair shown in Figure 39 and their route shares for different levels of waiting-time sensitivity

Path	Median <i>PT</i> (s)	Median <i>WT</i> (s)	Route share $\varphi = 0$	Route share $\varphi = 5$	Route share $\varphi = 10$
1	128	8.96	44%	24%	20%
2	146	0.75	5%	35%	42%
3	163	12.14	0%	3%	5%
4	136	10.64	19%	16%	15%
5	146	18.00	10%	4%	3%
6	173	21.00	0%	2%	4%
7	137	12.50	20%	15%	12%
8	230	78.50	3%	1%	1%

sensitivity ($\theta = 0$), Path 1 carries the largest route share at 44%, while Path 2 carries only 5%. Path 1 has the lowest median perceived travel time (128 s), making it an attractive route when only travel time is considered.

Under waiting-time sensitive populations ($\varphi = 5$ and $\varphi = 10$), Path 2 becomes the route with largest share: In particular Path 2 reaches 35% and 42%, respectively, while the share of Path 1 decreases to 24% and 20%. This shift can be explained by the substantially lower perceived waiting time of Path 2, while its perceived travel time remains relatively close to that of Path 1. As shown in Figure 36, Path 1 passes through the central part of the network, whereas Path 2 follows a more peripheral route, which may contribute to the difference in waiting time. This makes Path 2 more attractive to waiting-time-sensitive drivers.

Overall, the results indicate that the tested homogeneous populations of waiting-time sensitive drivers generally fail to converge towards the DUE state.

5.4.8. Nonlinear Response

Research Question: In the original Bush–Mosteller formulation, agents update their probabilities based on a linear response. This research question incorporates a nonlinear reinforcement mechanism (see Section 4.3.2) to account for the possibility that drivers may ignore small variations in travel time while reacting increasingly strongly to larger variations. The main objective is to assess how this nonlinear response affects the learning process and whether the resulting MARL route-choice model converges to a DUE state.

Experimental setup: The experiment uses the baseline hyperparameter configuration described in Section D, with the learning rate set to 0.3. It also uses different memory levels, disables the stopping rule of the MARL algorithm, and includes the `nonlinear_stimulus` and `stimulus_tau` hyperparameters. The complete configuration file used for the experiment is provided in Appendix 9.

Results and Interpretation:

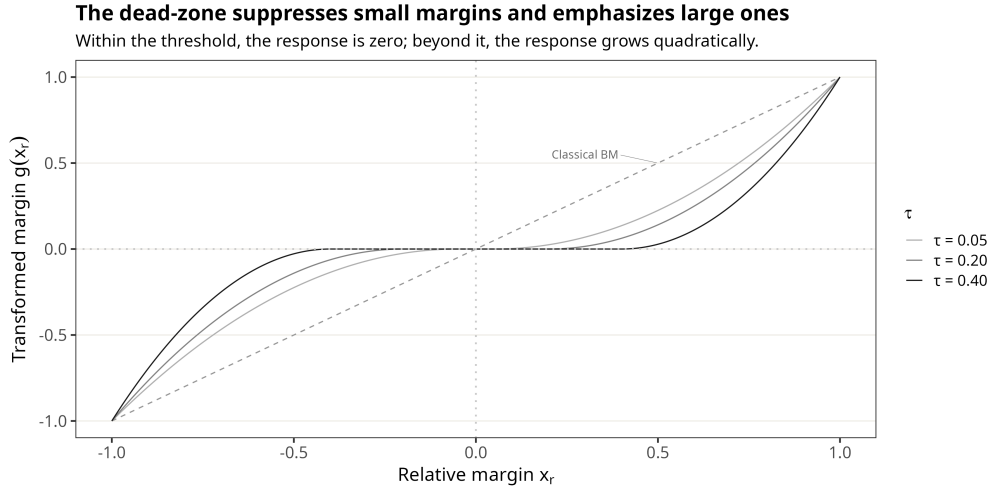


FIG. 40. Nonlinear transformation of the relative margin x_r (13) for different threshold values τ , illustrating the dead zone around $x_r = 0$ and the increasing response to larger relative margins.

Before interpreting the results, it is important to understand the two main quantities introduced during the formulation of the nonlinear mechanism: the *relative margin*, and the *transformed margin* (see Eqs. (13), and (14)).

To provide some intuition, a value of $\tau = 0.05$ means that the chosen route must be at least 5% better than the baseline, according to the agent's perception, to trigger reinforcement or penalization.

Figure 40 illustrates how the nonlinear mechanism operates. The nonlinear function is applied to the relative margin, so that relative margins within the threshold result in a stimulus of zero and therefore do not produce reinforcement or penalization. For relative margins above the threshold, the quadratic transformation produces an increasingly stronger response.

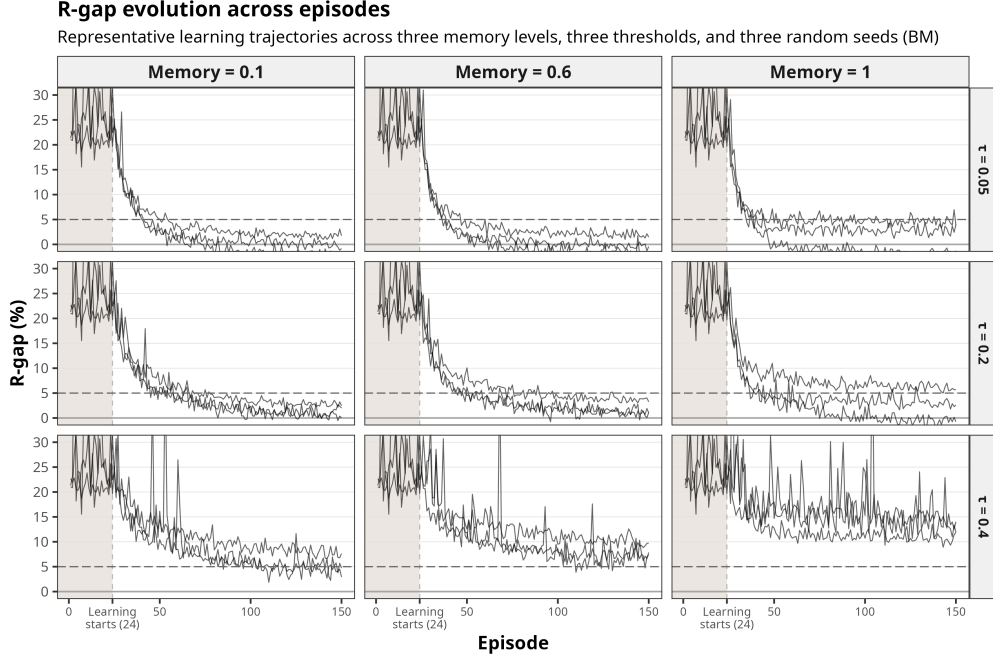


FIG. 41. R-gap evolution across episodes for different memory levels and nonlinear response thresholds, showing representative learning trajectories for three random seeds.

Figure 41 shows the R-gap evolution across memory levels and threshold values. Higher values of τ generally lead to higher final R-gap values, with the $\tau = 0.4$ cases failing to converge towards DUE. A smaller but consistent effect can also be observed for memory, with higher memory levels resulting in slightly higher final R-gap values.

As τ increases, larger relative margins are needed for agents to react, meaning that agents have fewer opportunities to update their route-choice probabilities. For $\tau = 0.05$ and $\tau = 0.2$, the obtained R-gap values are small, suggesting that agents are still able to obtain sufficient reinforcement signals to differentiate which are the near-optimal routes. With $\tau = 0.4$, agents only respond to relative margins of at least 40%, which leads to fewer opportunities for the agents to update their route-choice probabilities, limiting the ability to differentiate which are the near-optimal routes and preventing the traffic assignment from approaching DUE.

Lower memory places greater weight on recent observations, making perceived costs more sensitive to individual observations and therefore producing more variable relative margins. This makes it more likely for relative margins to exceed the threshold and trigger a response. In contrast, higher memory gives more weight to past observations, smoothing variations in perceived costs and making large relative margins less frequent. Consequently, higher memory levels tend to produce smaller

relative margins and, when the threshold is relatively high, reduce the number of opportunities for agents to update their route-choice probabilities.

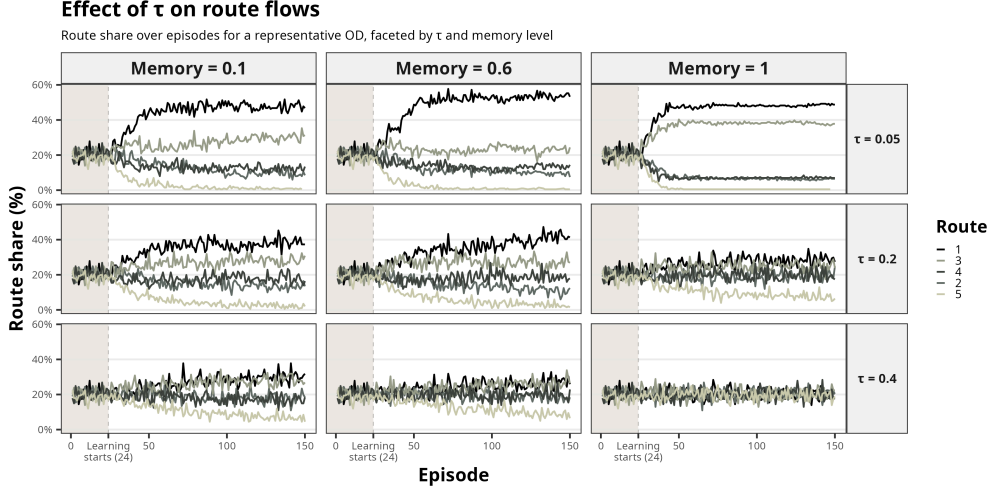


FIG. 42. Route flow evolution across episodes for different memory levels and nonlinear response thresholds under a high congestion regime. The plot shows the flow of the available routes for a representative origin–destination pair and random seed. The shaded area indicates the 24-episode warm-up phase.

Figure 42 shows the consequences of the previously discussed effects of τ and memory on route flows. As discussed above, higher values of τ result in fewer opportunities for agents to update their route-choice probabilities. Consequently, their probability distributions remain more dispersed across routes, leading to more dispersed route flows. This pattern can be observed across all memory levels. A similar effect is observed for memory: higher memory levels tend to produce smaller relative margins and for high thresholds, it results in fewer opportunities to update route-choice probabilities. Consequently, agents end up with more dispersed probability distributions over routes.

Overall, even when agents have fewer opportunities to update their route choice probabilities, they can still distinguish between attractive and unattractive routes. However, the smaller number of updates limits their ability to concentrate probability mass on the most attractive routes.

The previously discussed effects of higher τ and memory levels, which result in more dispersed probability distributions, are quantified in Table 7 through the metric *number of effective routes*. This entropy-based measure captures how concentrated or dispersed a probability distribution is. The higher its value, the more dispersed is the probability distribution.

For example, let us consider an agent with three available routes, referred to as Route 1, Route 2 and Route 3. A probability distribution that assigns a probability of 0.98 to Route 1 and 0.01 to both Route 2 and Route 3 should not be considered more dispersed than a distribution that assigns 0.50 to Route 1, 0.50 to Route 2, and 0 to Route 3. The metric captures this intuition by accounting for the concentration of probability mass, rather than simply counting the number of routes with nonzero probability. The number of effective routes is calculated as follows:

$$H = - \sum_{r=1}^K p_r \log p_r,$$

$$N_{\text{eff}} = e^H$$

where K is the number of available routes, and p_r is the probability assigned to route r .

TABLE 7. Mean number of effective routes (N_{eff}) for all agents of a representative OD pair shown in Figure 42 across different memory levels and nonlinear response thresholds.

τ	Memory level		
	0.1	0.6	1.0
0.05	1.61	1.35	1.04
0.20	2.43	2.76	4.07
0.40	3.84	4.35	4.98

Overall, the results show that the nonlinear reinforcement mechanism affects the learning process and, consequently, the convergence of the MARL route-choice model towards DUE. Higher values of the threshold τ reduce the number of opportunities for agents to update their route-choice probabilities, resulting in more dispersed probability distributions and, consequently, more dispersed route flows. For sufficiently high values of τ , this reduction in the number of policy updates becomes substantial enough to prevent the model from converging towards DUE. Memory has a similar effect, but it is smaller and only occurs when the thresholds are sufficiently high.

Therefore, under this synthetic scenario, the nonlinear mechanism can substantially influence the learning process and the ability of the model to reach a DUE state, particularly when the response threshold is high.

6. Conclusion and Future Work

The objective of this thesis was to investigate whether a day-to-day multi-agent reinforcement learning (MARL) approach to route choice in which agents learn solely from their own experiences converges to a Dynamic User Equilibrium (DUE) within a dynamic traffic assignment (DTA) setting using a microscopic traffic simulation. First, the study examined whether a DUE would emerge from the learning process of rational agents using a reinforcement learning approach. Second, more realistic behavioral traits were integrated into the agents' decision-making to examine how these traits affect the learning process and whether a DUE would still emerge.

The first research question examined whether a MARL approach based on the Bush–Mosteller algorithm could lead to a DUE state under the assumption of rational drivers. The results show that, under these assumptions, the agents converge to a DUE state through their individual learning process. Consequently, DUE can emerge not only through a conventional iterative assignment procedure but also through a MARL approach, where equilibrium results from individual route-choice decisions.

The second research question compared the spatial traffic patterns produced by the proposed Bush–Mosteller MARL approach with those obtained using the `duaIterate` benchmark. The resulting flows on the links were similar between the two approaches. However, at the path level, the Bush–Mosteller MARL approach distributed traffic more broadly among the available routes, whereas `duaIterate` concentrated traffic on fewer routes.

The third research question investigated the effect of memory. Lower memory levels increased route-flow variability. However, for the synthetic network and demand configuration considered, all tested memory levels still converged to DUE. The influence of memory became more apparent in the fourth research question, when considering a temporary disruption: lower memory enabled faster adaptation to the disruption and partial recovery after the network was restored, whereas perfect memory resulted in slower adaptation and persistent avoidance of the previously disrupted routes.

The fifth research question examined the effect of learning rate. For the synthetic network and demand configuration considered, all tested learning rates converged towards DUE. Higher learning rates accelerated both the learning process and the attainment of DUE, although they resulted in slightly higher final R-gap values. These results may differ in more complex network and demand configurations with more frequent changes in the cost ranking of routes across episodes, where memory and learning rate may have a stronger influence on the learning dynamics and convergence towards DUE.

The sixth research question investigated the effect of risk-sensitive behavior. The results indicate that the tested homogeneous populations, in which all drivers shared the same risk-aversion parameter, generally failed to converge towards DUE.

The seventh research question investigated the effect of waiting-time sensitive behavior. The results indicate that the tested homogeneous populations, in which all drivers shared the same waiting-time sensitivity parameter, generally failed to converge towards DUE.

Finally, the eighth research question investigated the nonlinear reinforcement mechanism and its interaction with memory. The results showed that high values of τ can prevent convergence towards DUE, particularly when combined with higher memory levels.

Overall, the results suggest that the proposed model converges to DUE under its more rational formulation, while variations in the existing behavioral parameters, namely memory level and learning rate, do not necessarily prevent convergence under the tested scenario. However, their influence may become stronger in more complex network-demand configurations with more frequent changes in the cost ranking of routes across episodes. Previous research has shown that lower memory levels can prevent convergence to UE in a STA setting where changes in route-cost ranking occur more frequently [49].

The introduction of additional behavioral mechanisms, such as waiting-time sensitivity, risk aversion, and nonlinear response, can prevent convergence towards DUE. These results suggest that incorporating more realistic behavioral mechanisms into the agents' decision-making can prevent the network from reaching DUE, as reflected by higher R-gap values, although the effect depends on the behavioral parameters and the network-demand configuration.

The main contribution of this thesis is to further investigate the relatively understudied application of MARL route choice models to traffic assignment in a DTA setting. While MARL has previously been applied to route choice in STA settings, relatively few studies have extended this approach to DTA coupled with a microscopic traffic simulator. This thesis further considers day-to-day route-choice, in which agents learn solely from their own experiences, without relying on externally provided network information. In addition, the model extends the original Bush–Mosteller MARL route choice model proposed by [49], which considers route choice in an STA setting, by incorporating behavioral mechanisms representing bounded rationality, including risk-sensitive and waiting-time sensitive behavior and a nonlinear response mechanism.

Several limitations of the present study also motivate directions for future research. First, the experiments were conducted using a relatively simple synthetic network. The network relies on several simplifications, such as identical lane speeds, the absence of traffic signals, and a single lane. Second, the experiments were also conducted using a relatively simple synthetic demand configuration. The demand relies on several simplifications, such as all agents representing drivers with the default values for the parameters related to car-following, lane-changing, and vehicle characteristics 14. In addition, the OD matrix was constrained to a limited number of OD pairs through the hyperparameter `max_od_space`. More complex

network and demand configurations could be considered in future work, in which the cost ranking of routes changes more frequently across episodes. Such configurations could provide a more challenging environment in which the role of behavioral parameters such as memory and learning rate could be further investigated. An example of a small benchmark network that may be relevant is the Braess network, such as the one presented in [4].

Demand could be made more complex and realistic by considering mixed populations representing different types of drivers, such as drivers who rely solely on their own experience and drivers who use external navigation apps.

Third, the thesis studied a fixed degradation scenario in Section 5.4.4, in which particular links were degraded over a given period and then restored. However, it could be interesting to study a random degradation scenario in which different links can be degraded on different days. Such a scenario would better represent the disruptions that can occur in real-world networks due to incidents, maintenance, and other unforeseen events, and would allow investigating how such disruptions influence the learning process and the final network state.

Fourth, when studying the behavioral parameters, homogeneous populations were assumed, in which all drivers used the same parameter values. A heterogeneous case was studied for memory (see Appendix 0.1), although the results showed that the mean memory level of the population was more important than the variation in memory levels between drivers. Future work could study the effects of heterogeneity in other behavioral parameters.

Fifth, the experiments used uniform initial route-choice probabilities and a predefined initial exploration phase (see Section 5.1.2), which may influence the learning process and the final state reached by the agents.

The proposed framework could also be extended to alternative learning algorithms, such as Multi-Agent Thompson Sampling and Deep Reinforcement Learning methods such as Deep Q-Learning and actor-critic methods [41]. These algorithms could be compared with the proposed Bush–Mosteller approach to investigate whether they produce different learning and convergence behavior. Their performance could also be evaluated on larger networks and with larger agent populations to assess their scalability. The author has previously explored the use of Thompson Sampling for route choice in SUMO in a course project, in which only one driver was modelled as an RL agent while the remaining vehicles represented background traffic [14].

Finally, calibration and validation using real-world data would be an important step towards assessing the applicability of the model. It would allow the estimation of behavioral parameters, including learning rate, memory, risk sensitivity, waiting-time sensitivity, and the nonlinear response threshold. The calibrated model could then be validated to assess its ability to reproduce observed route choices and traffic flows.

7. Declaration of AI Use

During the preparation of this thesis, the authors used the following AI tools for specific purposes:

- ChatGPT Deep Research (GPT-5): Used to identify and discover relevant papers during the literature review, and to explore and analyze quantitative metrics that could be used to compare the spatial traffic patterns generated by the proposed framework and `duaIterate`.
- Claude Code (Claude 2): Used to obtain suggestions regarding code architecture and software implementation, sketch implementation details, perform code audits, refine code comments and module docstrings, and obtain suggestions for the Git commit messages and assist to debug and understand the learning process of a representative agents in order to understand why lower memory levels were still leading to a DUE.
- ChatGPT (GPT-5): Used to address conceptual questions and support reasoning and thinking during the development of the thesis. It was also used to refine the wording and clarity of the thesis and to support the overall structure and organization of its content. In addition, it was used to assist with the formulation and refinement of titles and captions for the plots.

The authors reviewed, edited, and critically evaluated all AI-generated content and retain full responsibility for the content, analysis, methodology, and conclusions presented in this thesis.

References

- [1] Ernest Adams and Herbert A. Simon. Models of Man, Social and Rational: Mathematical Essays on Rational Human Behavior in a Social Setting. *The Journal of Philosophy*, 59(7):177, March 1957.
- [2] Lucas N. Alegre, Luiz A. Thomasini, Gabriel de O. Ramos, and Ana L. C. Bazzan. RouteChoiceEnv: a Route Choice Library for Multiagent Reinforcement Learning, January 2023.
- [3] Apache Parquet. Apache parquet documentation. <https://parquet.apache.org/docs/>, 2026. Accessed: 18 September 2026.
- [4] Behzad Bamdad Mehrabani, Jakob Erdmann, Luca Sgambi, and Maaïke Snelder. Proposing a Simulation-Based Dynamic System Optimal Traffic Assignment Algorithm for SUMO: An Approximation of Marginal Travel Time. *SUMO Conference Proceedings*, 3:121–143, September 2022.
- [5] J Barcelo, J. Casas, D. García, and J. Perarnau. A methodological approach combining macro, meso and micro simulation models for transportation analysis, May 2005.
- [6] Ana L. C. Bazzan and Ricardo Grunitzki. A multiagent reinforcement learning approach to en-route trip building. In *2016 International Joint Conference on Neural Networks (IJCNN)*, pages 5288–5295, Vancouver, BC, Canada, July 2016. IEEE.
- [7] Enide A. I. Bogers, Francesco Viti, and Serge P. Hoogendoorn. Joint Modeling of Advanced Travel Information Service, Habit, and Learning Impacts on Route Choice by Laboratory Simulator Experiments. *Transportation Research Record: Journal of the Transportation Research Board*, 1926(1):189–197, January 2005.
- [8] Stephen D. Boyles, Nicholas E. Lownes, and Avinash Unnikrishnan. Transportation Network Analysis, Volume I: Static and Dynamic Traffic Assignment, 2025. Version Number: 1.
- [9] John Conlisk. Why Bounded Rationality? *Journal of Economic Literature*, 34:669–700, June 1996.
- [10] Adam Danczyk, Xuan Di, Henry X. Liu, and David M. Levinson. Unexpected versus expected network disruption: Effects on travel behavior. *Transport Policy*, 57:68–78, July 2017.
- [11] André De Palma and Nathalie Picard. Route choice decision under travel time uncertainty. *Transportation Research Part A: Policy and Practice*, 39(4):295–324, May 2005.

- [12] Xuan Di and Henry X. Liu. Boundedly rational route choice behavior: A review of models and methodologies. *Transportation Research Part B: Methodological*, 85:142–179, March 2016.
- [13] Xuan Di, Henry X. Liu, Shanjiang Zhu, and David M. Levinson. Indifference bands for boundedly rational route switching. *Transportation*, 44(5):1169–1194, September 2017.
- [14] Miguel Angel Donado Fernandez. Learning Route Choice via Thompson Sampling, 2026.
- [15] Manal Elimadi, Abdeljalil Abbas-Turki, Abder Koukam, Mahjoub Dridi, and Yazan Mualla. Review of Traffic Assignment and Future Challenges. *Applied Sciences*, 14(2):683, January 2024.
- [16] Patrick Gerland, Sara Hertog, Lubov Zelfman, and Guangyu Zhang. World Urbanization Prospects 2025. Summary of results., 2025.
- [17] Humberto González Ramírez, Ludovic Leclercq, Nicolas Chiabaut, Cécile Becarie, and Jean Krug. Travel time and bounded rationality in travellers’ route choice behaviour: A computer route choice experiment. *Travel Behaviour and Society*, 22:59–83, January 2021.
- [18] Ziyuan Gu, Xun Yang, Qi Zhang, Wenwu Yu, and Zhiyuan Liu. TERL: Two-Stage Ensemble Reinforcement Learning Paradigm for Large-Scale Decentralized Decision Making in Transportation Simulation. *IEEE Transactions on Knowledge and Data Engineering*, 35(12):13043–13054, December 2023.
- [19] Xiaolei Guo and Henry X. Liu. Bounded rationality and irreversible network change. *Transportation Research Part B: Methodological*, 45(10):1606–1618, December 2011.
- [20] Andrew Haigh, Deborah Apthorp, and Lewis A. Bizo. The role of Weber’s law in human time perception. *Attention, Perception, & Psychophysics*, 83(1):435–447, January 2021.
- [21] Ke Han, W.Y. Szeto, and Terry L. Friesz. Formulation, existence, and computation of boundedly rational dynamic user equilibrium with fixed or endogenous user tolerance. *Transportation Research Part B: Methodological*, 79:16–49, September 2015.
- [22] W. Burke Jackson and James V. Jucker. An Empirical Study of Travel Time Variability and Travel Choice Behavior. *Transportation Science*, 16(4):460–475, November 1982.
- [23] W. Jager. Breaking ‘bad habits’: a dynamical perspective on habit formation and change, 2003.
- [24] Bruce N. Janson. Dynamic traffic assignment for urban road networks. *Transportation Research Part B: Methodological*, 25(2-3):143–161, April 1991.
- [25] Jiayang Li, Zhaoran Wang, and Yu Marco Nie. Wardrop Equilibrium Can Be Boundedly Rational: A New Behavioral Theory of Route Choice, 2023. Version Number: 2.
- [26] María Paz Linares Herreros and Jaime Barceló Bugeda. *A mesoscopic traffic simulation based dynamic traffic assignment*. PhD thesis, Universitat Politècnica de Catalunya, April 2014.

- [27] Tsippy Lotan. Effects of familiarity on route choice behavior in the presence of information. *Transportation Research Part C: Emerging Technologies*, 5(3-4):225–243, August 1997.
- [28] Michael W. Macy and Andreas Flache. Learning dynamics in social dilemmas. *Proceedings of the National Academy of Sciences*, 99(suppl_3):7229–7236, May 2002.
- [29] Michael Mahut, Michael Florian, and Nicolas Tremblay. Space-Time Queues and Dynamic Traffic Assignment: A Model, Algorithm and Applications, July 2002.
- [30] MLflow. MLflow documentation. <https://mlflow.org/docs/latest/>, 2026. Accessed: 18 September 2026.
- [31] Takayuki Morikawa, Tomio Miwa, Shinya Kurauchi, Toshiyuki Yamamoto, and Kei Kobayashi. Driver’s Route Choice Behavior and its Implications on Network Simulation and Traffic Assignment. In Ryuichi Kitamura and Maso Kuwahara, editors, *Simulation Approaches in Transportation Analysis*, volume 31, pages 341–369. Springer-Verlag, New York, 2005. Series Title: Operations Research/Computer Science Interfaces Series.
- [32] Edgar Elías Osuna. The psychological cost of waiting. *Journal of Mathematical Psychology*, 29(1):82–105, March 1985.
- [33] Darshana Othayoth and K.V. Krishna Rao. Assessing the relationship between perceived waiting time and level of service at signalized intersection under heterogeneous traffic conditions. *Asian Transport Studies*, 6:100024, 2020.
- [34] Jamal Amani Rad, Stephane Hess, Thomas O. Hancock, Michiel Bliemer, Matthew Beck, Muhammad Fayyaz, and Eran Ben-Elia. Where does this road go? Insights from reinforcement learning models on route choice behaviour, June 2025.
- [35] Gabriel De O. Ramos, Bruno C. Da Silva, and Ana L.C. Bazzan. Learning to Minimise Regret in Route Choice. pages 846–855, New York, New York, May 2017.
- [36] Bin Ran and David Boyce. *Modeling Dynamic Transportation Networks*. Springer Berlin Heidelberg, Berlin, Heidelberg, 1996.
- [37] William Samuelson and Richard Zeckhauser. Status quo bias in decision making. *Journal of Risk and Uncertainty*, 1(1):7–59, March 1988.
- [38] Song Sang Koh. Thesis: Realtime Vehicle Route Optimisation via DQN for Sustainable and Resilient Urban Transportation Network. 2022.
- [39] David Schrank, Bill Eisele, Tim Lomax, and Jim Bak. 2015 Urban Mobility Scorecard, 2015.
- [40] Zhenyu Shou, Xu Chen, Yongjie Fu, and Xuan Di. Multi-agent reinforcement learning for Markov routing games: A new modeling paradigm for dynamic traffic assignment. *Transportation Research Part C: Emerging Technologies*, 137:103560, April 2022.
- [41] David Silver. Reinforcement Learning Notes, 2015.
- [42] Herbert A. Simon. A Behavioral Model of Rational Choice. *The Quarterly Journal of Economics*, 69(1):99, February 1955.

- [43] Herbert A. Simon. Rationality in Psychology and Economics. *The Journal of Business*, 59(4):S209–S224, 1986.
- [44] Herbert A. Simon. Bounded Rationality. In John Eatwell, Murray Milgate, and Peter Newman, editors, *Utility and Probability*, pages 15–18. Palgrave Macmillan UK, London, 1990.
- [45] Hanlin Sun and Jiayang Li. LLM-Guided Reinforcement Learning with Representative Agents for Traffic Modeling, 2025. Version Number: 1.
- [46] TomTom. TomTom Traffic Index. Technical report, TomTom, 2026.
- [47] Leizhen Wang, Peibo Duan, Cheng Lyu, Zewen Wang, Zhiqiang He, Nan Zheng, and Zhenliang Ma. Scalable and reliable multi-agent reinforcement learning for traffic assignment. *Communications in Transportation Research*, 5:100225, December 2025.
- [48] J G Wardrop and J I Whitehead. Some Theoretical Aspects of Road Traffic Research. *Proceedings of the Institution of Civil Engineers*, 1(5):767–768, October 1952.
- [49] Fangfang Wei, Shoufeng Ma, and Ning Jia. A Day to Day Route Choice Model Based on Reinforcement Learning. *Mathematical Problems in Engineering*, 2014(1):646548, January 2014.
- [50] Wikipedia contributors. Weber–fechner law. https://en.wikipedia.org/wiki/Weber%E2%80%93Fechner_law, 2025. Accessed: 27 August 2026.
- [51] Xinkai Wu, David M. Levinson, and Henry X. Liu. Perception of Waiting Time at Signalized Intersections. *Transportation Research Record: Journal of the Transportation Research Board*, 2135(1):52–59, January 2009.
- [52] Shanjia Zhu, David Levinson, Henry X. Liu, and Kathleen Harder. The traffic and behavioral effects of the I-35W Mississippi River bridge collapse. *Transportation Research Part A: Policy and Practice*, 44(10):771–784, December 2010.

Appendix A

Derivation of the corrected probability update of the non selected routes

This appendix derives the probability update for the non-selected routes in the negative-stimulus case. The formulation in Wei et al. [49] is missing a term in this case. The corrected expression used in this thesis is derived by imposing the normalization condition that the probabilities of all routes must sum to one.

Consider a negative stimulus, $S_i^t(c) < 0$, for the selected route c . The probability of the selected route is updated according to

$$(16) \quad p_i^{t+1}(c) = p_i^t(c) + \lambda p_i^t(c) S_i^t(c).$$

The probabilities of the non-selected routes must satisfy the normalization condition

$$(17) \quad \sum_{k \neq c} p_i^{t+1}(k) = 1 - p_i^{t+1}(c).$$

$$(18) \quad \sum_{k \neq c} p_i^t(k) = 1 - p_i^t(c).$$

Following the assumption that the probabilities of the non-selected routes are re-distributed proportionally, let

$$(19) \quad p_i^{t+1}(k) = r p_i^t(k), \quad k \neq c,$$

where r is a common scaling factor.

Substituting Eq. (19) into (17) gives

$$(20) \quad r \sum_{k \neq c} p_i^t(k) = 1 - p_i^{t+1}(c).$$

Substituting Eq. (18) into the left side of Eq. (20) gives

$$(21) \quad r = \frac{1 - p_i^{t+1}(c)}{1 - p_i^t(c)}.$$

Using Eq. (16), this becomes

$$(22) \quad r = \frac{1 - p_i^t(c) - \lambda p_i^t(c) S_i^t(c)}{1 - p_i^t(c)}.$$

Substituting this scaling factor into Eq. (19) yields the corrected update for every non-selected route $k \neq c$:

$$(23) \quad \boxed{p_i^{t+1}(k) = p_i^t(k) \frac{1 - p_i^t(c) - \lambda p_i^t(c) S_i^t(c)}{1 - p_i^t(c)}}$$

or equivalently,

$$(24) \quad p_i^{t+1}(k) = \frac{p_i^t(k) - p_i^t(k)p_i^t(c) - \lambda p_i^t(k)p_i^t(c)S_i^t(c)}{1 - p_i^t(c)}.$$

The additional term

$$-p_i^t(k)p_i^t(c)$$

is missing from the corresponding expression in Wei et al. [49]. Without this term, the updated probabilities do not satisfy the required normalization condition Eq. (17). In contrast, the corrected formulation satisfies

$$(25) \quad p_i^{t+1}(c) + \sum_{k \neq c} p_i^{t+1}(k) = 1.$$

Appendix B

Justification of the Risk-Averse and Waiting-Time-Averse Formulations

For the risk-sensitive extension, several considerations motivate the specific formulation adopted:

- (1) **Measure of variability** Perceived travel-time standard deviation is used as a measure of variability because it is expressed in seconds, the same units as travel time.
- (2) **Memory** The memory mechanism is also applied to the risk-sensitivity extension. This captures the intuition that a route that was previously reliable but has recently become risky should have a different perceived variability from a route that was previously risky but has recently become more reliable.
- (3) **Reference-dependent risk perception** Following the intuition of the reference-dependent theory discussed in Section 4.3.2, in which outcomes are assessed relative to a reference point, a route is considered risky only when its variability exceeds the baseline variability, that is, the variability expected for the trip from A to B. The risk term is therefore included both in the aspiration cost and in the perceived cost of each route.
- (4) **Additive risk term** The risk term is introduced additively rather than as a weight. This gives the risk-sensitivity parameter θ a direct behavioural interpretation: it represents the additional seconds of perceived cost per extra second of travel-time variability.

Similar reasoning applies to the waiting-time-averse extension.

Appendix C

Software Implementation

This appendix provides the detailed software implementation corresponding to the separation of concerns described in Section 5.1.1. For each component, the main implementation logic is briefly described, followed by a summary of its functions/methods.

0.0.1. Demand Generation

The demand generation component is responsible for creating the agent data structure, including the origin, destination, and departure time of each agent. The main entry point, `demand_from_count()`, determines the number of warm-up agents and post-warm-up agents and coordinates their generation. OD pairs are generated using `randomTrips`, parsed and restricted to the k most frequent pairs. The agents are then assigned OD pairs from this restricted OD space through weighted sampling. Departure times are subsequently generated for the corresponding agents. The demand generation component only creates the agent data and does not instantiate `BMAgent` objects.

The agent data structure and unique OD pairs generated by this component are passed to the `Scenario` class during its initialization.

The OD space is restricted because `randomTrips` can generate a lot of OD pairs. Since each OD pair requires a set of alternative routes, restricting the OD space reduces the computational cost of route set construction. The `restrict_od_space` function therefore retains the `max_size_od_space` most frequent OD pairs (see Section D).

The main functions implemented by the demand generation component are:

Function	Task
<code>demand_from_count()</code>	Main entry point that determines the warm-up and post-warm-up agents and coordinates demand generation.
<code>generate_agents()</code>	Creates the agent data structure by generating OD pairs and departure times for the specified agents.
<code>build_od_space()</code>	Coordinates the generation, parsing, and restriction of the OD space.
<code>generate_random_trips()</code>	Generates candidate trips using the SUMO <code>randomTrips</code> tool, with the <code>fringe-factor</code> and <code>min-distance</code> hyperparameters. More details about the hyperparameters in Section D.
<code>parse_ods()</code>	Extracts the OD pairs from the generated trips.
<code>restrict_od_space()</code>	Restricts the OD space to the <code>max_size_od_space</code> most frequent OD pairs.
<code>sample_ods()</code>	Assigns OD pairs to agents by weighted sampling from the restricted OD space according to their frequencies.
<code>generate_departure_times()</code>	Generates uniformly distributed departure times for agents over the specified simulation horizon.

TABLE 1. Main functions implemented by the demand generation component.

0.0.2. Scenario

The `Scenario` class is created once before the training starts and contains the static data shared across all episodes. During its initialization, it constructs the route sets for all OD pairs, stores the scenario data required for analysis, and generates the SUMO configuration files used by the `Environment`. The stored scenario data include the route sets generated using `duarouter`, the agent data structure, the OD matrix, and the free-flow link costs. The class then provides the agent data structure, route sets, and SUMO configuration to the other components during training.

The agent data structure and the unique OD pairs are generated externally by `generate_agents()` and passed to the `Scenario` class during initialization.

The main functions implemented by the `Scenario` component are:

Function	Task
<code>compute_k_routes()</code>	Constructs up to (k) alternative routes for each OD pair by repeatedly calling <code>duarouter</code> with randomly perturbed link costs (see Section 5.1.2).
<code>_run_duarouter()</code>	Executes <code>duarouter</code> to compute a shortest path between an origin and destination.
<code>_create_sumo_config()</code>	Generates the SUMO configuration and mean-data files used during the simulations.
<code>_save_scenario_data()</code>	Stores the scenario data required for simulation and subsequent analysis.

TABLE 2. Main functions implemented by the **Scenario** component.

0.0.3. Environment

The **Environment** class is responsible for running individual episodes in the SUMO simulator and obtaining the resulting travel times. It receives the action selected by each agent, corresponding to the route chosen for that episode, generates the corresponding SUMO route file, runs the simulation, and processes the simulation output. Travel times are used as the reward, while waiting times are obtained for the waiting-time-averse driver extension (see 4.3.1). Routes are passed to SUMO through a route file rather than through TraCI.

The main functions associated with this component are:

Function	Task
<code>generate_routes_file()</code>	Generates the SUMO route file from the routes selected by the agents.
<code>run_episode()</code>	Runs SUMO using the scenario configuration and generated route file.
<code>get_rewards()</code>	Extracts the experienced travel time for each agent from the SUMO <code>tripinfo</code> output.
<code>get_waiting_times()</code>	Extracts the waiting time for each agent for the waiting-time-averse driver extension.

TABLE 3. Main functions associated with the **Environment** component.

0.0.4. Bush–Mosteller Agent

The Bush–Mosteller Agent component implements the route-choice behavior of each driver. Each agent maintains a probability distribution over its available routes and a history of past route choices, and travel times. For the waiting-time-averse driver extension (see Section 4.3.1, waiting times are also stored in the

agent’s history. At each episode, the agent samples a route according to its current probability distribution. Once learning is active, the agent’s history is used to update the route-choice probabilities according to the Bush–Mosteller algorithm described in Section 4.2.

The Bush–Mosteller Agent implementation also supports the behavioral extensions introduced in Sections 4.3.1 and 4.3.2.

The agent has two main operations during training. `select_action()` samples a route according to the current probability distribution, while `update()` uses the agent’s history to update its policy once the warm-up conditions are satisfied.

A small set of auxiliary orchestration functions coordinates these operations across all agents.

The main functions implemented by the Bush–Mosteller agent component are:

Function	Task
<code>initialize_agents()</code>	Creates the BMAgent objects from the generated agent data structure.
<code>select_actions()</code>	Coordinates route selection for all agents.
<code>select_action()</code>	Samples a route from the agent's current route choice probability distribution.
<code>update_agents()</code>	Coordinates the policy update of all agents after an episode.
<code>update()</code>	Updates the agent's history, computes the learning quantities, and updates the route choice probabilities.
<code>update_history()</code>	Stores the selected route, travel time, and waiting time from the current episode.
<code>compute_expected_travel_time()</code>	Computes the expected travel time from the agent's history.
<code>compute_perceived_travel_times()</code>	Computes the perceived travel time for each route.
<code>compute_travel_time_std()</code>	Computes the dispersion of the agent's past travel times.
<code>compute_route_travel_time_std()</code>	Computes the dispersion of travel times for each route.
<code>compute_expected_waiting_time()</code>	Computes the expected waiting time from the agent's history.
<code>compute_perceived_waiting_times()</code>	Computes the perceived waiting time for each route.
<code>compute_perceived_costs()</code>	Computes the aspiration cost and perceived cost of each route.
<code>compute_stimulus()</code>	Computes the stimulus for the selected route.
<code>apply_stimulus_threshold()</code>	Applies the nonlinear stimulus transformation when enabled.
<code>update_probabilities()</code>	Updates the route choice probabilities based on the stimulus.
<code>reinforce_chosen()</code>	Reinforces the selected route when the stimulus is non-negative.
<code>penalise_chosen()</code>	Penalises the selected route when the stimulus is negative.
<code>snapshot()</code>	Creates a JSON-serializable representation of the agent's internal state for debugging and analysis.

TABLE 4. Main functions implemented by the BM agent component.

0.0.5. Orchestration

The components described above are coordinated by a small set of orchestration functions. The `main` function is the entry point of the program and initializes the experiment before calling `orchestrate_training`. The `orchestrate_training` function creates the `Scenario`, `Environment`, and `BMAgent` objects and calls `run_training_loop` to execute the training process.

The `run_training_loop` function handles the episode-by-episode execution. At each episode, it collects the route choices of all agents, runs the simulation, obtains the travel times, updates the agents' policies, processes and stores the results, and checks the MARL convergence criteria. Once training is complete, the processed data are exported and the DUE convergence check is performed for both the proposed Bush–Mosteller model and the `duaIterate` benchmark using the R-gap described in Section 5.3.4.

The main functions implemented for orchestration are:

Function	Task
<code>main()</code>	Entry point of the program that initializes the experiment and calls <code>orchestrate_training()</code> .
<code>orchestrate_training()</code>	Creates the main components, calls <code>run_training_loop()</code> , and completes the training process.
<code>run_training_loop()</code>	Executes the episode loop, including route selection, simulation, agent updates, result processing, and MARL convergence checks.

TABLE 5. Main functions implemented for orchestration.

The remaining modules provide supporting functionality for configuration management, data parsing, experiment tracking, analysis, and DUE convergence evaluation and are not described further here.

Appendix D

SA and Non-SA Hyperparameters

1. SA hyperparameters

The analyses of the SA hyperparameters were carried out using the configuration described in Section F.

If different networks are considered, the analyses should be carried out separately for each network.

1.1. `min_distance_factor`

The `min_distance_factor` α defines the minimum distance that an origin–destination (OD) pair must satisfy to be considered valid for the experiments. Specifically, the minimum distance is defined as a fraction of the network diagonal:

$$(26) \quad d_{\min} = \alpha \cdot d_{\text{diag}},$$

where d_{diag} denotes the diagonal length of the network.

A sensitivity analysis was conducted by varying α and examining the resulting free-flow travel times of the shortest paths. There is a trade-off: origin-destination pairs that are farther apart are desirable, but beyond a certain point, there may not be enough pairs that meet this requirement.

Additional considerations: The value of α also affects the initial R-gap indirectly through its interaction with `random_factor` hyperparameter. When the minimum-distance constraint is restrictive, the shortest and alternative routes tend to have more similar lengths. Conversely, with a less restrictive distance constraint and a high `random_factor`, an OD pair may have a relatively short shortest path while alternative routes are substantially longer and, consequently, higher R-gaps during the initial exploration phase.

Although a high initial R-gap followed by a reduction during learning would be a desirable property for illustrating the learning dynamics, the `min_distance_factor`

hyperparameter was not selected with the objective of maximizing the initial R-gap. Instead, the parameter was selected to impose a minimum trip length and avoid extremely short trips.

Results and Interpretation:

After looking at the Figures 1 and Figure 2 the final value was set to $\alpha = 0.8$.

Many other values could have been selected, with only minor differences in the resulting distributions. However, a value of 0.8 was chosen because it satisfies the considerations described above: it avoids extremely short trips while not excessively constraining the set of feasible OD pairs for the particular network.

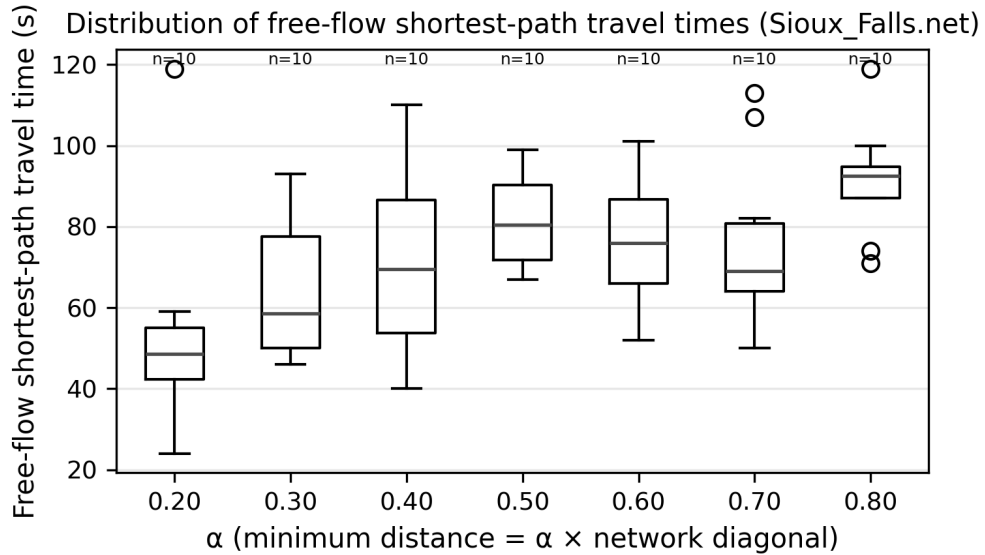


FIG. 1. Sensitivity analysis hyperparameter `min_distance_factor`. Range 0.20 - 0.80.

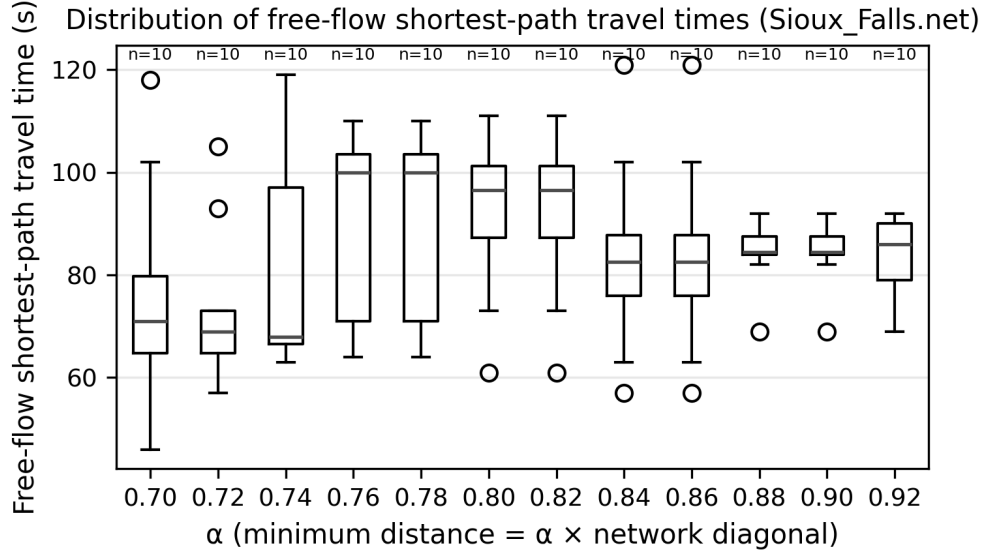


FIG. 2. Sensitivity analysis hyperparameter `min_distance_factor`. Range 0.70 - 0.92

In the plots, $n = 10$ indicates that 10 OD pairs were requested and that 10 valid pairs were identified.

1.2. random_factor

The `random_factor` controls the magnitude of the random perturbations applied by `duarouter` to link costs when generating alternative routes. To understand its effect, it is first necessary to describe how the alternative routes are generated.

For a given OD pair, `duarouter` first computes the shortest path using the original link costs. To obtain alternative paths, the link costs are randomly perturbed and a new shortest-path search is performed using the perturbed costs. Repeating this process with different perturbations allows `duarouter` to generate a set of alternative routes.

The magnitude of these perturbations is controlled by `random_factor`. When the value is close to one, the perturbations are small and the modified link costs remain similar to their original values. Consequently, repeated shortest-path searches are likely to produce the same or very similar routes. As the value of `random_factor` increases, larger differences can be introduced into the link costs, increasing the likelihood that different paths are selected during the alternative-route search. This increases route diversity and makes it more likely than an OD pair reaches the target number of routes.

However, increasing `random_factor` also affects the quality of the generated alternatives. Because the perturbed costs can cause `duarouter` to select paths that

would not be competitive under the original link costs, the resulting alternative routes can be increasingly longer than the original shortest path. Thus, increasing `random_factor` tends to improve route-set completeness, but may also result in increasingly suboptimal alternative routes.

Two criteria were used to select the value of the random factor. The first was the number of OD pairs for which the target number of alternative routes was found. The second was the average increase in free-flow travel time of the alternative routes compared with the shortest path.

The selected value was intended to provide both a high degree of route-set completeness and sufficiently high route suboptimality to generate meaningful variation in route quality. In particular, a higher initial R-gap is desirable for illustrating the learning process.

Additional considerations: The effect of the `random_factor` on the initial R-gap also depends on the number of routes per OD pair. As the random factor increases, the route set may include more costly alternatives. With more routes per OD pair, this is more likely to happen, which can lead to a higher initial R-gap.

Results and Interpretation:

After looking at the Figures 3 and Figure 4 the final value was set to `random_factor` = 50.

Looking at the figures, a value of 50 achieves the highest number of OD pairs with the target of 8 routes, while producing a route-set quality comparable to that obtained with higher values of the random factor. Therefore, increasing the random factor beyond 50 provides little additional benefit in terms of route-set completeness or quality.

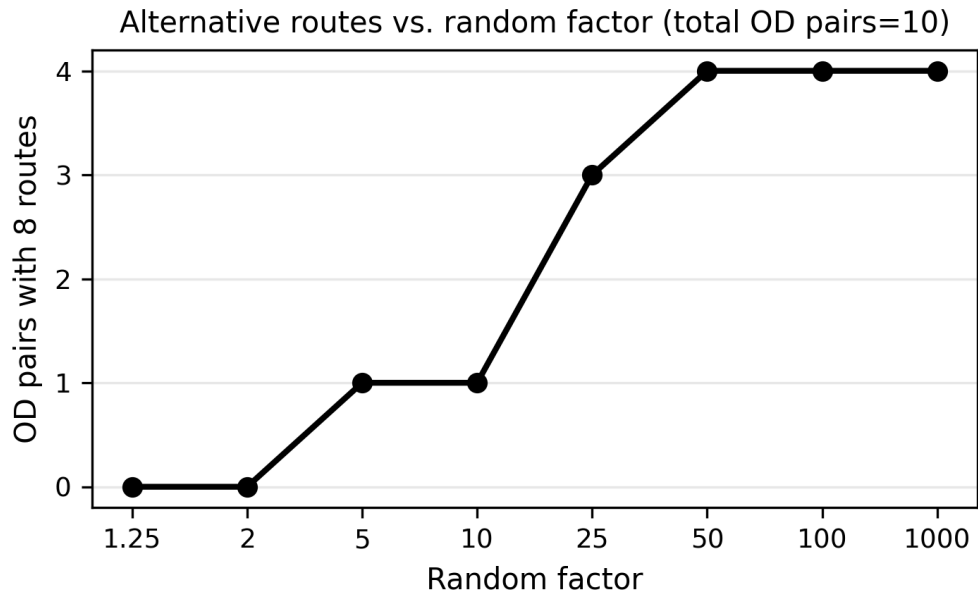


FIG. 3. Route generation as a function of the random factor

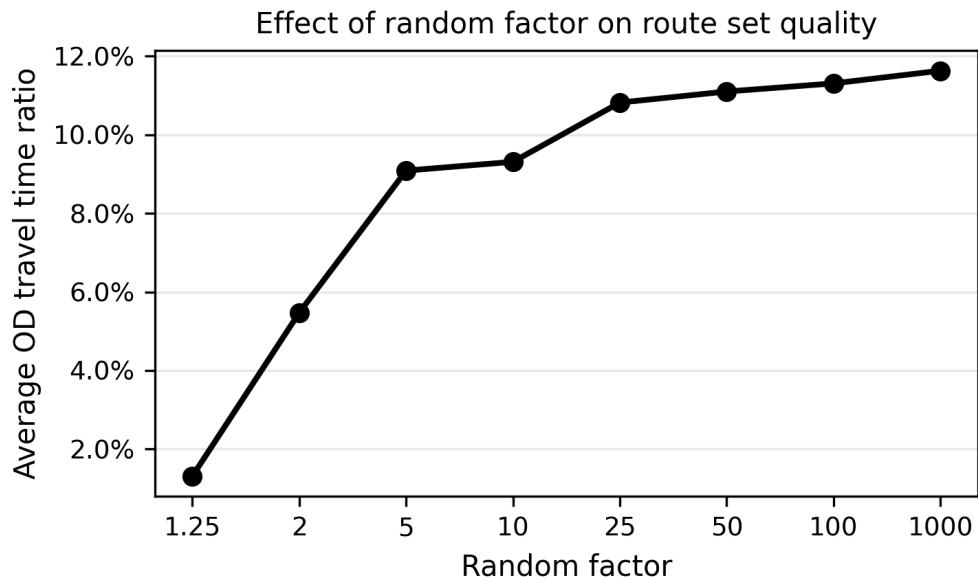


FIG. 4. Route-set quality as a function of the random factor

1.3. `n_routes_per_od`

The `n_routes_per_od` determines the number of alternative routes available to each OD pair during the learning process. Increasing this parameter gives agents greater

flexibility in their route choices, but also enlarges the action space, increasing the amount of exploration required before convergence.

The sensitivity analysis considered three metrics:

- First-episode R-gap: It reflects the quality of the initial route set. Increasing the number of available routes can increase the initial R-gap as more traffic is distributed across less efficient alternatives.
- Final-episode R-gap: It was included to determine whether providing more route alternatives could reduce the R-gap after learning.
- Number of episodes required for convergence: Lower values are preferable.

Results and Interpretation:

As shown in Figure 5 the expected relationship for the first-episode R-gap is observed, with higher values of `n_routes_per_od` resulting in a higher initial R-gap. The figure also confirms that increasing the number of available routes increases the number of episodes required for convergence. However, the hypothesized monotonic decrease in the final R-gap was not observed.

Based on the results shown in Figure 5, `n_routes_per_od = 8` was selected as it provides a sufficiently high initial R-gap while keeping low final R-gap without excessively increasing the number of episodes required for convergence.

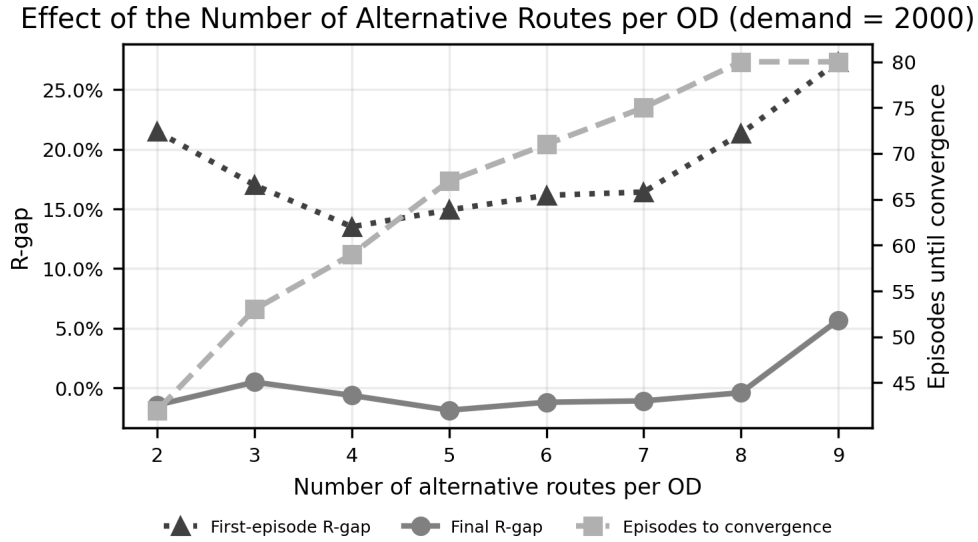


FIG. 5. Effect of the number of alternative routes per OD on the initial and final R-gap and convergence speed to (demand = 2000).

Additional Considerations: The effect of `n_routes_per_od` depends on `random_factor`. A higher number of routes per OD only has an effect when `random_factor` is sufficiently high to generate them.

1.4. `threshold_density`

The `threshold_density` parameter determines how missing values in the average link travel time table are imputed before computing the time-dependent shortest paths (TDSP) (see Section 5.3.4). When the observed density of an link exceeds `threshold_density`, the travel time from the previous interval is used; otherwise, the free-flow travel time is assigned.

The sensitivity analysis evaluated the first-episode and final R-gap for different values of `threshold_density`. The purpose was to assess whether the choice of threshold has a meaningful effect on the R-gap and to identify a value beyond which further increases have little effect.

Results and Interpretation:

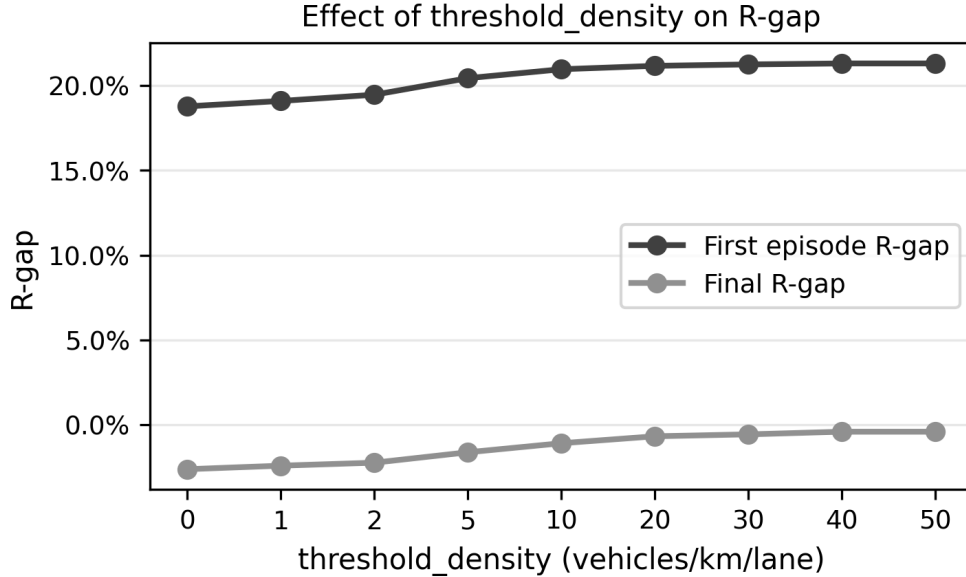


FIG. 6. Sensitivity of the first-episode and final R-gap to hyperparameter `threshold_density` (demand = 2000).

As shown in Figure 6 the R-gap shows some variation for threshold values below approximately 10 vehicles/km/lane, while only minor changes are observed for higher values. Therefore, `threshold_density` = 40 was selected as the final value, as it provides a threshold more representative of a congested link while having only a minor effect on the resulting R-gap.

1.5. `warm_up_time`

The simulation starts from an empty network, which creates an initial transient as vehicles enters the network and traffic conditions develop. During this period, the

observed traffic conditions are not representative of the steady operating conditions of interest. A warm-up period is therefore used to exclude this initial phase from the analysis.

The minimum warm-up time was determined from the first simulation episode under a random routing policy by analysing the number of vehicles in the network over time. This metric directly shows how the network fills and when this process stabilises. Only the first episode was considered.

Results and Interpretation:

Based on Figure 7 a warm-up time of 5 minutes was selected as the smallest candidate that provided stable occupancy across all considered demand scenarios.

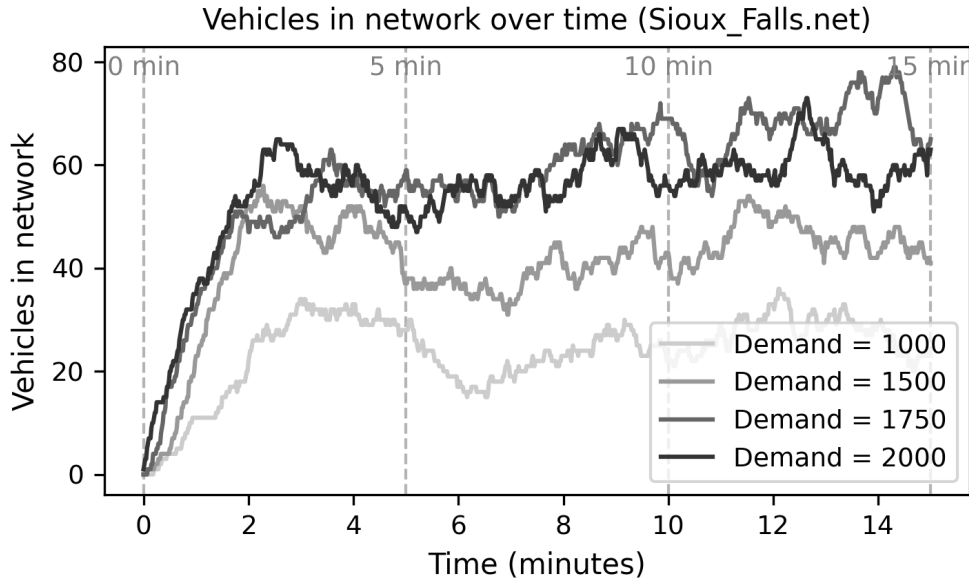


FIG. 7. Network occupancy over time for different demand levels, with candidate warm-up times indicated by vertical lines.

Additional Considerations:

The required warm-up time may depend on the traffic demand, network topology, and vehicle departure schedule (uniform in all experiments).

1.6. max_od_space

The `max_od_space` parameter determines the maximum number of distinct OD pairs in the OD matrix. Increasing this value allows the fixed traffic demand to be distributed across a larger number of OD pairs, reducing the concentration of vehicles and the competition for shared network resources. The sensitivity analysis

evaluated the first-episode R-gap, final R-gap, and number of episodes required for convergence.

Results and Interpretation:

As shown in Figure 8, both the first-episode and final R-gap decrease sharply between 5 and 10 OD pairs. With only 5 OD pairs, the demand of 2000 vehicles is highly concentrated, leading to strong competition for the same network resources, high congestion, and preventing convergence to DUE. Beyond 10 OD pairs, the demand is sufficiently distributed across the network and convergence to DUE is possible. Further increases in the number of OD pairs have little effect.

Based on the results presented in Figure 8, `max_od_space` = 10 was selected. A relatively high initial R-gap that can be substantially reduced to low values is considered a desirable property. The selected value provides this behavior while also requiring a reasonable number of episodes to reach convergence.

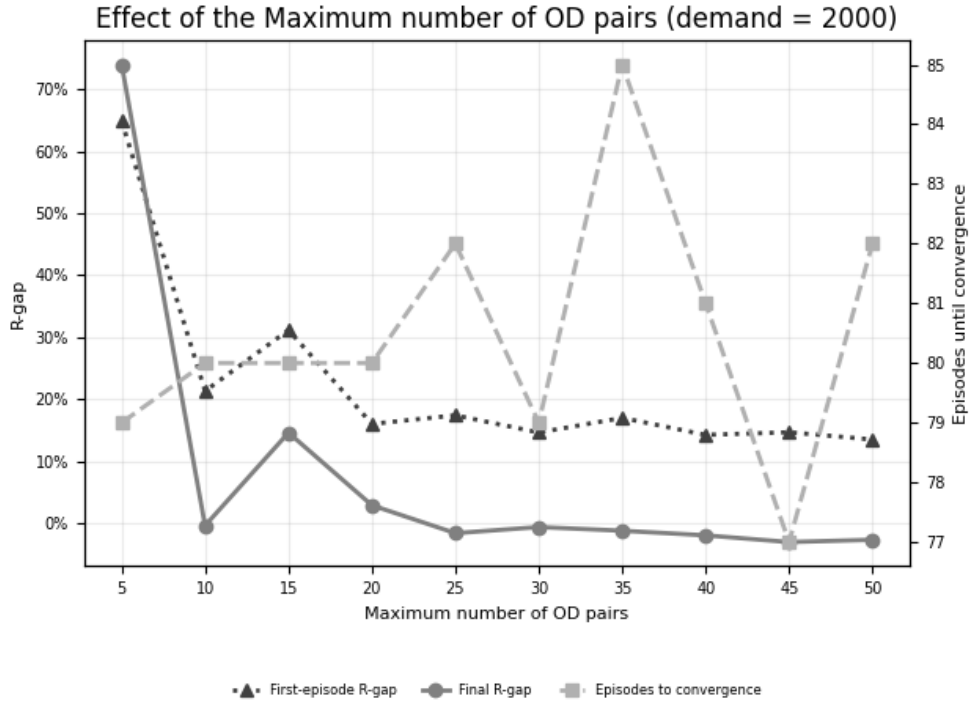


FIG. 8. Effect of the hyperparameter `max_od_space` on the first and final R-gap and convergence speed.

Additional Considerations:

The effect of `max_od_space` may depend on the traffic demand and network topology, as these determine how vehicles are distributed and the extent to which they compete for shared network resources. A one-off analysis is used to determine a representative demand level for each regime.

1.7. congestion_regimes

Three congestion regimes are considered: near-free-flow, moderate congestion, and high congestion.

The regimes are defined using congestion thresholds. For each regime, the representative demand is the first demand for which the mean (across seeds) congestion metric exceeds the corresponding threshold.

For each demand level, a congestion metric is calculated for each seed, and the mean is used to determine the first demand that exceeds each congestion threshold.

The congestion metric measures the average trip duration relative to free-flow conditions:

$$C = \frac{1}{N} \sum_{i=1}^N \frac{T_i}{T_i^{\text{free}}},$$

where T_i is the actual trip duration of agent i , T_i^{free} is its free-flow trip duration, and N is the total number of agents. The metric is dimensionless and indicates how much longer trips take compared with free-flow conditions. For example, $C = 2$ means that trips take, on average, twice as long as under free-flow conditions.

Additional Considerations:

The free-flow trip duration is obtained from the shortest path for each OD pair. The same routes are used in the congested simulations. Free-flow times are obtained from a simulation with almost no vehicles, rather than from link length and maximum speed, as this also accounts for acceleration and deceleration.

Results and Interpretation:

As shown in Figure 9, for the tested Sioux Falls network, the resulting demand values are 1300 vehicles for the near free-flow regime, 1550 vehicles for the moderate congestion regime, and 2000 vehicles for the high congestion regime. These values differ from demand levels commonly reported for the Sioux Falls network in the literature, as those studies often use macroscopic traffic simulators or link performance functions, whereas the present study uses a microscopic traffic simulator.

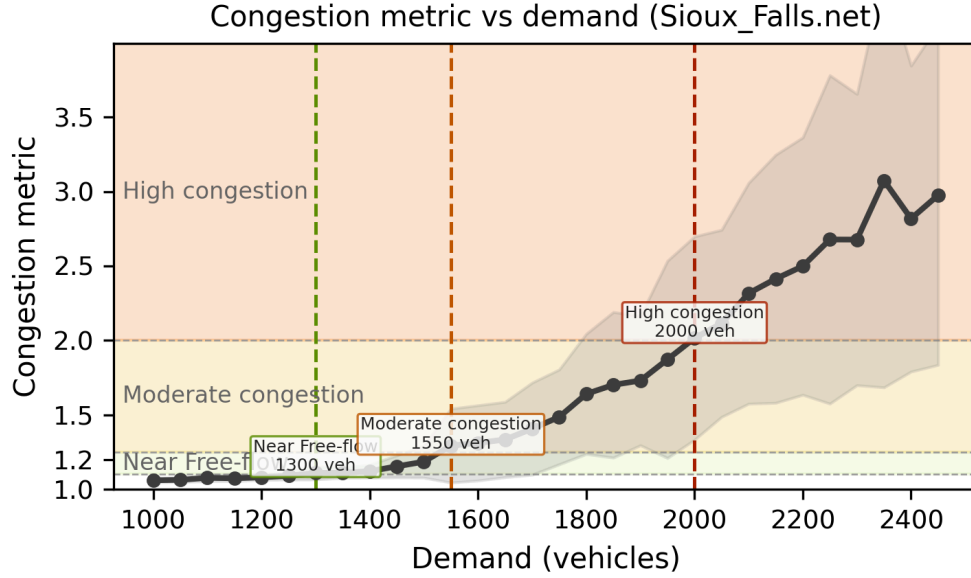


FIG. 9. Congestion metric for different demand levels, with the selected representative demands levels for each congestion regime highlighted.

1.8. fixed_time_min

The `fixed_time_min` hyperparameter determines the width of the time intervals (see Figure 18) used to aggregate link travel times. It is used during the R-gap computation, as described in Section 5.3.4.

Smaller intervals provide more accurate travel-time estimates because they capture changes in travel times over shorter periods, but result in more missing values in the average link travel time table because fewer vehicles are observed on each link within each interval.

Larger intervals reduce the amount of missing data and the need for imputation, but may result in less accurate travel-time estimates.

The sensitivity analysis evaluated the R-gap convergence curves for different values of `fixed_time_min`, together with the proportion of missing cells in the average link travel time table. The candidate intervals were explored progressively.

Results and Interpretation:

The sensitivity analysis presented in Figures 10–15 shows a clear effect of `fixed_time_min` on the R-gap. For large time intervals, the R-gap tends to take negative values, while decreasing the interval progressively shifts the R-gap towards positive values. Within the range of intervals considered, the benefit of more accurate travel-time estimates generally outweighs the drawback of additional missing data. However,

when the interval becomes excessively small, the amount of missing data starts to dominate.

The first experiment, presented in Figures 10–12, explores time intervals of 30 seconds (0.5 min), 1, 5, and 30.

The results indicate that the 30-second interval provides the best overall behavior, across all demand scenarios. After learning its R-gap curve remains close to zero and rarely takes values below zero.

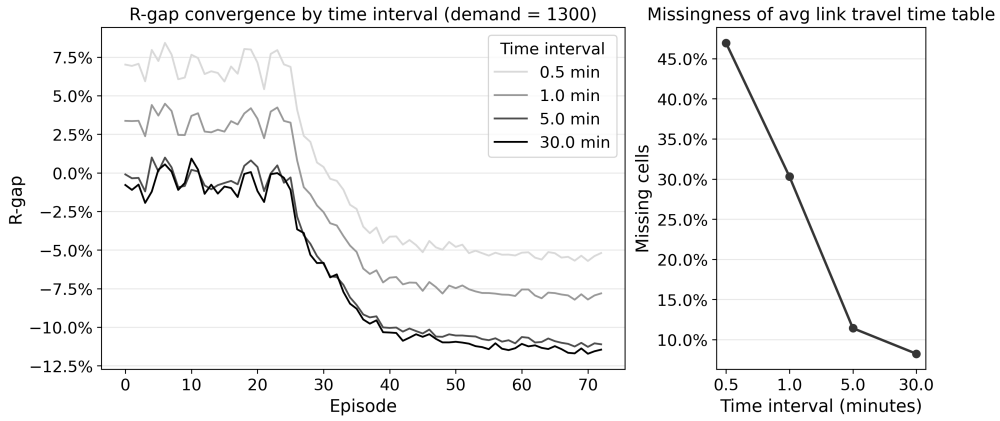


FIG. 10. Sensitivity of the R-gap to hyperparameter `fixed_time_min` over a broad range of time intervals for a demand of 1300.

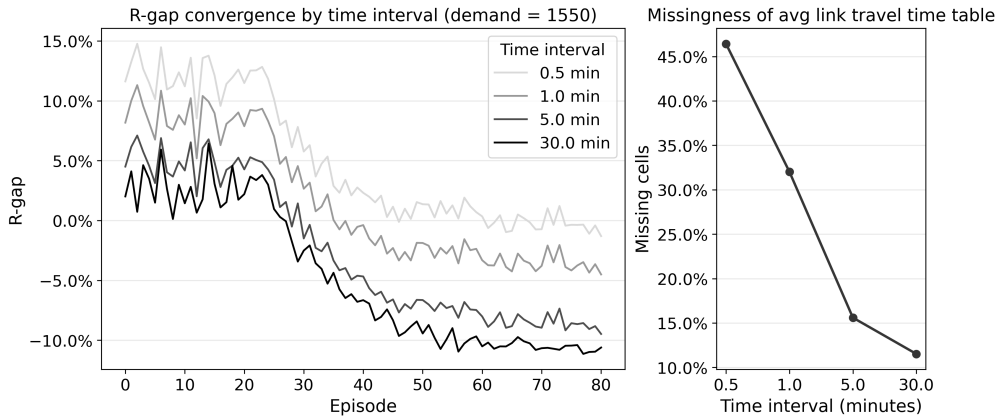


FIG. 11. Sensitivity of the R-gap to hyperparameter `fixed_time_min` over a broad range of time intervals for a demand of 1550.

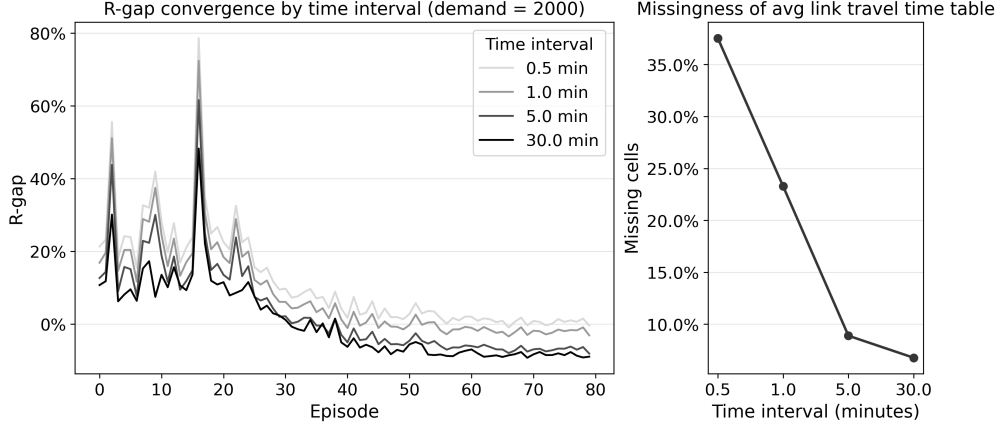


FIG. 12. Sensitivity of the R-gap to hyperparameter `fixed_time_min` over a broad range of time intervals for a demand of 2000.

The second experiment, presented in Figures 13–15 further refines the search by considering shorter intervals of 5, 10, 15, and 30 seconds. The results show that both the 15-second and 30-second intervals produce R-gap curves that after learning remain close to zero across all demand scenarios. The 30-second interval is preferred because it results in considerably fewer missing values than the 15-second interval. Moreover, its R-gap starts at relatively high values and decreases throughout learning, approaching zero while remaining mostly positive. It therefore provides a suitable trade-off between travel-time accuracy and data completeness. Therefore, a fixed time interval of 30 seconds (0.5 minutes) is selected.

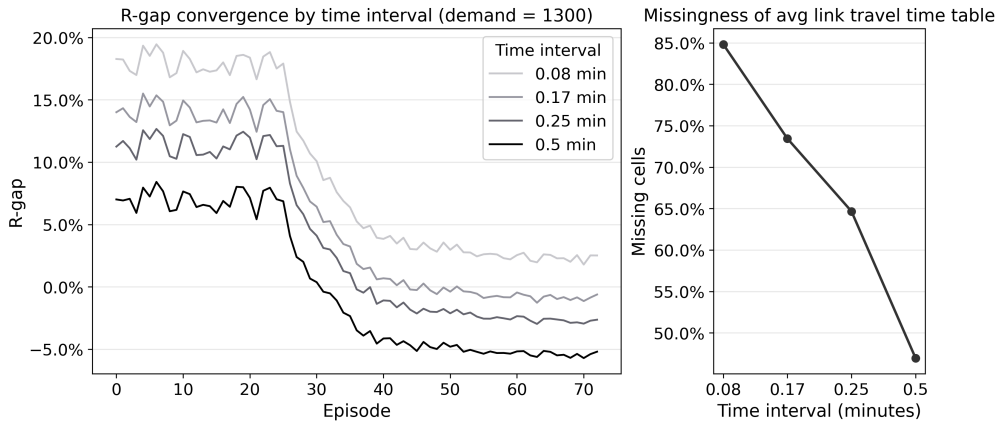


FIG. 13. Sensitivity of the R-gap to hyperparameter `fixed_time_min` for a range of short time intervals and a demand of 1300.

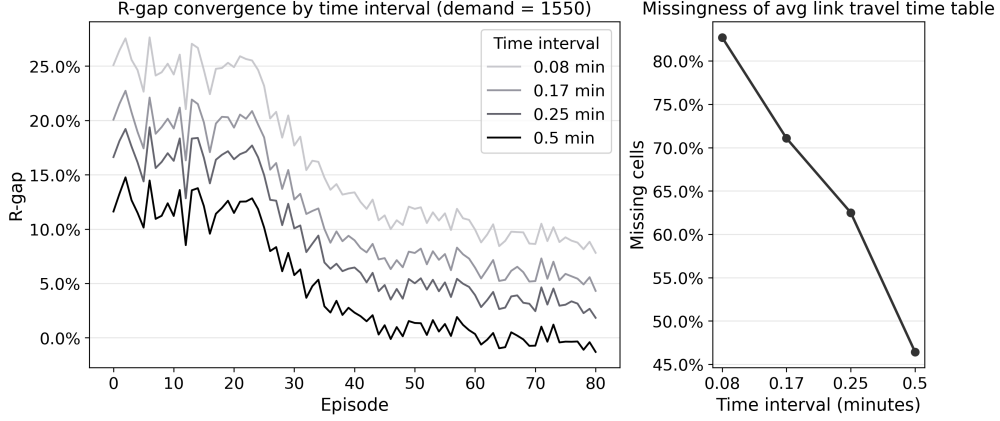


FIG. 14. Sensitivity of the R-gap to hyperparameter `fixed_time_min` for a range of short time intervals and a demand of 1550.

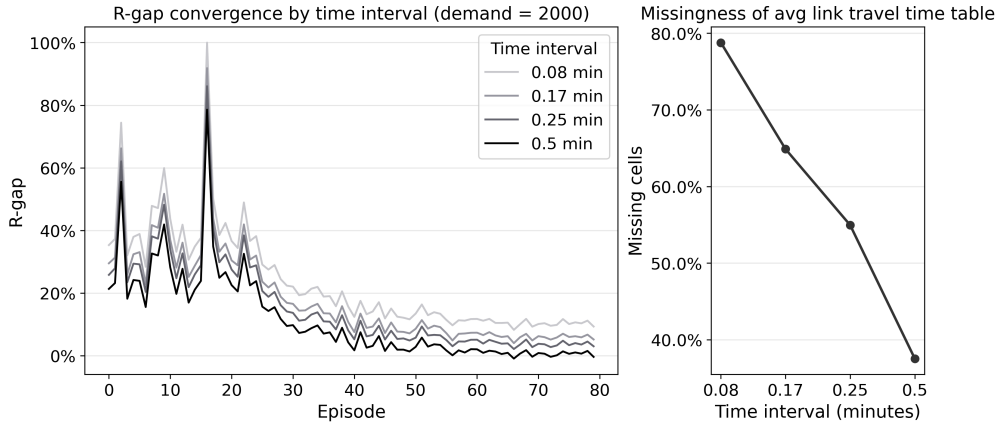


FIG. 15. Sensitivity of the R-gap to hyperparameter `fixed_time_min` for a range of short time intervals and a demand of 2000.

Additional Considerations:

The effect of `fixed_time_min` may depend on the network size and traffic demand. Larger networks contain more links and therefore more potentially missing cells, while higher demand results in denser observations and may reduce the amount of missing data.

2. Non-SA hyperparameters

Fixing the values of the non-SA hyperparameters did not require a sensitivity analysis. Their selected values are summarised in Table 1.

TABLE 1. Values of the Non-SA hyperparameters used in the experiments.

Component	Hyperparameter	Value
Numerical constants	<code>epsilon</code>	1.0×10^{-8}
Simulation	<code>simulation_time</code>	3600 s
Simulation	<code>maxtries</code>	1000
duarouter	<code>routing_algorithm</code>	CH
duarouter	<code>max_attempts</code>	25
duarouter	<code>n_threads</code>	7
Stopping rule	<code>max_episodes</code>	150
Stopping rule	<code>tolerance_stopping_rule</code>	0.01
Stopping rule	<code>k_no_change</code>	3
Random trips	<code>fringe_factor</code>	5
DUE convergence	<code>dualIterate_step_length</code>	0.1

The `simulation_time` is set to 3600 seconds, so each episode simulates one hour in addition to the warm-up period. The `max_episodes` value of 150 provides an upper limit on the number of learning episodes, ensuring that the simulation terminates if the MARL convergence criterion is not reached. The convergence of the MARL algorithm is assessed using the `tolerance_stopping_rule` and `k_no_change` parameters. When the mean policy update falls below `tolerance_stopping_rule`, agents are considered to stop learning. If this condition is met for `k_no_change` consecutive episodes, the MARL algorithm is considered to have converged.

The `fringe_factor` parameter controls the likelihood of selecting links at the boundaries of the network when generating trips with `randomTrips.py`. A value of 5 makes boundary links more likely to be selected as trip origins or destinations.

Appendix E

Additional Analyses

0.1. Heterogeneous Memory Levels

Research question: The original formulation of the proposed framework assumes a homogeneous population in which all agents share the same memory level. However, real drivers may differ in how strongly they rely on past travel experiences when making route choices. Introducing heterogeneous memory levels allows to analyze the impact of an heterogeneous populations with different memory levels on the learning dynamics.

Experimental setup: The experiment uses the baseline hyperparameter configuration described in Section D, with the learning rate fixed at 0.3. In the heterogeneous case, memory levels are sampled from a Beta distribution with a mean of 0.7, matching the memory level used in the homogeneous case, and a concentration parameter of 20. This results in approximately 95% of the memory levels falling within the range $[0.5, 0.9]$. The complete configuration file used for the experiment is provided in Appendix 5.

Results and Interpretation:

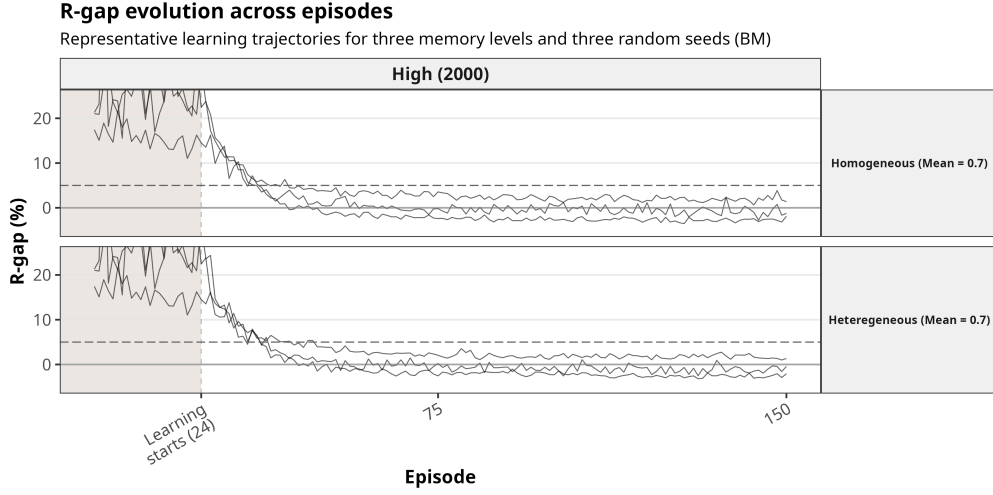


FIG. 1. R-gap evolution for populations with homogeneous or heterogeneous memory levels, both with a mean memory level of 0.7, across three random seeds.

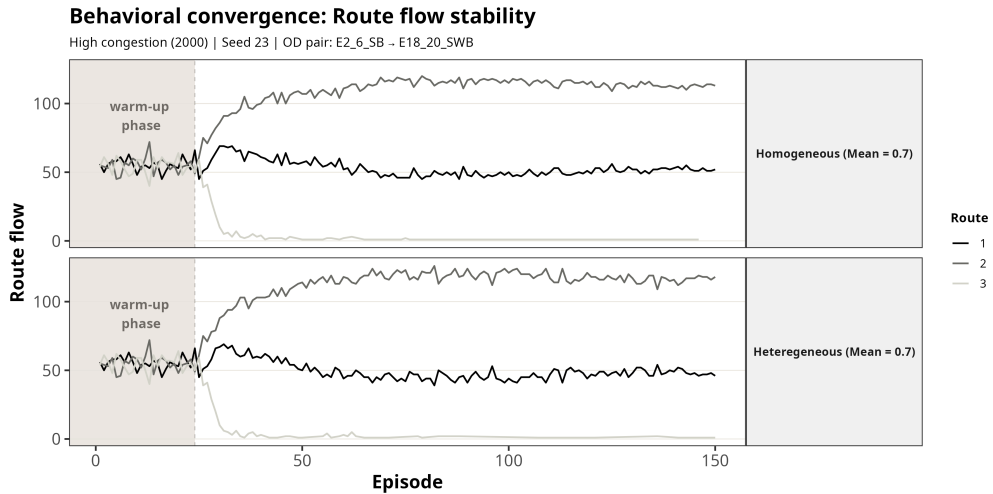


FIG. 2. Route-flow evolution for populations with homogeneous or heterogeneous memory levels, both with a mean memory level of 0.7, for one representative OD pair and random seed.

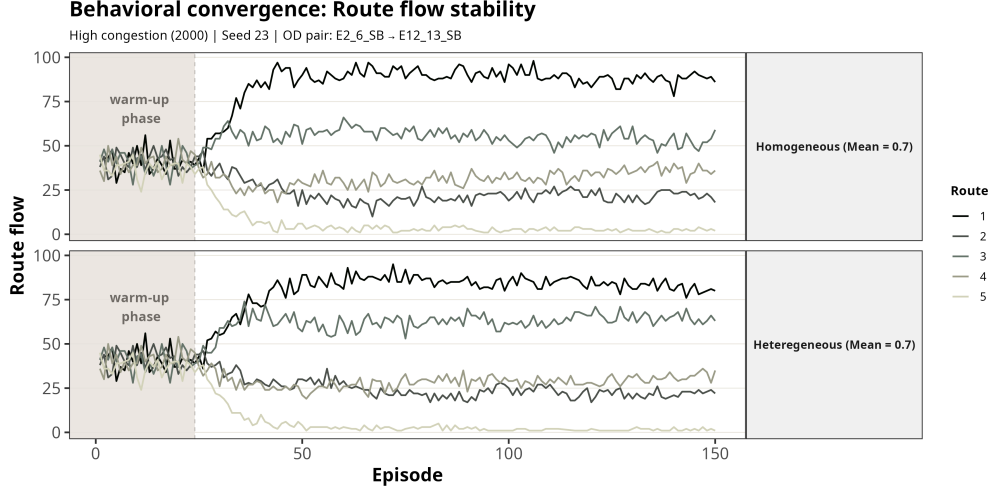


FIG. 3. Route-flow evolution for populations with homogeneous or heterogeneous memory levels, both with a mean memory level of 0.7, for a random seed and different representative origin–destination pair from that shown in Figure 2.

The memory sensitivity research question 5.4.3 established that the drivers' memory levels do not prevent DUE convergence under the tested scenario. Similar conclusions can be drawn for the heterogeneous case shown in Figure 1. The additional analysis therefore focuses on whether heterogeneity in memory levels affects the behavioral convergence of agents, particularly the stability of route flows.

The results show that heterogeneous memory levels do not substantially affect route-flow stability compared with a homogeneous population with the same mean memory level. For the OD pair shown in Figure 2, the heterogeneous population appears to exhibit somewhat greater route-flow variability, whereas for the OD pair shown in Figure 3, there is little difference between the homogeneous and heterogeneous populations. The lack of a consistent difference across OD pairs suggests that, under the tested conditions, the mean memory level of the population has a greater influence on route-flow stability than the heterogeneity in the population. This suggests that heterogeneous memory levels can be incorporated to represent differences in driver behavior without substantially affecting route-flow stability, provided that the population mean memory level remains unchanged.

0.2. Spatial Traffic Comparison

Figure 4 shows the qualitative comparison of link flows between BM and `duaIterate`.

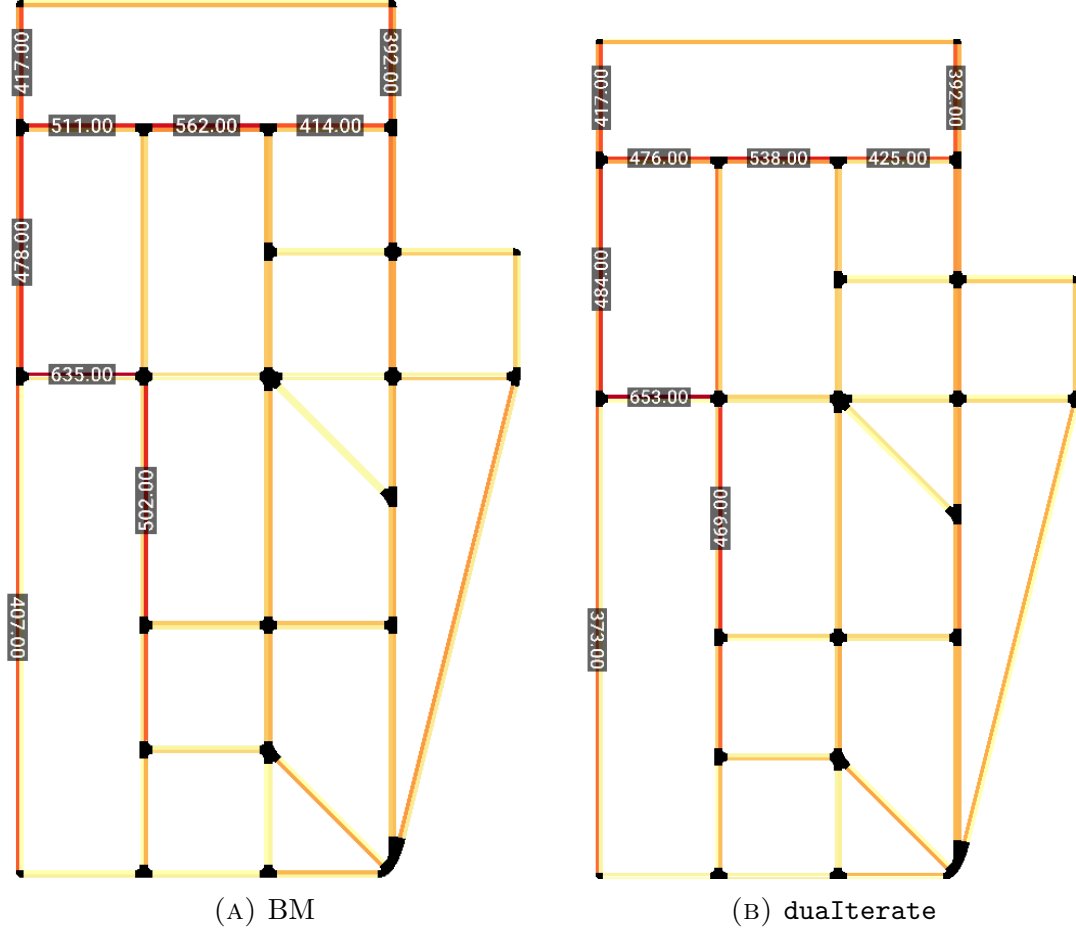


FIG. 4. Qualitative comparison of link flows between BM and `duaIterate` for the final network state.

The qualitative comparison shows that BM and `duaIterate` produce similar link flows across the network. Therefore, no separate quantitative link-flow comparison was considered necessary. If required, the similarity could be quantified using a metric such as RMSE.

Figures 5 and 6 show additional path-level comparisons.

Path-level route composition for a representative OD

Top 3 highest-flow paths per algorithm
 OD pair: E20_21_EB → E2_6_NB | Demand: 186 veh

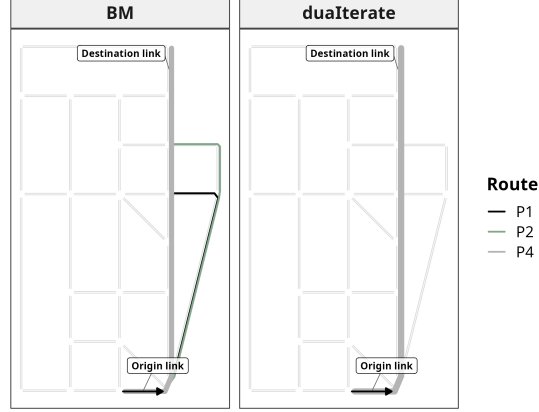


FIG. 5. Three highest-flow routes for a different representative origin–destination pair from that shown in Figure 23 under the proposed Bush–Mosteller MARL algorithm and the `duaIterate` tool, for one representative random seed.

TABLE 1. Percentage of traffic assigned to each route under the proposed Bush–Mosteller MARL algorithm and the `duaIterate` tool for the representative OD pair shown in Figure 5.

Path	BM share	duaIterate share
P4	76.9%	100.0%
P1	10.8%	0.0%
P2	12.4%	0.0%

Path-level route composition for a representative OD

Top 3 highest-flow paths per algorithm
 OD pair: E2_6_SB → E20_21_WB | Demand: 183 veh

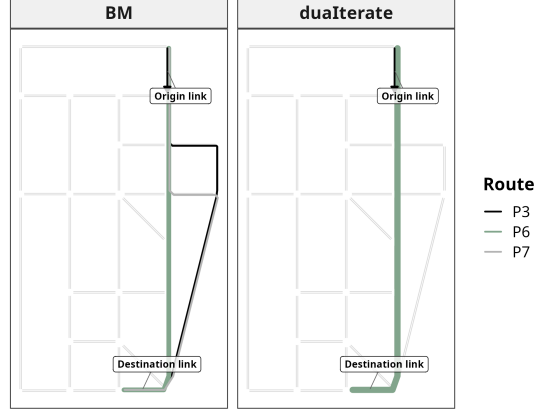


FIG. 6. Three highest-flow routes for a different representative origin–destination pair from that shown in Figure 23 and 5 under the proposed Bush–Mosteller MARL algorithm and the `duaIterate` tool, for one representative random seed.

TABLE 2. Percentage of traffic assigned to each route under the proposed Bush–Mosteller MARL algorithm and the `duaIterate` tool for the representative OD pair shown in Figure 6.

Path	BM share	duaIterate share
P6	69.9%	100.0%
P3	13.7%	0.0%
P7	15.3%	0.0%
P8	0.5%	0.0%
P4	0.5%	0.0%

Appendix F

SA hyperparameters Experimental Configuration

The complete YAML configuration used for the SA hyperparameter experiments is provided below.

LISTING F.1. YAML configuration used for SA hyperparameters analysis

```
configuration_name:
  config_name: production

randomness:
  seed: 42

numerical_constants:
  epsilon: 1.0e-8

learning:
  learning_rate: 0.3
  memory_level: 1

simulation:
  simulation_time: 3600
  warm_up_time: 300
  min_distance_factor: 0.8
  maxtries: 1000

network_path:
  # network: "/home/miguel/6.Projects/Thesis/sumo/net/Koh/1st.net.xml"
  network: "/home/miguel/6.Projects/Thesis/sumo/net/Popular/Sioux_Falls.
    net.xml"

duarouter:
  routing_algorithm: "CH"
  random_factor: 50
  max_attempts: 25
```

```
n_threads: 7
n_routes_per_OD: 8

stopping_rule:
  max_episodes: 150
  tolerance_stopping_rule: 0.01
  k_no_change: 3

demand:
  n_agents: 2000

interval_time:
  fixed_time_min: 0.5 # minutes

randomtrips:
  max_size_od_space: 10
  fringe_factor: 5

due_convergence:
  threshold_density: 40
  dualIterate_max_iterations: 1
  dualIterate_step_length: 0.1

mode_and_flags:
  last_episode_gui_BM: false
  last_episode_gui_dualIterate: false
```

Appendix G

Research Questions Experimental Configurations

This appendix provides the complete YAML configurations used for the experiments associated with the research questions presented in this thesis.

1. Rational Agents

The complete YAML configuration used for the Rational Agents experiments is provided below.

LISTING G.1. YAML configuration used for Rational Agents Research Question

```
configuration_name:
  config_name: production

randomness:
  seed:
    - 42
    - 5
    - 23
    - 1999
    - 8

numerical_constants:
  epsilon: 1.0e-8

learning:
  learning_rate: 0.3
  memory_level: 1

simulation:
  simulation_time: 3600
  warm_up_time: 300
  min_distance_factor: 0.8
```

```

    maxtries: 1000

network_path:
  network: "/home/miguel/6.Projects/Thesis/sumo/net/Popular/Sioux_Falls.
    net.xml"

duarouter:
  routing_algorithm: "CH"
  random_factor: 50
  max_attempts: 25
  n_threads: 7
  n_routes_per_OD: 8

stopping_rule:
  max_episodes: 150
  tolerance_stopping_rule: 0.01
  k_no_change: 3

demand:
  n_agents:
    - 1300
    - 1550
    - 2000

interval_time:
  fixed_time_min: 0.5 # minutes

randomtrips:
  max_size_od_space: 10
  fringe_factor: 5

due_convergence:
  threshold_density: 40
  dualIterate_max_iterations: 10
  dualIterate_step_length: 0.1

mode_and_flags:
  last_episode_gui_BM: false
  last_episode_gui_dualIterate: false

```

2. Spatial Traffic Comparison

The complete YAML configuration used for the Spatial Traffic Comparison experiments is provided below.

LISTING G.2. YAML configuration used for Spatial Traffic Comparison

```
configuration_name:
  config_name: production

randomness:
  seed: 42

numerical_constants:
  epsilon: 1.0e-8

learning:
  learning_rate: 0.3
  memory_level: 1

simulation:
  simulation_time: 3600
  warm_up_time: 300
  min_distance_factor: 0.8
  maxtries: 1000

network_path:
  network: "/home/miguel/6.Projects/Thesis/sumo/net/Popular/Sioux_Falls.
    net.xml"

duarouter:
  routing_algorithm: "CH"
  random_factor: 50
  max_attempts: 25
  n_threads: 7
  n_routes_per_OD: 8

stopping_rule:
  max_episodes: 150
  tolerance_stopping_rule: 0.01
  k_no_change: 3

demand:
  n_agents: 2000

interval_time:
  fixed_time_min: 0.5 # minutes

randomtrips:
  max_size_od_space: 10
  fringe_factor: 5

due_convergence:
  threshold_density: 40
  duaIterate_max_iterations: 10
```

```
    dualIterate_step_length: 0.1

mode_and_flags:
  last_episode_gui_BM: false
  last_episode_gui_dualIterate: false
```

3. Memory Sensitivity

The complete YAML configuration used for the Memory Sensitivity experiments is provided below.

LISTING G.3. YAML configuration used for Memory Sensitivity Research Question

```
configuration_name:
  config_name: production

randomness:
  seed:
    - 42
    - 5
    - 23
    - 8
    - 7

numerical_constants:
  epsilon: 1.0e-8

learning:
  learning_rate: 0.3
  memory_level:
    - 1.00
    - 0.90
    - 0.80
    - 0.70
    - 0.60
    - 0.50
    - 0.40
    - 0.30
    - 0.20
    - 0.10
    - 0.01

simulation:
  simulation_time: 3600
  warm_up_time: 300
  min_distance_factor: 0.8
  maxtries: 1000
```

```

network_path:
  network: "/home/miguel/6.Projects/Thesis/sumo/net/Popular/Sioux_Falls.
    net.xml"

duarouter:
  routing_algorithm: "CH"
  random_factor: 50
  max_attempts: 25
  n_threads: 7
  n_routes_per_OD: 8

stopping_rule:
  max_episodes: 150
  tolerance_stopping_rule: 0.01
  k_no_change: 3

demand:
  n_agents:
    - 1300
    - 1550
    - 2000

interval_time:
  fixed_time_min: 0.5 # minutes

randomtrips:
  max_size_od_space: 10
  fringe_factor: 5

due_convergence:
  threshold_density: 40
  duaIterate_max_iterations: 1
  duaIterate_step_length: 0.1

mode_and_flags:
  last_episode_gui_BM: false
  last_episode_gui_duaIterate: false

```

4. Disruption Recovery

The complete YAML configuration used for the Disruption Recovery experiments is provided below.

LISTING G.4. YAML configuration used for Disruption Recovery
Research Question

```

configuration_name:

```

```
    config_name: production

randomness:
  seed:
    - 42
    - 5
    - 23
    - 1999
    - 8

numerical_constants:
  epsilon: 1.0e-8

learning:
  learning_rate: 0.3
  memory_level:
    - 0.1
    - 0.25
    - 0.5
    - 0.75
    - 1

simulation:
  simulation_time: 3600
  warm_up_time: 300
  min_distance_factor: 0.8
  maxtries: 1000

network_path:
  network:
    - "/home/miguel/6.Projects/Thesis/experiments/rq4/net/Sioux_Falls.net.xml"
  network_normal:
    - "/home/miguel/6.Projects/Thesis/experiments/rq4/net/templates/Normal_Sioux_Falls.net.xml"
  network_degraded:
    - "/home/miguel/6.Projects/Thesis/experiments/rq4/net/templates/10km_h_Degraded_10-11_10-15_Sioux_Falls.net.xml"

duarouter:
  routing_algorithm: "CH"
  random_factor: 50
  max_attempts: 25
  n_threads: 7
  n_routes_per_OD: 8

stopping_rule:
  max_episodes: 450
```

```

tolerance_stopping_rule: 0.01
k_no_change: 5000
degradation_start_episode:
  - 151
degradation_end_episode:
  - 301

demand:
  n_agents: 2000

interval_time:
  fixed_time_min: 0.5 # minutes

randomtrips:
  max_size_od_space: 10
  fringe_factor: 5

due_convergence:
  threshold_density: 40
  duaIterate_max_iterations: 1
  duaIterate_step_length: 0.1

mode_and_flags:
  last_episode_gui_BM: false
  last_episode_gui_duaIterate: false

```

5. Heterogeneous Agents

The complete YAML configuration used for the Heterogeneous Agents experiments is provided below.

LISTING G.5. YAML configuration used for Heterogeneous Agents Research Question

```

configuration_name:
  config_name: production

randomness:
  seed:
    - 42
    - 5
    - 23
    - 1999
    - 8

numerical_constants:
  epsilon: 1.0e-8

```

```
learning:
  learning_rate: 0.3
  heterogeneous_memory:
    - true
    - false
  memory_mean:
    - 0.7
  memory_concentration:
    - 20

simulation:
  simulation_time: 3600
  warm_up_time: 300
  min_distance_factor: 0.8
  maxtries: 1000

network_path:
  network: "/home/miguel/6.Projects/Thesis/sumo/net/Popular/Sioux_Falls.
    net.xml"

duarouter:
  routing_algorithm: "CH"
  random_factor: 50
  max_attempts: 25
  n_threads: 7
  n_routes_per_OD: 8

stopping_rule:
  max_episodes: 150
  tolerance_stopping_rule: 0.01
  k_no_change: 3

demand:
  n_agents: 2000

interval_time:
  fixed_time_min: 0.5 # minutes

randomtrips:
  max_size_od_space: 10
  fringe_factor: 5

due_convergence:
  threshold_density: 40
  dualIterate_max_iterations: 10
  dualIterate_step_length: 0.1

mode_and_flags:
```

```
last_episode_gui_BM: false
last_episode_gui_dualIterate: false
```

6. Learning Rate Sensitivity

The complete YAML configuration used for the Learning Rate Sensitivity experiments is provided below.

LISTING G.6. YAML configuration used for Learning Rate Sensitivity

```
configuration_name:
  config_name: production

randomness:
  seed:
    - 42
    - 5
    - 23
    - 1999
    - 8

numerical_constants:
  epsilon: 1.0e-8

learning:
  learning_rate:
    - 0.99
    - 0.90
    - 0.80
    - 0.70
    - 0.60
    - 0.50
    - 0.40
    - 0.30
    - 0.20
    - 0.10
    - 0.01
  memory_level: 1

simulation:
  simulation_time: 3600
  warm_up_time: 300
  min_distance_factor: 0.8
  maxtries: 1000

network_path:
```

```

network: "/home/miguel/6.Projects/Thesis/sumo/net/Popular/Sioux_Falls.
  net.xml"

duarouter:
  routing_algorithm: "CH"
  random_factor: 50
  max_attempts: 25
  n_threads: 7
  n_routes_per_OD: 8

stopping_rule:
  max_episodes: 150
  tolerance_stopping_rule: 0.01
  k_no_change: 3

demand:
  n_agents:
    - 1300
    - 1550
    - 2000

interval_time:
  fixed_time_min: 0.5 # minutes

randomtrips:
  max_size_od_space: 10
  fringe_factor: 5

due_convergence:
  threshold_density: 40
  dualIterate_max_iterations: 1
  dualIterate_step_length: 0.1

mode_and_flags:
  last_episode_gui_BM: false
  last_episode_gui_dualIterate: false

```

7. Risk aversion

The complete YAML configuration used for the Risk Aversion experiments is provided below.

LISTING G.7. YAML configuration used for Risk Aversion

```

configuration_name:
  config_name: production

randomness:

```

```
seed:
  - 42
  - 5
  - 23
  - 1999
  - 8

numerical_constants:
  epsilon: 1.0e-8

learning:
  learning_rate: 0.3
  memory_level: 1
  reliability_sensitivity:
    - 0.00
    - 5.00
    - 10.00

simulation:
  simulation_time: 3600
  warm_up_time: 300
  min_distance_factor: 0.8
  maxtries: 1000

network_path:
  network: "/home/miguel/6.Projects/Thesis/sumo/net/Popular/Sioux_Falls.
    net.xml"

duarouter:
  routing_algorithm: "CH"
  random_factor: 50
  max_attempts: 25
  n_threads: 7
  n_routes_per_OD: 8

stopping_rule:
  max_episodes: 150
  tolerance_stopping_rule: 0.01
  k_no_change: 3

demand:
  n_agents: 2000

interval_time:
  fixed_time_min: 0.5 # minutes

randomtrips:
  max_size_od_space: 10
```

```
fringe_factor: 5

due_convergence:
  threshold_density: 40
  dualIterate_max_iterations: 1
  dualIterate_step_length: 0.1

mode_and_flags:
  last_episode_gui_BM: false
  last_episode_gui_dualIterate: false
```

8. Waiting-time aversion

The complete YAML configuration used for the Waiting-Time Aversion experiments is provided below.

LISTING G.8. YAML configuration used for Waiting-Time Aversion

```
configuration_name:
  config_name: production

randomness:
  seed:
    - 23
    - 42
    - 5
    - 1999
    - 8

numerical_constants:
  epsilon: 1.0e-8

learning:
  learning_rate: 0.3
  memory_level: 1
  waiting_time_sensitivity:
    - 0.00
    - 1.00
    - 5.00
    - 10.00

simulation:
  simulation_time: 3600
  warm_up_time: 300
  min_distance_factor: 0.8
  maxtries: 1000

network_path:
```

```

network: "/home/miguel/6.Projects/Thesis/sumo/net/Popular/Sioux_Falls.
        net.xml"

duarouter:
  routing_algorithm: "CH"
  random_factor: 50
  max_attempts: 25
  n_threads: 7
  n_routes_per_OD: 8

stopping_rule:
  max_episodes: 150
  tolerance_stopping_rule: 0.01
  k_no_change: 3

demand:
  n_agents: 2000

interval_time:
  fixed_time_min: 0.5 # minutes

randomtrips:
  max_size_od_space: 10
  fringe_factor: 5

due_convergence:
  threshold_density: 40
  dualIterate_max_iterations: 1
  dualIterate_step_length: 0.1

mode_and_flags:
  last_episode_gui_BM: false
  last_episode_gui_dualIterate: false

```

9. Nonlinear Response

The complete YAML configuration used for the Nonlinear Response experiments is provided below.

LISTING G.9. YAML configuration used for Nonlinear Response

```

configuration_name:
  config_name: production

randomness:
  seed:
    - 23
    - 42

```

```
- 5

numerical_constants:
  epsilon: 1.0e-8

learning:
  learning_rate: 0.3
  memory_level:
    - 1.00
    - 0.60
    - 0.10
  nonlinear_stimulus:
    - True
    - False
  stimulus_tau:
    - 0
    - 0.05
    - 0.20
    - 0.40

simulation:
  simulation_time: 3600
  warm_up_time: 300
  min_distance_factor: 0.8
  maxtries: 1000

network_path:
  network: "/home/miguel/6.Projects/Thesis/sumo/net/Popular/Sioux_Falls.
    net.xml"

duarouter:
  routing_algorithm: "CH"
  random_factor: 50
  max_attempts: 25
  n_threads: 7
  n_routes_per_OD: 8

stopping_rule:
  max_episodes: 150
  tolerance_stopping_rule: 0.01
  k_no_change: 5000

demand:
  n_agents: 2000

interval_time:
  fixed_time_min: 0.5 # minutes
```

```
randomtrips:
  max_size_od_space: 10
  fringe_factor: 5

due_convergence:
  threshold_density: 40
  dualIterate_max_iterations: 1
  dualIterate_step_length: 0.1

mode_and_flags:
  last_episode_gui_BM: false
  last_episode_gui_dualIterate: false
```